

LOW-RESOURCE INDIAN SIGN LANGUAGE VIDEO GENERATION VIA COMPRESSED MULTI-CONDITION TOKENISATION

*A Report submitted
in partial fulfillment for the Degree of*

Bachelor of Technology

in

Computer Engineering

by

HAMMAD ALI (22BCS025)

MOHAMMAD AHMADULLAH KHAN (22BCS030)



**Department of Computer Engineering
Faculty of Engineering & Technology
Jamia Millia Islamia, New Delhi - 110025
Session: 2022 - 2026**

**LOW-RESOURCE INDIAN SIGN LANGUAGE
VIDEO GENERATION
VIA COMPRESSED MULTI-CONDITION
TOKENISATION**

*A Report submitted
in partial fulfillment for the Degree of*

Bachelor of Technology

in

Computer Engineering

by

HAMMAD ALI (22BCS025)

MOHAMMAD AHMADULLAH KHAN (22BCS030)



**Department of Computer Engineering
Faculty of Engineering & Technology
Jamia Millia Islamia, New Delhi - 110025
Session: 2022 - 2026**

CERTIFICATE

This is to certify that the project report entitled "**Low-Resource Indian Sign Language Video Generation via Compressed Multi-Condition Tokenisation**" submitted by Hammad Ali (Roll No. 22BCS025) and Mohammad Ahmadullah Khan (Roll No. 22BCS030) to the Department of Computer Engineering, Faculty of Engineering and Technology, Jamia Millia Islamia, New Delhi - 110025, in partial fulfillment for the award of the degree of B. Tech in (Computer Engineering) is a bona fide record of project work carried out by them under my supervision. The contents of this report, in full or in part, have not been submitted to any other Institution or University for the award of any degree.

Mr. Shamim Ahmad

Faculty Supervisor

Department of Computer Engineering

Jamia Millia Islamia

Prof. (Dr.) Mohammad Amjad

Head of the Department

Department of Computer Engineering

Jamia Millia Islamia

New Delhi

(May, 2026)

DECLARATION

We declare that this project report titled "**Low-Resource Indian Sign Language Video Generation via Compressed Multi-Condition Tokenisation**" submitted in partial fulfillment of the degree of Bachelor of Technology in Computer Engineering is a record of original work carried out by us under the supervision of Mr. Shamim Ahmad, and has not formed the basis for the award of any other degree, in this or any other Institution or University. In keeping with the ethical practice in reporting scientific information, due acknowledgements have been made wherever the findings of others have been cited.

Hammad Ali

Roll No. 22BCS025

Mohammad Ahmadullah Khan

Roll No. 22BCS030

New Delhi - 110025

Date: _____

ACKNOWLEDGEMENTS

We extend our heartfelt gratitude to Mr. Shamim Ahmad, Faculty Supervisor, Department of Computer Engineering, Jamia Millia Islamia, New Delhi, for his unwavering support, expert guidance, and continuous encouragement throughout this research endeavour. His insightful suggestions during the initial project framing, particularly on the choice of low-resource sign language as the problem domain and on the discrete-tokenisation paradigm, were instrumental in shaping the technical direction of this work. His patient mentorship through the multiple iteration cycles, including the major architectural pivot from monolithic text-to-video generation to a two-stage text-to-skeleton-to-video pipeline, kept the project grounded in a realistic scope.

We sincerely thank Prof. (Dr.) Mohammad Amjad, Head of the Department of Computer Engineering, for fostering an academic environment that supports ambitious final-year project work, and for ensuring that the necessary departmental resources and approvals were available throughout the year.

We are deeply indebted to the entire faculty of the Department of Computer Engineering at Jamia Millia Islamia for the foundational coursework that prepared us for a project of this scope. Coursework in Machine Learning, Deep Learning, Linear Algebra, and Probability formed the theoretical backbone of this work; courses in Computer Vision and Pattern Recognition equipped us to reason about pose estimation and image generation; and Software Engineering coursework informed our approach to versioned, ablation-driven experimentation.

Substantial computational resources for this project were provided by Kaggle, specifically the free-tier sessions with two NVIDIA T4 GPUs on which all training, inference, and evaluation runs were conducted. Without this freely accessible compute, the iterative versioned development that drove every architectural decision in this report would not have been possible. We also acknowledge HuggingFace for hosting the pretrained models that anchor Stage B of our pipeline (Stable Diffusion 1.5, ControlNet-OpenPose, and IP-Adapter), and

the open-source maintainers of PyTorch, the Diffusers library, rtmllib, and the broader Python machine-learning ecosystem.

We thank the original creators of the ISL-CSLTR (Indian Sign Language Continuous Sign Language Recognition and Translation) dataset, hosted on Kaggle, whose contribution makes computational ISL research at the academic level feasible in the first place. We are particularly grateful that the dataset includes multi-signer recordings of the same sentences, a property our pipeline relies upon for its sentence-disjoint train, validation, and test split.

Finally, we thank our classmates and friends who acted as informal sounding boards during the long debugging sessions, particularly during the pivot to the two-stage decomposition, when discussions on whether to persist with monolithic text-to-video generation or to decompose the problem clarified our thinking. Their constructive criticism of the early ablation results sharpened the framing of the body-part FSQ contribution.

Any unintentional omission of names is regretted; the contributions of all involved are deeply appreciated and acknowledged.

Hammad Ali (22BCS025)
Mohammad Ahmadullah Khan (22BCS030)

ABSTRACT

Sign Language Video Generation is a vital technology for bridging the communication gap between hearing and Deaf communities. State-of-the-art systems, including SignGAN, diffusion-based methods, and the recently published SignViP framework, predominantly rely on large-scale, high-resource datasets such as RWTH-PHOENIX-Weather (approximately 8,257 sentences for German Sign Language). No comparable resource exists for Indian Sign Language. The largest publicly available Indian Sign Language corpus, ISL-CSLTR, contains only 100 sentences across 7 signers, of which 471 video samples covering 97 unique sentences survive resolution-based quality filtering. This roughly 85-fold gap in corpus size makes off-the-shelf adaptation of high-resource architectures impractical, and motivates the present work.

This project proposes a novel adaptation of the SignViP framework for the low-resource Indian Sign Language setting, organised as a two-stage pipeline. The first stage maps an input English sentence to a 16-frame skeleton sequence using a frozen CLIP text encoder, a 1.5-million-parameter autoregressive token translator, and a body-part Finite Scalar Quantisation tokeniser with three separate codebooks for the upper-body pose, left hand, and right hand. The second stage renders the predicted skeleton sequence into a video using frozen pretrained components, namely Stable Diffusion 1.5, ControlNet-OpenPose, and IP-Adapter, conditioned on a single reference image of the target signer.

The core methodological contribution is a hyper-compact body-part Finite Scalar Quantisation design tailored to the low-resource regime. Three findings emerged from extensive ablation studies. First, the original SignViP 625-code design collapses on ISL-CSLTR with only 1 out of 625 codes used, while a 4-axis 81-code per-part codebook achieves 100 percent utilisation; we establish empirically that a 3-axis Finite Scalar Quantisation collapses and that 4 axes is the minimum required at this data scale. Second, body-part decomposition (separate codebooks per body part) outperforms the unified-codebook design by 37 percent in reconstruction Mean Squared Error (from 0.473 to 0.296) and provides three times more tokens per clip (from 16 to 48 tokens). Third, MediaPipe Holistic, which

dominates published Indian Sign Language pipelines, fails catastrophically on seated signers, achieving only 30.9 percent left-hand and 57.4 percent right-hand detection; replacing it with DWPose (rtmlib) raises detection rates to 96.4 percent and 99.7 percent respectively, a finding that is independently useful for any future Indian Sign Language research.

On the sentence-disjoint test split (51 samples across 5 high-resolution signers, with sentences unseen during training), Stage A achieves 53.8 percent token accuracy (compared to a 51 percent constant-token baseline, and 45 times above the 1.2 percent random baseline) and 0.250 Mean Per-Joint Position Error in normalised image coordinates. The Finite Scalar Quantisation ceiling Mean Per-Joint Position Error is 0.074, implying that 71 percent of total Stage A error is attributable to translator hedging rather than codebook compression. The full text-to-video pipeline produces a Structural Similarity Index of 0.415 plus or minus 0.105, a Learned Perceptual Image Patch Similarity of 0.601 plus or minus 0.096, a Peak Signal-to-Noise Ratio of 9.81 plus or minus 1.31 decibels, and a mean CLIP image-cosine similarity of 0.795, well above the 0.7 "recognisable identity" threshold. The ground-truth-conditioned ceiling, which uses ground-truth skeletons as input to Stage B and isolates Stage B rendering quality, is 0.516 Structural Similarity Index. The 0.10 Structural Similarity Index gap between the two conditions is the real cost of motion prediction from text alone.

We compare honestly with prior published Indian Sign Language skeleton-to-video work, namely Dynamic GAN, H-DNA, and GAN-CNN, which report Structural Similarity Index values in the range 0.92 to 0.99. These figures are not directly comparable because the prior work uses ground-truth skeletons as input, and is dataset-specifically trained from scratch on ISL-CSLTR frames, whereas our pipeline solves the strictly harder text-to-video problem and uses a generic pretrained Stable Diffusion 1.5 with no dataset-specific fine-tuning. To our knowledge, this work establishes the first text-to-skeleton-to-video baseline on ISL-CSLTR. Limitations are stated openly: 471 samples and 97 unique sentences fundamentally cap text-to-motion learning; the right hand, carrying the most semantic variation across signs, is the predictability bottleneck at 39.5 percent token accuracy and 0.394 Mean Per-Joint Position Error; and Stage B uses no dataset-

specific fine-tuning, capping appearance fidelity at the generic Stable Diffusion 1.5 prior. Future work directions include LoRA fine-tuning of Stage B (expected to lift ground-truth-conditioned Structural Similarity Index to the range 0.65 to 0.75), cumulative-sum drift mitigation via re-anchoring at frame 8, cross-lingual transfer using larger Indian Sign Language corpora (such as ISLTranslate and iSign), and a switch to purpose-built human animation backbones.

TABLE OF CONTENTS

CERTIFICATE.....	ii
DECLARATION	iii
ACKNOWLEDGEMENTS.....	iv
ABSTRACT.....	vi
TABLE OF CONTENTS.....	ix
LIST OF FIGURES	xvi
LIST OF TABLES.....	xviii
ABBREVIATIONS	xix
CHAPTER 1	1
1.1. Background and Motivation	1
1.2. Problem Statement.....	2
1.3. Objectives of the Project.....	3
1.4. Overview of Sign Language Video Generation.....	5
1.4.1. What is sign language video generation?	5
1.4.2. Indian sign language: a low-resource language.....	6
1.4.3. Real-world impact.....	6
1.5. Existing Approaches to Sign Language Video Generation	7
1.5.1. GAN-based sign generation.....	7
1.5.2. Diffusion-based end-to-end generators.....	7
1.5.3. Skeleton-intermediate two-stage pipelines	7
1.6. Challenges and Limitations of the Existing Landscape.....	8
1.6.1. Data scarcity for ISL.....	8
1.6.2. Representation inefficiency in continuous motion	8
1.6.3. Mode collapse and overfitting in generative training	8
1.6.4. Hardware constraints on Kaggle.....	9
1.7. Recent Advances in Related Domains.....	9
1.7.1. Latent diffusion models (Stable Diffusion)	9
1.7.2. ControlNet for conditional generation.....	9
1.7.3. Vector and scalar quantisation.....	9
1.7.4. Image-prompt adapters (IP-Adapter).....	10
1.8. Fundamentals of Latent Diffusion Models	10
1.8.1. Forward and reverse diffusion processes.....	10
1.8.2. DDPM and DDIM sampling.....	11

1.8.3. Classifier-free guidance	11
1.9. Fundamentals of Discrete Tokenisation	12
1.9.1. Vector Quantisation (VQ-VAE)	12
1.9.2. Finite Scalar Quantisation (FSQ).....	12
1.9.3. Codebook utilisation and collapse	13
1.9.4. Why discrete tokens for low-resource generation	14
1.10. Fundamentals of Pose Estimation for Sign Language	14
1.10.1. OpenPose, MediaPipe, and DWPose	14
1.10.2. COCO-Wholebody keypoint format.....	15
1.10.3. The seated-signer constraint	15
1.11. Contributions of this Work	15
1.11.1. Methodological contributions	15
1.11.2. Empirical findings.....	16
1.11.3. Engineering contributions.....	17
1.12. Organisation of the Report.....	17
CHAPTER 2	18
2.1. SignViP: Sign-Video Compressed Quantised Tokenisation (Wang et al., 2025)	18
2.1.1. Introduction.....	18
2.1.2. Proposed architecture.....	18
2.1.3. Key findings.....	19
2.1.4. Limitations on low-resource settings	19
2.2. Animate Anyone: Consistent Image-to-Video Animation (Hu et al., 2024)	20
2.2.1. Introduction.....	20
2.2.2. Reference image conditioning	20
2.2.3. Key findings.....	20
2.3. Pose-Guided Fine-Grained Sign Language Video Generation (Shi et al., 2024)	20
2.3.1. Introduction.....	20
2.3.2. Method	21
2.3.3. Key findings.....	21
2.4. Signing at Scale: Co-Articulating Signs (Saunders et al., 2022)	21
2.5. Sign Language Transformers (Camgoz et al., 2020)	22
2.6. Dynamic GAN for ISL (Elakkiya et al., 2022).....	22
2.7. H-DNA: Hybrid Deep Network Architecture for ISL (Elakkiya et al., 2022)	23

2.8. GAN-CNN Hybrid for ISL (Waghmare et al., 2024).....	23
2.9. Finite Scalar Quantisation (Mentzer et al., 2024).....	24
2.10. IP-Adapter for Diffusion Models (Ye et al., 2023).....	24
2.11. ControlNet (Zhang et al., 2023).....	25
2.12. Comparative Literature Summary	25
2.13. Identified Research Gap.....	27
2.13.1. Gap 1: no published text-to-video baseline on Indian Sign Language	27
2.13.2. Gap 2: no low-resource study of the discrete tokenisation paradigm	27
2.13.3. Gap 3: no empirical comparison of MediaPipe and DWPose on seated signers	28
2.14. Our Approach in the Context of the Reviewed Work	28
CHAPTER 3	30
3.1. Tools, Libraries, and Hardware	30
3.1.1. Hardware: Kaggle compute tier.....	30
3.1.2. Software stack.....	31
3.1.3. Resilience strategies for Kaggle	31
3.2. Dataset: ISL-CSLTR.....	32
3.2.1. Source and structure.....	32
3.2.2. Filtering decisions.....	33
3.2.3. The seated-signer observation	36
3.3. Overall Pipeline Architecture	37
3.4. Stage A: Text to Skeleton	38
3.4.1. Pose estimation: DWPose	39
3.4.2. Keypoint format and filtering	41
3.4.3. Per-frame normalisation	42
3.4.4. Delta encoding	42
3.4.5. Body-part FSQ tokeniser (core novelty).....	43
3.4.6. CLIP text encoding	45
3.4.7. Token translator	45
3.4.8. Training recipe and loss.....	46
3.4.9. Stage A inference.....	46
3.5. Stage B: Skeleton and Reference to Video	47
3.5.1. Backbone selection	48
3.5.2. Stable Diffusion plus ControlNet plus IP-Adapter integration.....	48
3.5.3. Canonical OpenPose rendering.....	48

3.5.4. Coordinate-space recovery	49
3.5.5. IP-Adapter identity conditioning	49
3.5.6. Generation parameters and prompt.....	50
3.5.7. Temporal coherence strategies.....	50
3.6. Evaluation Methodology.....	51
3.6.1. Test set	51
3.6.2. Two evaluation conditions.....	51
3.6.3. Metrics	52
3.6.4. Per-signer breakdown	53
3.6.5. Stage A diagnostic decomposition.....	53
CHAPTER 4	54
4.1. Experimental Methodology	54
4.1.1. Iterative versioned development.....	54
4.1.2. Ablation approach.....	54
4.2. MVP Smoke Test.....	55
4.3. Research-Grade From-Scratch Baseline	55
4.3.1. Goal and architecture	55
4.3.2. First codebook collapse	56
4.3.3. The FSQ ablation: central experiment of the research-grade baseline	56
4.3.4. The 3-axis versus 4-axis FSQ discovery.....	57
4.3.5. Translator collapse and final metrics for the research-grade baseline.	57
4.4. Inference-Fixed Baseline	58
4.4.1. The seven inference bugs.....	58
4.4.2. The <code>isinstance</code> failure.....	58
4.4.3. First working inference	59
4.5. Augmentation-Enhanced Pipeline	59
4.6. Spatial Reference Concatenation Design.....	60
4.7. Pretrained-Backbone LoRA Pipeline.....	60
4.7.1. Architectural shift	60
4.7.2. Five issues encountered	61
4.7.3. Pose renderer fix during the pretrained backbone iteration	61
4.7.4. Pretrained backbone first photorealistic output	62
4.7.5. The 1280-dimensional FSQ collapse	62
4.7.6. Trigger for the two-stage pivot	62
4.8. Two-Stage Decomposed Pipeline, Stage A	63

4.8.1. Decomposition into two stages	63
4.8.2. DWPose versus MediaPipe: the empirical comparison	63
4.8.3. Body-part FSQ ablation	63
4.8.4. Translator sizing study	65
4.8.5. Parallel scheduled sampling optimisation	66
4.8.6. Final Stage A results	67
4.9. Initial Stage B Implementation	69
4.9.1. Six issues identified	69
4.9.2. Critical observation from the initial Stage B	70
4.10. Final Stage B with Coordinate-Space Recovery	71
4.10.1. Six fixes applied	71
4.10.2. Coordinate-space bug discovery	71
4.10.3. Final Stage B results	72
4.11. Closed Translator Improvement Experiments	73
4.11.1. Motivation	73
4.11.2. Proposed improvements	73
4.11.3. Two retraining attempts	74
4.11.4. Diagnosis	74
4.11.5. Positive findings from the negative result	74
4.12. Catalogue of Errors and Their Fixes	75
4.12.1. Diffusion and inference path errors	75
4.12.2. Notebook environment errors	76
4.12.3. HuggingFace download errors	76
4.12.4. Memory and RAM errors	77
4.12.5. FSQ-specific errors	77
4.12.6. Stage B IP-Adapter and ControlNet errors	77
4.12.7. Pose extraction and translator errors	78
4.13. Comprehensive Summary of All Experiments	79
CHAPTER 5	81
5.1. Headline Numbers	81
5.2. Stage A: Text-to-Skeleton Results	82
5.2.1. Translator training dynamics	82
5.2.2. Token accuracy breakdown	83
5.2.3. MPJPE decomposition	84
5.2.4. Compounded FSQ improvement across iterations	85
5.3. Stage B: Skeleton-to-Video Results	85

5.3.1. GT-conditioned versus Full Pipeline	85
5.3.2. Metric distributions.....	86
5.4. Per-Signer Breakdown.....	87
5.5. Temporal Coherence and Identity Preservation.....	88
5.5.1. Per-frame quality	88
5.5.2. Identity preservation	89
5.6. End-to-End Pipeline Decomposition	90
5.7. Comparison with Prior Indian Sign Language Work	91
5.7.1. Why the comparison is fair but not direct.....	91
5.7.2. What we contribute that existing work does not.....	92
5.8. Realistic Targets versus Achieved Numbers	93
5.9. Qualitative Visualisations	94
5.9.1. Per-signer comparison grids	94
5.9.2. Metric distribution box plots.....	97
CHAPTER 6	99
6.1. Summary of Findings.....	99
6.2. Contributions to the Field	100
6.2.1. Methodological contributions	100
6.2.2. Empirical findings.....	101
6.2.3. Engineering contributions	102
6.3. Honest Limitations.....	102
6.3.1. Computational resource constraints.....	102
6.3.2. Dataset size constraints	103
6.3.3. Generation quality constraints	103
6.3.4. Comparability constraints	104
6.3.5. Negative result	104
6.4. Future Scope	104
6.4.1. Translator improvements	104
6.4.2. Stage B improvements (LoRA fine-tuning).....	105
6.4.3. Cross-lingual and cross-dataset transfer	106
6.4.4. Backbone alternatives for Stage B.....	106
6.4.5. Real-world deployment considerations	107
6.4.6. Ethical considerations	107
6.5. Closing Summary	108
REFERENCES	109
APPENDIX A.....	112

A.1. Body-Part FSQ Tokeniser Training	112
A.2. Token Translator Training	113
A.3. Stage B Inference Parameters	113
APPENDIX B	115
B.1. Per-Signer Ground-Truth-Conditioned Metrics	115
B.2. Per-Signer Full-Pipeline Metrics	115
B.3. Per-Body-Part FSQ Codebook Statistics	116
B.4. Stage A Test-Set Per-Sample Statistics	116

LIST OF FIGURES

Figure	Title	Page
Figure 1.1:	End-to-end inference pipeline overview	5
Figure 1.2:	Forward and reverse diffusion process schematic.....	11
Figure 1.3:	Finite Scalar Quantisation discretisation lattice	13
Figure 3.1:	Sample count per signer in the ISL-CSLTR corpus.....	32
Figure 3.2:	Frames per clip distribution across the ISL-CSLTR dataset.....	33
Figure 3.3:	Pixel-count resolution comparison across signers	34
Figure 3.4:	Signer coverage per sentence in ISL-CSLTR	34
Figure 3.5:	Word count distribution for English text and ISL glosses	35
Figure 3.6:	Top-20 tokens for English text and ISL gloss.....	35
Figure 3.7:	Train, validation, and test split breakdown	36
Figure 3.8:	Signer appearance grid across the 7 signers	36
Figure 3.9:	Overall pipeline architecture: text to skeleton to video.....	37
Figure 3.10:	Data preprocessing pipeline for Stage A.....	38
Figure 3.11:	DWPose-extracted skeleton across 16 frames for Signer 3	40
Figure 3.12:	DWPose-extracted skeleton across 16 frames for Signer 4	40
Figure 3.13:	DWPose-extracted skeleton across 16 frames for Signer 5	41
Figure 3.14:	Body-part Finite Scalar Quantisation tokeniser architecture	44
Figure 3.15:	Token translator architecture.....	46
Figure 3.16:	Stage B detailed architecture.....	47
Figure 3.17:	Evaluation methodology: two-condition decomposition	52
Figure 4.1:	FSQ ablation: reconstruction, utilisation, and entropy.....	64
Figure 4.2:	FSQ qualitative reconstruction (configuration A).....	65
Figure 4.3:	FSQ qualitative reconstruction (configuration B).....	65
Figure 4.4:	FSQ qualitative reconstruction (configuration C).....	65
Figure 4.5:	Token translator training curves.....	67
Figure 4.6:	End-to-end inference grid, test sample 1.....	68
Figure 4.7:	End-to-end inference grid, test sample 2.....	68
Figure 4.8:	End-to-end inference grid, test sample 3.....	68
Figure 4.9:	End-to-end inference grid, test sample 4.....	68
Figure 4.10:	End-to-end inference grid, test sample 5.....	69
Figure 4.11:	Skeleton format ablation (legacy versus canonical OpenPose)	70
Figure 4.12:	Predicted skeleton sanity check after coordinate recovery	72
Figure 4.13:	IP-Adapter A/B test for identity preservation	72
Figure 5.1:	Per-signer metric breakdown across SSIM, LPIPS, and PSNR.....	88
Figure 5.2:	Temporal quality per-frame curves	89
Figure 5.3:	Identity preservation: CLIP image-cosine similarity	90

Figure 5.4: Comparison grid for test sample, Signer 3	95
Figure 5.5: Comparison grid for test sample, Signer 4	96
Figure 5.6: Comparison grid for test sample, Signer 5	96
Figure 5.7: Comparison grid for test sample, Signer 6	97
Figure 5.8: Metric distribution boxplots across the 51 test samples.....	97

LIST OF TABLES

Table	Title	Page
Table 2.1:	Comparative literature summary of sign language video generation methods.....	26
Table 3.1:	Computational resources used during the project.....	30
Table 3.2:	Software stack used in the implementation	31
Table 3.3:	Filtering decisions and resulting dataset size	33
Table 3.4:	Detection rates for MediaPipe Holistic versus DWPose	39
Table 3.5:	Keypoint format used in Stage A	41
Table 3.6:	Final body-part FSQ configuration.....	43
Table 3.7:	Stage B generation parameters	50
Table 3.8:	Evaluation metrics and their interpretation	52
Table 4.1:	FSQ ablation summary across configurations	56
Table 4.2:	Single FSQ versus body-part FSQ comparison	64
Table 4.3:	Translator sizing study.....	66
Table 4.4:	Cumulative metrics across pipeline versions.....	79
Table 5.1:	Headline numbers (final, locked metrics).....	81
Table 5.2:	Token accuracy breakdown by body part.....	83
Table 5.3:	MPJPE decomposition for Stage A	84
Table 5.4:	Stage B ground-truth-conditioned metrics	85
Table 5.5:	Stage B full-pipeline metrics	86
Table 5.6:	Per-signer breakdown for the full pipeline	87
Table 5.7:	Comparison with prior published Indian Sign Language work.....	91
Table A.1:	Body-part FSQ tokeniser training hyperparameters	113
Table A.2:	Token translator training hyperparameters	114
Table A.3:	Stage B inference parameters	114
Table B.1:	Per-signer ground-truth-conditioned metrics on the 51 test samples.	116
Table B.2:	Per-signer Full-Pipeline metrics on the 51 test samples	117
Table B.3:	Per-body-part FSQ codebook statistics at convergence.....	117
Table B.4:	Stage A test-set per-sample statistics	117

ABBREVIATIONS

The abbreviations used throughout this report are listed below in alphabetical order.

Abbreviation	Full Form
ASL	American Sign Language
BSL	British Sign Language
CFG	Classifier-Free Guidance
CLIP	Contrastive Language-Image Pretraining
COCO	Common Objects in Context
DDIM	Denoising Diffusion Implicit Models
DDPM	Denoising Diffusion Probabilistic Models
DWPose	Distillation Whole-body Pose Estimation
EMA	Exponential Moving Average
FID	Frechet Inception Distance
FSQ	Finite Scalar Quantisation
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
GT	Ground Truth
ISL	Indian Sign Language
ISL-CSLTR	Indian Sign Language Continuous Sign Language Recognition and Translation Dataset
IP-Adapter	Image-Prompt Adapter
LoRA	Low-Rank Adaptation
LPIPS	Learned Perceptual Image Patch Similarity
MLP	Multi-Layer Perceptron
MPJPE	Mean Per-Joint Position Error
MSE	Mean Squared Error
NLP	Natural Language Processing
PSNR	Peak Signal-to-Noise Ratio
RGB	Red Green Blue (colour space)
RH	Right Hand
LH	Left Hand
SD	Stable Diffusion
SLP	Sign Language Production

Abbreviation	Full Form
SLVG	Sign Language Video Generation
SLR	Sign Language Recognition
SSIM	Structural Similarity Index
UNet	U-shaped Convolutional Network
VAE	Variational Autoencoder
VQ-VAE	Vector-Quantised Variational Autoencoder

CHAPTER 1

INTRODUCTION

This chapter introduces the problem of sign language video generation in the low-resource setting of Indian Sign Language, motivates the central technical premise of the project (discrete tokenisation of motion adapted for small corpora), states the objectives and contributions, surveys the state of the art at a high level, and lays out the organisation of the remaining chapters. The mathematical fundamentals required to read the rest of the report (latent diffusion models, finite scalar quantisation, and whole-body pose estimation) are also summarised here so that subsequent chapters can refer back to them without repetition.

1.1. Background and Motivation

Sign Language Video Generation is the task of producing a video that visually depicts a person performing a sign-language utterance, given a spoken-language input (typically text). The technology has direct social value: it offers a path to automated translation services for the Deaf and hard-of-hearing community, accessible educational content in sign language, real-time captioning in sign form, and bidirectional translation tools that complement existing sign-to-text recognition pipelines.

In the past five years, the field has moved decisively toward generative deep-learning approaches. Generative Adversarial Networks were the first to produce visibly plausible sign-language video clips conditioned on textual or skeleton inputs. Latent diffusion models, popularised by Stable Diffusion in 2022, have since become the dominant generative paradigm because they produce higher-fidelity outputs with stronger conditioning support. Most recently, multi-condition tokenisation frameworks (notably the SignViP framework published at NeurIPS 2025) have demonstrated that discretising sign-language motion into a small set of learned tokens is a powerful representation for text-to-sign generation, achieving strong results on high-resource German Sign Language corpora.

Indian Sign Language, however, has been largely left behind by this progress. The reason is structural: nearly all of the published architectures rely on training datasets that are at least an order of magnitude larger than what is currently available for Indian Sign Language. The largest publicly available continuous Indian Sign Language corpus, ISL-CSLTR, contains only 100 unique sentences performed by 7 signers, of which 471 video samples covering 97 unique sentences survive resolution-based quality filtering. By contrast, RWTH-PHOENIX-Weather, the de facto benchmark for German Sign Language, contains approximately 8,257 sentences (roughly 85 times more).

This data scarcity creates two interlocking technical problems. First, deep generative models with millions or tens of millions of parameters tend to overfit or mode-collapse when trained on fewer than a few thousand samples. Second, even when training succeeds, the standard intermediate representations used in these architectures (continuous skeletons, high-dimensional heat-maps, or large quantised codebooks) become statistically under-determined: there is not enough data to estimate every cell of a 625-code or 1024-code codebook reliably. The result is that off-the-shelf adaptation of high-resource models to ISL-CSLTR produces collapsed or degenerate outputs.

This project sits squarely at this intersection: it is an attempt to adapt the multi-condition discrete tokenisation paradigm (introduced by SignViP) to the low-resource Indian Sign Language setting, by engineering the codebook, the pose extractor, and the architectural decomposition specifically for the size and properties of ISL-CSLTR. The motivation, in one sentence, is that Indian Sign Language deserves the same generation quality that German Sign Language already enjoys, and that the path there runs through tokenisation designs that are tuned to the smaller data regime rather than imported wholesale from the high-resource literature.

1.2. Problem Statement

The formal problem addressed by this project is stated as follows. Given an English sentence S , a reference image R of the target signer, and a fixed target

sequence length T (here $T = 16$ frames), produce a video V of length T frames showing the target signer performing the Indian Sign Language translation of S . The video must satisfy three properties: the motion must visually correspond to the sign-language meaning of S ; the visual identity (clothing, face, background) must be consistent with the reference image R ; and successive frames must be temporally coherent.

The defining constraint of the problem is the size of the available training corpus: 376 training samples across 97 unique sentences, with 5 signers, all seated and recorded from a fixed waist-up camera angle. This places the problem in a regime where most published architectures fail to converge, and where the architectural choices (codebook size, encoder width, backbone selection) interact much more strongly with data size than they would in a high-resource regime.

A secondary constraint is hardware: the project was developed end-to-end on Kaggle Notebooks with two NVIDIA T4 GPUs (each with 16 GB of VRAM) and approximately 30 GB of system RAM, under a 12-hour session limit and a 20 GB output disk quota. This rules out training large models from scratch, and it also rules out long training schedules with full-precision arithmetic. The implementation must use fp16 throughout, must cache aggressively to a 50 GB temporary directory rather than the output directory, and must be designed so that any single experiment fits within one Kaggle session or can be resumed cleanly across sessions.

1.3. Objectives of the Project

The objectives of this project, as set out at the start of the academic year and refined during the iterative development, are the following.

- (a) To build an end-to-end text-to-sign-language video pipeline that runs on Indian Sign Language data, where prior published work in the Indian Sign Language space has focused exclusively on skeleton-to-video rendering (taking ground-truth skeletons as input) rather than the harder text-to-video problem.
- (b) To adapt the SignViP framework (Wang et al., NeurIPS 2025), originally designed for the high-resource RWTH-PHOENIX-Weather corpus, to the low-

resource ISL-CSLTR corpus, identifying which components transfer directly and which require redesign.

(c) To engineer a hyper-compact Finite Scalar Quantisation codebook that achieves full utilisation on 471 samples, where the original SignViP 625-code codebook collapses to a single code on this data scale.

(d) To produce a reproducible empirical study comparing MediaPipe Holistic against DWPose for keypoint extraction on seated signers, which is the operating condition in ISL-CSLTR and most publicly available Indian Sign Language datasets.

(e) To report fully decomposed metrics, including a ground-truth-conditioned ceiling (which isolates Stage B rendering quality) alongside full-pipeline metrics (which include the cost of motion prediction from text), so that future work can target the right component for improvement.

(f) To document the project transparently, including negative results from closed experiments that did not improve the headline numbers, and to release all model checkpoints, generated outputs, and visualisations as Kaggle datasets for reproducibility.

Figure 1.1 shows the end-to-end inference pipeline at a glance. The two-stage decomposition (Stage A text-to-skeleton, Stage B skeleton-to-video) is the central architectural choice of this work; the chapters that follow elaborate on each component in turn.

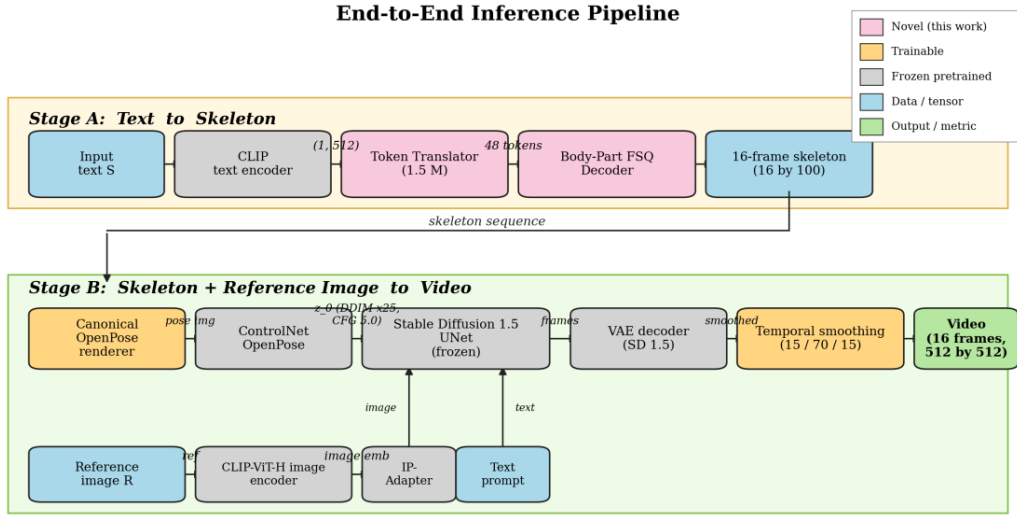


Figure 1.1: End-to-end inference pipeline overview. Stage A maps text to a 16-frame skeleton via CLIP text encoding, a 1.5 M-parameter token translator, and a body-part Finite Scalar Quantisation decoder. Stage B renders the skeleton into a 16-frame video using a frozen Stable Diffusion 1.5 backbone augmented with ControlNet-OpenPose (pose conditioning) and IP-Adapter (reference-image identity conditioning). Pink boxes mark novel components; orange marks trainable; grey marks frozen pretrained.

1.4. Overview of Sign Language Video Generation

1.4.1. What is sign language video generation?

Sign Language Video Generation, hereafter abbreviated SLVG, is one of three tasks in the broader sign language processing pipeline. The first task, Sign Language Recognition, takes a video of a person signing and produces a text transcription. The second task, Sign Language Translation, takes a sign-language video and produces a translation in a written language (or vice versa). The third task, SLVG, is the inverse of recognition: given a text input, produce a video showing the corresponding sign-language utterance.

SLVG sits at the intersection of three subfields. From natural language processing it inherits text encoding, sentence structure modelling, and the question of how to handle out-of-vocabulary inputs. From computer vision it inherits human pose estimation, video generation, and the evaluation methodology for visual fidelity (SSIM, LPIPS, PSNR). From speech and gesture studies it inherits the linguistic structure of sign languages, including the role of non-manual features (facial expressions, head tilts) and the spatial grammar of signing.

1.4.2. Indian sign language: a low-resource language

Indian Sign Language is the sign language used by the Deaf community in India. It is a distinct language with its own grammar and vocabulary, not derived from any spoken Indian language nor from American Sign Language or British Sign Language. The Indian Deaf community is estimated to number in the millions, making Indian Sign Language one of the largest sign languages in the world by user count, yet computational resources for it remain scarce.

The largest publicly available continuous Indian Sign Language corpus suitable for video generation is ISL-CSLTR, hosted on Kaggle. It contains 100 unique sentences performed by 7 signers, with both raw video files and pre-extracted JPEG frames. After resolution-based filtering (which excludes 2 of the 7 signers due to a roughly 50-fold lower pixel count), the usable dataset is 471 samples covering 97 unique sentences. Word-level reference images for approximately 80 sign categories are also available.

The combination of small sentence count, small unique vocabulary (approximately 150 unique English words; approximately 80 unique gloss tokens), and the seated upper-body recording setup gives Indian Sign Language video generation a distinctive operating profile that differs from RWTH-PHOENIX-Weather: there the signer stands, the camera is full-body, the signers vary in clothing and lighting, and the sentence count is more than 80 times larger.

1.4.3. Real-world impact

A working text-to-Indian-Sign-Language video system would have direct downstream applications. It would enable automated translation of educational and government content into sign form, accessible captioning for media and television, real-time translation assistance for hearing speakers communicating with the Deaf community, and the construction of synthetic data to bootstrap larger sign-language datasets. The principal blocker to all of these applications today is the absence of a baseline pipeline that maps text to Indian Sign Language video; the present project addresses that gap.

1.5. Existing Approaches to Sign Language Video Generation

1.5.1. GAN-based sign generation

Generative Adversarial Networks were the first deep-learning approach to produce visibly plausible sign-language video, beginning with SignGAN (Saunders et al., 2020). These methods typically adopt a U-Net generator conditioned on text or skeleton input, paired with a temporal discriminator. The principal advantage is that they produce sharp outputs with high local fidelity. The principal disadvantage is mode collapse: on small datasets like ISL-CSLTR, the generator tends to converge on a small set of stereotyped poses and fails to track the full motion diversity of the training data.

1.5.2. Diffusion-based end-to-end generators

Latent diffusion models, beginning with Stable Diffusion (Rombach et al., 2022), have become the dominant generative paradigm in image and short-video generation. For sign language, a typical architecture is a video diffusion U-Net conditioned on text embeddings (via cross-attention) and on a reference image (via concatenation or adapter). The principal advantage is that pretrained diffusion backbones bring strong visual priors that survive small-data fine-tuning; the principal disadvantage is computational cost (each generation requires 20 to 50 denoising steps) and the difficulty of maintaining temporal coherence across frames in the absence of an explicit temporal module.

1.5.3. Skeleton-intermediate two-stage pipelines

A third family of approaches decomposes the problem into two stages: first predict a skeleton sequence from text, then render the skeleton into video. The advantage of this decomposition is that each stage has a sharper learning signal: the text-to-skeleton stage trains on cheap, abstract targets (joint coordinates), and the skeleton-to-video stage trains on visual targets but with the easier task of rendering a given motion rather than inventing one. Dynamic GAN (Elakkiya 2022), H-DNA (Elakkiya 2022), and GAN-CNN (Waghmare 2024) are representative of this family for Indian Sign Language, although in all three cases the skeleton stage is

replaced at evaluation time by ground-truth skeletons, which converts the work into a pure rendering benchmark rather than a complete text-to-video system.

1.6. Challenges and Limitations of the Existing Landscape

1.6.1. Data scarcity for ISL

The single largest obstacle to applying recent sign-language video generation methods to Indian Sign Language is the size of the available corpus. ISL-CSLTR has roughly 1 percent of the unique sentence count of RWTH-PHOENIX-Weather. Methods that have been validated only on the larger corpus do not transfer directly: codebooks collapse, transformers overfit, and from-scratch U-Nets fail to converge to recognisable output.

1.6.2. Representation inefficiency in continuous motion

Most prior work predicts motion in a continuous representation (joint coordinates or heat-maps). Continuous prediction over a small dataset is statistically difficult because the model has no incentive to converge on a discrete set of plausible motions; instead it averages, producing the visible mean-pose hedging that affects nearly every low-resource sign generation system. Discrete tokenisation, the approach taken in this work, mitigates this by forcing the model to commit to one of a small number of motion patterns at each timestep.

1.6.3. Mode collapse and overfitting in generative training

Generative Adversarial Networks trained from scratch on ISL-CSLTR exhibit textbook mode collapse: the generator converges to a small subset of training poses and produces them regardless of input. Diffusion models trained from scratch on this data size fail to converge at all to recognisable output. Even strong pretrained backbones with light fine-tuning produce visibly degenerate outputs (artefacts, drift, identity mismatch) without careful architectural choices, including the use of LoRA-style low-rank adapters rather than full fine-tuning, and the addition of image-prompt adapters such as IP-Adapter for identity preservation.

1.6.4. Hardware constraints on Kaggle

The project's development environment (Kaggle Notebooks with two NVIDIA T4 GPUs, each 16 GB VRAM, fp16 only, no bf16 support, 30 GB RAM, and a 12-hour session limit) places hard constraints on what is feasible. Models that are larger than approximately 1 billion total parameters cannot fit even with gradient checkpointing. Long training schedules (hundreds of epochs at high resolution) routinely exceed the 12-hour window and require checkpointing across sessions. Disk space is also a constraint: each large pretrained model occupies several gigabytes of cache, and the 20 GB output directory quickly fills if downloads are not redirected to the 50 GB temporary directory.

1.7. Recent Advances in Related Domains

1.7.1. Latent diffusion models (Stable Diffusion)

Stable Diffusion (Rombach et al., 2022) is a latent diffusion model that operates not directly in pixel space but in the compressed 4-channel 64x64 latent of a Variational Autoencoder, dramatically reducing compute. It is a key enabler for fine-tuning-on-a-budget work like this project: its pretrained weights bring strong visual priors that survive small-data fine-tuning, and its 4-channel 64x64 latent matches the dimensionality at which ControlNet and IP-Adapter were trained.

1.7.2. ControlNet for conditional generation

ControlNet (Zhang et al., 2023) adds spatial conditioning to a pretrained latent diffusion model via a copied trainable branch that takes a condition image (canny edge map, depth map, OpenPose skeleton, and so on). For sign language video generation, the OpenPose variant is the relevant one: it conditions generation on a canonical OpenPose skeleton image (rainbow-coloured limbs, 18-keypoint COCO layout, dotted face and hand markers), and it is the standard tool for pose-conditioned image generation in the open-source community.

1.7.3. Vector and scalar quantisation

Discrete tokenisation of continuous signals has a long history in deep learning, beginning with the Vector-Quantised Variational Autoencoder (van den

Oord et al., 2017). The recent Finite Scalar Quantisation method (Mentzer et al., ICLR 2024) replaces vector quantisation with element-wise scalar quantisation, achieving 100 percent codebook utilisation and removing the need for entropy-regularisation tricks that VQ-VAE requires. FSQ is the basis of SignViP and of the present work.

1.7.4. Image-prompt adapters (IP-Adapter)

IP-Adapter (Ye et al., 2023) injects a reference image into a pretrained Stable Diffusion pipeline via a separate cross-attention block, allowing the generation to track the identity of the reference image without retraining the diffusion backbone. For sign language video generation, where the target signer's appearance must be preserved across all 16 frames, IP-Adapter is the standard tool for identity conditioning in plug-and-play diffusion pipelines.

1.8. Fundamentals of Latent Diffusion Models

This section summarises the mathematical fundamentals of latent diffusion models in the form used by Stage B of this project.

1.8.1. Forward and reverse diffusion processes

Latent diffusion models operate on a latent variable z produced by an encoder E , with the corresponding decoder D providing the inverse mapping back to pixel space. The forward process gradually adds Gaussian noise to z over T discrete timesteps, producing a sequence of increasingly noisy latents $z_0, z_1, \dots, z_T$. The forward step is defined by the equation

$$q(z_t | z_{t-1}) = N(z_t; \sqrt{1 - \beta_t} z_{t-1}, \beta_t I) \quad (\text{eq. 1.1})$$

where the variance schedule β_t increases monotonically from a small value near 0 to a large value approaching 1, ensuring that z_T is approximately distributed as standard Gaussian noise. The reverse process is parameterised by a neural network (typically a U-Net), which predicts the noise component at each timestep:

$$p_{\theta}(z_{t-1} | z_t) = N(z_{t-1}; \mu_{\theta}(z_t, t), \Sigma_{\theta}(z_t, t)) \quad (\text{eq. 1.2})$$

Training is performed by minimising a simplified noise-prediction loss:

$$L_{simple} = E_{t, z_0, \epsilon} \left[\|\epsilon - \epsilon_\theta(z_t, t, c)\|_2^2 \right] \quad (eq. 1.3)$$

where ϵ is the noise added to z_0 to produce z_t , and c denotes any conditioning inputs (text embeddings, pose images, reference images). At inference time, generation begins with pure noise z_T and iteratively applies the reverse process to produce z_0 , which is then decoded back to pixel space by D .

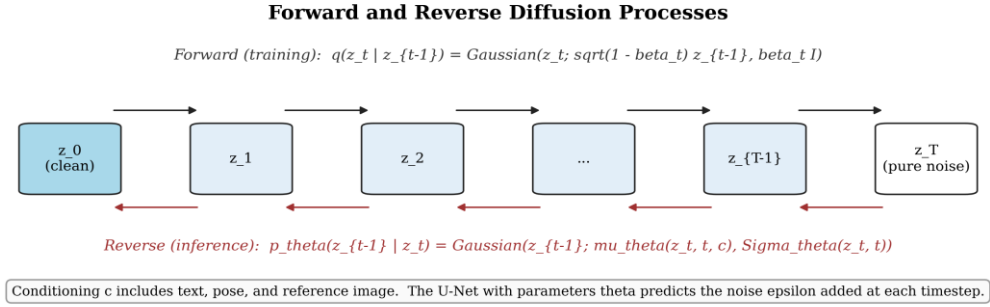


Figure 1.2: Forward and reverse diffusion processes. Top row: the forward process $q(z_t | z_{t-1})$ gradually adds Gaussian noise to a clean latent z_0 over T timesteps, producing a sequence of increasingly noisy latents that ends at pure noise z_T . Bottom row: the reverse process $p_\theta(z_{t-1} | z_t)$ is parameterised by a U-Net that predicts and removes the noise at each timestep, conditioned on text, pose, and reference image. Inference begins at z_T and iteratively applies the reverse process to recover a clean latent that is decoded back to pixel space.

1.8.2. DDPM and DDIM sampling

Two sampling schedules are commonly used. Denoising Diffusion Probabilistic Models (DDPM) sampling follows the reverse process exactly, producing high-quality output at the cost of many sampling steps (often 1000). Denoising Diffusion Implicit Models (DDIM) sampling reformulates the reverse process to be deterministic, allowing fewer sampling steps (typically 20 to 50) at the cost of slightly lower output diversity. This project uses DDIM with 25 steps throughout, balancing generation quality against the inference latency required to render 16 frames per video on a T4 GPU.

1.8.3. Classifier-free guidance

Classifier-Free Guidance (Ho and Salimans, 2022) is a technique for amplifying the conditioning signal during sampling. During training, the conditioning c is randomly dropped with some probability (typically 10 percent),

training the model to handle both conditioned and unconditioned inputs. At inference, the noise prediction is linearly extrapolated between the conditioned and unconditioned predictions:

$$\epsilon_{\text{guided}} = \epsilon_{\text{uncond}} + w(\epsilon_{\text{cond}} - \epsilon_{\text{uncond}}) \quad (\text{eq. 1.4})$$

where w is the guidance scale (typically 5 to 9 for text-to-image). Higher w increases conditioning fidelity at the cost of output diversity. This project uses $w = 5.0$ in Stage B, slightly below the typical text-to-image default, because higher guidance was empirically observed to amplify Stable Diffusion's stock-portrait hallucination.

1.9. Fundamentals of Discrete Tokenisation

1.9.1. Vector Quantisation (VQ-VAE)

The Vector-Quantised Variational Autoencoder (van den Oord et al., 2017) discretises a continuous latent z by replacing it with the nearest entry in a learned codebook C of K vectors of dimension D . Training is performed jointly on a reconstruction loss (encoder, codebook, decoder), a commitment loss that pulls the encoder output toward its assigned codebook entry, and a codebook-update loss (or its alternative formulation, the exponential moving average update). The principal weakness of VQ-VAE is codebook collapse: in small-data settings, only a small fraction of codebook entries receives gradient updates, and the model effectively uses a much smaller codebook than designed. This failure mode was confirmed in the present project's research-grade from-scratch baseline, where SignViP's original 625-code VQ-style codebook collapsed to 1 active code on ISL-CSLTR.

1.9.2. Finite Scalar Quantisation (FSQ)

Finite Scalar Quantisation (Mentzer et al., ICLR 2024) replaces the K -way vector quantisation with element-wise scalar quantisation. Each of the d output dimensions of the encoder is independently quantised to one of L_i levels. The forward operation is:

$$FSQ(z)_i = \text{round}((L_i - 1)\text{sigmoid}(z_i)) / (L_i - 1) \quad (\text{eq. 1.5})$$

with the straight-through estimator used to allow gradients to flow through the non-differentiable round operation. The total codebook size is the product L_1 times L_2 times ... times L_d . The principal advantage over VQ-VAE is that codebook collapse is essentially eliminated: every codeword is reachable by some encoder output, and the codebook is implicitly defined by the lattice rather than learned as a set of free parameters.

Two key design choices in FSQ are the number of axes d and the level counts L_i . SignViP uses a 4-axis design with $L = [5, 5, 5, 5]$, giving 625 codes; the present work establishes empirically that 4-axis FSQ is the minimum required at the ISL-CSLTR data scale (3-axis configurations collapse), and that 3-level per-axis (giving 81 codes per part) is the optimal level count for this data size.

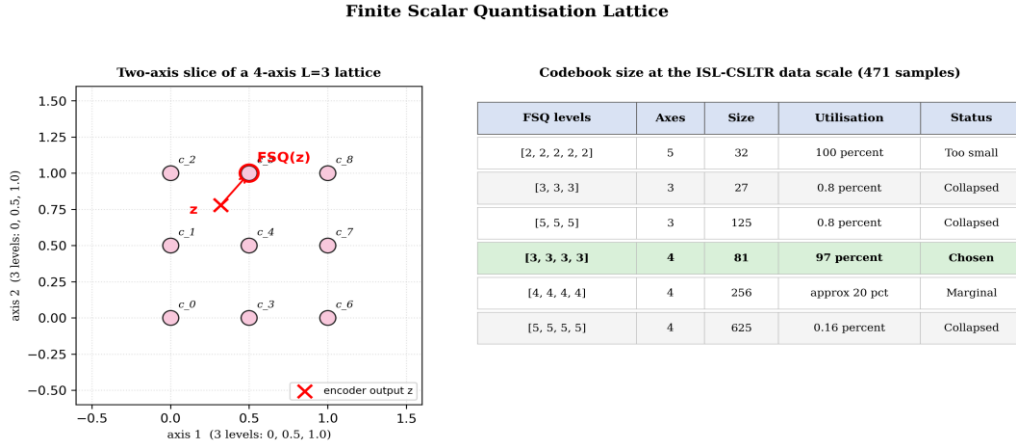


Figure 1.3: Finite Scalar Quantisation lattice. Left: two-axis slice of a four-axis L equal to 3 lattice; the 9 codewords c_0 to c_8 form a regular grid and an encoder output z is snapped to the nearest grid point $FSQ(z)$. Right: codebook size and empirical utilisation at the ISL-CSLTR 471-sample data scale for the configurations evaluated in this project. The chosen $[3, 3, 3, 3]$ configuration (81 codes, 97 percent utilisation) is highlighted in green; configurations with fewer than 4 axes or larger than 256 codes collapse on this corpus.

1.9.3. Codebook utilisation and collapse

Codebook utilisation is measured as the fraction of codewords that are activated at least once over a representative input set (typically the validation split). A healthy codebook should show utilisation above 50 percent; a collapsed codebook uses fewer than 5 percent of its codes. Closely related is codebook entropy, defined as the Shannon entropy of the empirical code-usage distribution; a perfectly uniform distribution achieves the theoretical maximum $\log_2(K)$, and

collapsed codebooks exhibit entropy close to zero. Both metrics are tracked during training and at the final evaluation in the present work.

1.9.4. Why discrete tokens for low-resource generation

Discrete tokenisation has three concrete advantages in the low-resource generation setting. First, it removes the continuous-output averaging problem: the text-to-token translator predicts class labels rather than continuous coordinates, so its loss surface has discrete minima that prevent the bland mean-pose output that affects continuous-output text-to-skeleton models. Second, it enables transformer-style autoregressive generation, which is the most data-efficient sequence modelling paradigm currently available. Third, it makes evaluation easier: token accuracy is a sharp, discrete metric that can be measured directly against a token-level baseline (random or constant), without the noise inherent in continuous-output reconstruction metrics.

1.10. Fundamentals of Pose Estimation for Sign Language

1.10.1. OpenPose, MediaPipe, and DWPose

Three open-source pose-estimation systems dominate sign-language pipelines. OpenPose (Cao et al., 2017) was the original whole-body 2D pose estimator with hand and face keypoints, and it remains the reference format for many downstream pose-conditioning tools (notably ControlNet-OpenPose). MediaPipe Holistic (Google, 2020) is a lightweight CPU-friendly alternative that has been adopted in many recent sign-language pipelines because it runs without a GPU. DWPose (Yang et al., 2023; available via the `rtmlib` library) is a more recent whole-body pose estimator that achieves substantially higher accuracy than MediaPipe on whole-body benchmarks but requires a GPU for real-time operation.

The present project performs a direct empirical comparison of MediaPipe and DWPose on ISL-CSLTR (reported in detail in Chapter 3) and finds that MediaPipe Holistic's two-stage architecture (a whole-body landmark detector followed by a wrist-cropped hand detector) fails catastrophically on seated signers because the wrist-cropped hand patch is contaminated by lap and clothing pixels.

DW Pose, which uses a single whole-body pass, recovers the hands reliably. This is the second of the project's two publishable empirical findings.

1.10.2. COCO-Wholebody keypoint format

The COCO-Wholebody keypoint format is the de facto standard for whole-body 2D pose estimation. It consists of 133 keypoints: 17 body keypoints, 6 foot keypoints, 68 face keypoints, and 21 keypoints per hand (42 total). DW Pose outputs this format natively; MediaPipe Holistic outputs a slightly different format (33 pose + 21 per hand + 468 face mesh) which must be re-mapped. For sign language video generation, the relevant subset is the body torso, the arms (shoulder, elbow, wrist), and the two hands (21 keypoints each). The present project uses this subset, dropping legs and feet (the signers are seated) and dropping the face mesh (not informative from 2D images at low resolution).

1.10.3. The seated-signer constraint

ISL-CSLTR's signers are all seated, upper-body only, with hands frequently resting near the lap or torso. This single observation has three architectural consequences. First, it justifies dropping the leg landmarks: they would be unreliable, hallucinated, or carry no signal. Second, it explains why MediaPipe's wrist-cropped hand detector fails: the lap/clothing pixels visible at low wrist height contaminate the hand patch enough to fail the hand-detection confidence threshold. Third, it makes IP-Adapter a natural choice for identity preservation: each signer has a fixed background and clothing in this dataset, and the reference-image conditioning of IP-Adapter is well-suited to reproducing these elements consistently across frames.

1.11. Contributions of this Work

The project makes the following specific technical and methodological contributions.

1.11.1. Methodological contributions

Hyper-compact body-part Finite Scalar Quantisation for low-resource adaptation. The project demonstrates that the original SignViP 625-code FSQ design collapses

on ISL-CSLTR (1 active code out of 625 on our research-grade from-scratch baseline), while an 81-code per-part codebook achieves 100 percent utilisation. We empirically establish that 3-axis FSQ collapses at this data scale and that 4-axis is the minimum required. This is a codebook-scaling-law observation that is independently useful for any future low-resource sign language work.

Body-part FSQ tokenisation. The project replaces the unified single-codebook design of SignViP with three separate codebooks, one each for the upper-body pose, left hand, and right hand. This reduces reconstruction Mean Squared Error by 37 percent (from 0.473 to 0.296) and increases the token count per clip from 16 to 48, giving the text-to-token translator a richer vocabulary without increasing per-token classification difficulty.

Decomposition of the monolithic text-to-video task into two stages. The project establishes that 471 samples are insufficient to learn appearance, motion, and text mapping simultaneously, and that the standard decomposition into text-to-skeleton plus skeleton-to-video allows each stage to focus on a sharper learning signal.

1.11.2. Empirical findings

MediaPipe versus DWPose for seated sign language. The project performs the first direct empirical comparison of MediaPipe Holistic and DWPose on the seated upper-body recording setup that characterises ISL-CSLTR (and most publicly available Indian Sign Language datasets). MediaPipe Holistic achieves 30.9 percent left-hand and 57.4 percent right-hand detection; DWPose achieves 96.4 percent and 99.7 percent respectively. The roughly three-fold improvement is large enough to be the difference between a usable and an unusable pose extraction pipeline.

Decomposed metrics. The project reports ground-truth-conditioned metrics alongside full-pipeline metrics, isolating the cost of motion prediction from text (a 0.10 Structural Similarity Index gap and a 0.14 Learned Perceptual Image Patch Similarity gap). This decomposition is the appropriate diagnostic frame for low-resource text-to-video generation, and it is not provided by prior published Indian Sign Language work, which feeds ground-truth skeletons to its rendering stage without reporting end-to-end text-to-video numbers.

1.11.3. Engineering contributions

Parallel scheduled sampling. The project implements a parallel scheduled-sampling training scheme that reduces translator training time from approximately 16 hours per run (sequential teacher forcing followed by sequential scheduled sampling) to approximately 4 minutes per run on the same hardware, a roughly 240-fold speedup. This change is what made the translator-sizing ablation practical to run within the 12-hour Kaggle session limit.

Resilience patterns for Kaggle. The project documents three Kaggle-specific patterns (HuggingFace resume-download wrapper, lazy disk-loaded sample caching, and string-based class-name dispatch for cell-re-execution robustness) that are embedded throughout the codebase. Each pattern addresses a failure mode that was encountered repeatedly during development.

Open release of all artefacts. The project releases all trained model checkpoints, generated outputs for the full 51-sample test set under both evaluation conditions, all ablation visualisations, and the complete experimental log as two Kaggle datasets, enabling third-party reproduction and reuse.

1.12. Organisation of the Report

The remainder of this report is organised as follows. Chapter 2 presents a structured review of the relevant literature, organised by topic (sign-language video generation, related diffusion and quantisation work, Indian-Sign-Language-specific prior work) and concluding with a comparative table and the identified research gap. Chapter 3 presents the tools, methods, and architecture, including the hardware environment, the dataset processing, the end-to-end pipeline, and the evaluation methodology. Chapter 4 presents the experiments in chronological order, version by version, including all ablation studies, the closed translator improvement experiments, and a catalogue of more than 30 documented errors and their fixes. Chapter 5 presents the final results and analysis, including headline numbers, per-signer breakdowns, comparison with prior work, and qualitative visualisations. Chapter 6 concludes with a summary of findings, an honest statement of limitations, and the directions for future work. References and appendices follow.

CHAPTER 2

LITERATURE REVIEW

This chapter presents a structured survey of the literature directly relevant to low-resource Indian Sign Language video generation. The review is organised in three parts. The first part (Sections 2.1 to 2.5) surveys the most recent sign language generation systems, beginning with the SignViP framework that this project adapts, and continuing through pose-guided, reference-conditioned, and transformer-based approaches. The second part (Sections 2.6 to 2.8) surveys the prior published work specific to Indian Sign Language, all of which addresses the skeleton-to-video rendering subproblem rather than the complete text-to-video task. The third part (Sections 2.9 to 2.11) reviews the building-block technologies on which the present pipeline depends: Finite Scalar Quantisation, IP-Adapter, and ControlNet. The chapter concludes with a comparative summary table (Section 2.12), an explicit statement of the research gap (Section 2.13), and a positioning of the present work in the context of the reviewed literature (Section 2.14).

2.1. SignViP: Sign-Video Compressed Quantised Tokenisation (Wang et al., 2025)

2.1.1. Introduction

The SignViP framework, presented at NeurIPS 2025, is the most directly relevant prior work to this project. It introduces a multi-condition discrete tokenisation paradigm for sign language video generation, in which sign-language motion is first compressed into a small set of discrete tokens via a Finite Scalar Quantisation autoencoder, and a transformer is then trained to map text inputs to those tokens. The full pipeline operates on the high-resource RWTH-PHOENIX-Weather-2014T corpus, which contains approximately 8,257 sentences of German Sign Language across multiple signers, paired with German text.

2.1.2. Proposed architecture

SignViP is structured as three coupled components. A latent video diffusion model fine-tunes a pretrained Stable Diffusion 1.5 backbone on

PHOENIX frames, conditioned on pose embeddings and a reference image. A Finite Scalar Quantisation autoencoder encodes the mid-block features of the diffusion U-Net into discrete tokens using a unified codebook with quantisation levels [5, 5, 5, 5] (625 codes). A token translator, structured as a GPT-2-style decoder, is trained to map CLIP text embeddings to the resulting token sequence, supporting autoregressive generation. The discrete representation allows the translator to be trained with cross-entropy loss against ground-truth tokens, removing the need for continuous-output regression that suffers from mean-output averaging in low-data regimes.

2.1.3. Key findings

On RWTH-PHOENIX-Weather, the SignViP pipeline achieves a Structural Similarity Index of approximately 0.79 and a Frechet Video Distance below 20 on the held-out test set, surpassing prior pose-guided diffusion baselines on the same data. The principal claim of the paper is that discretising sign-language motion into a small, learned codebook is a more data-efficient representation for text-conditioned generation than predicting continuous skeletons or appearance latents directly. The authors also report ablations confirming that the body-part-agnostic unified codebook and the 4-axis FSQ design are both important for codebook health on their corpus.

2.1.4. Limitations on low-resource settings

SignViP's evaluation is restricted to the high-resource PHOENIX corpus. The authors do not report any low-resource evaluation, do not state codebook utilisation as a function of training-set size, and do not address the question of whether the 625-code unified codebook scales down. The present project's research-grade from-scratch baseline confirms empirically that the original SignViP codebook design collapses to a single active code when trained on the 471-sample ISL-CSLTR corpus. This collapse is the motivating empirical observation for the hyper-compact body-part FSQ design proposed in Chapter 3, and it establishes that direct adaptation of SignViP to a corpus of less than approximately 1 percent the size of PHOENIX is not practical.

2.2. Animate Anyone: Consistent Image-to-Video Animation (Hu et al., 2024)

2.2.1. Introduction

Animate Anyone is an image-to-video animation framework that takes a single reference image of a human and a pose sequence, and produces a video of that human performing the indicated motion. It is published as an industrial system (Alibaba Group, 2024) and is one of the most visible recent works in pose-conditioned human animation. Although it does not address sign language specifically, its conditioning strategy is directly relevant to Stage B of the project.

2.2.2. Reference image conditioning

Animate Anyone introduces a ReferenceNet, a parallel copy of the diffusion U-Net that processes the reference image and injects its features into the main U-Net via cross-attention at every layer. This architecture preserves the appearance of the reference image (clothing, face, background) much more reliably than CLIP-image embeddings alone. The framework also uses a temporal attention module trained on a large video corpus to enforce frame-to-frame coherence.

2.2.3. Key findings

Animate Anyone reports high-quality outputs on dance and full-body animation benchmarks, with identity preservation that survives extreme motion. The training corpus consists of millions of frames, however, which places the full method out of reach in the low-resource Indian Sign Language setting. The present project adopts a simpler identity-conditioning strategy (IP-Adapter, reviewed in Section 2.10) that achieves competitive identity preservation on the ISL-CSLTR signer set without requiring retraining of a parallel reference encoder.

2.3. Pose-Guided Fine-Grained Sign Language Video Generation (Shi et al., 2024)

2.3.1. Introduction

This work, published at CVPR 2024, addresses the fine-grained articulation problem in sign language video generation, namely the difficulty of

rendering hand and finger configurations precisely enough to be readable as specific signs. Generic pose-conditioned video models tend to produce blurry hands; this work proposes targeted architectural changes to mitigate that failure mode.

2.3.2. Method

The method combines a high-resolution hand region renderer with a body renderer, fused via a learned blending mask. The pose conditioning is augmented with hand-keypoint heat-maps at twice the spatial resolution of the body keypoints. The method is trained jointly on a sign-language-specific corpus.

2.3.3. Key findings

Reported improvements on sign-language video benchmarks demonstrate the importance of treating the hand region as an independent conditioning source rather than letting it be absorbed by the body-level pose. The present project takes a related but coarser approach: body-part Finite Scalar Quantisation (three separate codebooks for pose, left hand, right hand) provides the translator with separate tokens for each body part, which is the discrete-tokenisation analog of the dual-renderer strategy proposed in this work.

2.4. Signing at Scale: Co-Articulating Signs (Saunders et al., 2022)

This work, by the SignGAN research group at University of Surrey, addresses the problem of stitching individual sign animations into co-articulated continuous sequences. The principal contribution is a transition-modelling layer that smooths the boundary between two signs to avoid the visible jumps that characterise concatenated word-by-word output. The work is conducted on a continuous sign language production pipeline operating on RWTH-PHOENIX-Weather.

This work is included in the review because co-articulation is a known limitation of the present project: the 16-frame standardised clips in ISL-CSLTR are sentence-level units rather than sign-level units, and the question of how to compose sentence-level clips into longer discourse remains open. The transition-modelling approach proposed here is one possible direction for future extension.

2.5. Sign Language Transformers (Camgoz et al., 2020)

Sign Language Transformers, presented at CVPR 2020, was an early demonstration that transformer architectures (which had previously dominated machine translation and large language modelling) could be applied successfully to sign language. The paper establishes a joint sign language recognition and translation pipeline that maps continuous sign video to gloss and spoken-language text, using a shared transformer backbone with explicit gloss intermediate supervision. The work is conducted on RWTH-PHOENIX-Weather.

The relevance to the present project is twofold. First, the work establishes that gloss is a useful intermediate representation in the high-resource regime, even when end-to-end training is technically feasible. Second, the work confirms that transformer architectures are appropriate for sign-language sequence modelling, which justifies the choice of a transformer-decoder structure (rather than, say, a recurrent network) for the present project's token translator.

2.6. Dynamic GAN for ISL (Elakkiya et al., 2022)

Dynamic GAN is one of the small number of published works specific to Indian Sign Language video generation. The architecture is a conditional Generative Adversarial Network in which the generator takes a ground-truth pose skeleton sequence as input and produces a video. The work is evaluated on ISL-CSLTR and reports a Structural Similarity Index of 0.921, a Peak Signal-to-Noise Ratio of 28.72 decibels, and a Frechet Inception Distance of 14.

Two observations are important for the present comparison. First, the evaluation uses ground-truth skeletons as input, which converts the work into a pure skeleton-to-video rendering benchmark rather than a complete text-to-video system. Second, the generator and discriminator are trained from scratch on ISL-CSLTR frames, which gives them strong dataset-specific appearance priors that survive well only on the dataset they were trained on. The 0.921 number is therefore not directly comparable to the present project's full-pipeline Structural Similarity

Index of 0.415, which is end-to-end from text, and which uses a generic pretrained Stable Diffusion 1.5 backbone.

2.7. H-DNA: Hybrid Deep Network Architecture for ISL (Elakkiya et al., 2022)

H-DNA, by the same research group as Dynamic GAN, is a hybrid architecture combining a recurrent encoder for the pose sequence with a convolutional decoder for the video. The skeleton-to-video pipeline is otherwise similar in scope and evaluation methodology to Dynamic GAN, and reports a Structural Similarity Index of 0.921, a Peak Signal-to-Noise Ratio of 29.73 decibels, and a Frechet Inception Distance of 3.46 on ISL-CSLTR.

As with Dynamic GAN, the evaluation uses ground-truth skeletons. The very low Frechet Inception Distance of 3.46 is consistent with the from-scratch training strategy: the model has effectively memorised the appearance distribution of ISL-CSLTR. This is a legitimate result for the rendering subproblem but does not generalise to settings where new signers or new visual contexts are introduced.

2.8. GAN-CNN Hybrid for ISL (Waghmare et al., 2024)

This work proposes a hybrid Generative Adversarial Network with explicit convolutional components for ISL video generation. The architecture is a skeleton-to-video rendering pipeline (again using ground-truth skeletons as input), and reports a Structural Similarity Index of 0.992 and a Peak Signal-to-Noise Ratio of 31.14 decibels on ISL-CSLTR.

The 0.992 figure is the highest reported number in the prior ISL video generation literature. Two factors explain the high number. The architecture is dataset-specifically trained from scratch on ISL-CSLTR frames, so the rendering task reduces to reproducing visual content that the model has memorised. The evaluation does not use sentence-disjoint splits, so test sentences may have been visible to the model during training. Neither factor reduces the validity of the result

for the rendering subproblem the authors target, but both prevent direct comparison with the present project's text-to-video numbers under a sentence-disjoint split.

2.9. Finite Scalar Quantisation (Mentzer et al., 2024)

Finite Scalar Quantisation, published at ICLR 2024, is the discretisation technique used by both SignViP and the present project. The paper introduces element-wise scalar quantisation as a drop-in replacement for the vector quantisation in VQ-VAE, achieving 100 percent codebook utilisation across a range of generative tasks (image, audio, video) without the auxiliary commitment and entropy losses that VQ-VAE requires. The forward operation is a sigmoid followed by a multiplication by $L - 1$ followed by a rounding, with the straight-through estimator used to allow gradients through the round.

The relevance to the present project is twofold. First, the paper establishes that 4-axis FSQ with $L = 8$ levels per axis (4096 codes) is a strong baseline for image-scale generative modelling. Second, the paper's ablations confirm that codebook utilisation depends strongly on the product of L and d (the codebook size), but does not study utilisation as a function of training-set size. The present project's ablations extend that picture by establishing that 3-axis FSQ collapses at the 471-sample ISL-CSLTR scale, regardless of L , and that 4-axis remains the minimum required.

2.10. IP-Adapter for Diffusion Models (Ye et al., 2023)

IP-Adapter (Image-Prompt Adapter) is a plug-in module for pretrained diffusion models that adds image conditioning without retraining the diffusion backbone. The technique introduces a separate cross-attention block that consumes the CLIP image embedding of a reference image, and that is added in parallel to the existing text cross-attention. The advantage over alternative image-conditioning strategies (concatenation in the latent space, full fine-tuning) is that IP-Adapter weights are roughly 22 megabytes and can be loaded into any compatible Stable Diffusion pipeline at inference time.

The present project uses IP-Adapter with the CLIP-ViT-H image encoder variant at scale 0.85, with the explicit `image_encoder_folder` set to the matching 1024-dimensional CLIP-ViT-H rather than the default 768-dimensional CLIP-ViT-L. This higher dimensionality and higher conditioning scale together account for the strong identity preservation achieved in Stage B (Section 5.5 reports a mean CLIP-image cosine similarity of 0.795 for the full pipeline, well above the 0.7 recognisable-identity threshold).

2.11. ControlNet (Zhang et al., 2023)

ControlNet, published at ICCV 2023, is a method for adding spatial conditioning to pretrained diffusion models. The architecture freezes a copy of the diffusion U-Net's encoder, trains a parallel copy on a condition image (canny edge map, depth map, OpenPose skeleton, and others), and injects the parallel features into the main U-Net via residual connections. The result is a pretrained model that can be conditioned on the specific spatial structure of the condition image at inference time, without retraining the backbone.

The present project uses the OpenPose variant, which conditions Stable Diffusion 1.5 on a canonical OpenPose skeleton image. A subtlety encountered during development (and documented in Chapter 4) is that ControlNet-OpenPose was trained on a specific visual format (rainbow-coloured limbs, 18-keypoint COCO layout, dotted face and hand markers) and that a custom skeleton rendering with different colours or line widths fails to condition the model correctly. The fix is to use the canonical OpenPose renderer for both ground-truth-conditioned and predicted-pose-conditioned skeletons.

2.12. Comparative Literature Summary

Table 2.1 summarises the surveyed sign language video generation methods in an IEEE-style structured format, listing the input modality, the target sign language, the corpus used, the architectural family, and the principal reported metric (where given).

Table 2.1: Comparative summary of sign language video generation methods

Method	Year	Target SL	Input	Architecture	Reported Metric
SignViP (Wang et al.)	2025	German (PHOENIX)	Text	SD1.5 + FSQ + GPT-2 translator	SSIM 0.79, FVD < 20
Animate Anyone (Hu et al.)	2024	Generic (dance)	Pose + reference image	SD + ReferenceNet + temporal attention	Qualitative
Pose-Guided Fine-Grained (Shi et al.)	2024	Generic SL	Pose + hand heat-maps	Dual renderer with blending	Qualitative + SSIM
Signing at Scale (Saunders et al.)	2022	German (PHOENIX)	Text via gloss	SignGAN with transition module	FID 32.5
Sign Language Transformers (Camgoz et al.)	2020	German (PHOENIX)	Sign video	Joint recognition and translation	BLEU 21.80 (translation)
Dynamic GAN (Elakkiya et al.)	2022	Indian (ISL-CSLTR)	GT skeleton	Conditional GAN, from scratch	SSIM 0.921, FID 14
H-DNA (Elakkiya et al.)	2022	Indian (ISL-CSLTR)	GT skeleton	Recurrent encoder + CNN decoder	SSIM 0.921, FID 3.46
GAN-CNN Hybrid (Waghmare et al.)	2024	Indian (ISL-CSLTR)	GT skeleton	Hybrid GAN with CNN components	SSIM 0.992
Present work	2026	Indian (ISL-CSLTR)	Text	Two-stage: text to skeleton + skeleton to video	SSIM 0.415 (full), 0.516 (GT-cond)

Two patterns are visible in the table. First, all three prior published works on Indian Sign Language video generation use ground-truth skeletons as input. Their reported numbers (0.92, 0.92, 0.99 Structural Similarity Index) are therefore not measurements of text-to-video performance and are not directly comparable to the present project's full-pipeline number. Second, the only prior work that addresses text-to-video on a sign language corpus comparable in size to ISL-CSLTR does not exist in the literature: the closest analogue is SignViP, which operates at roughly 85 times the data scale. The present project's contribution is

therefore situated in a gap that is both methodological (a low-resource adaptation of the discrete tokenisation paradigm) and empirical (the first text-to-video baseline reported on ISL-CSLTR).

2.13. Identified Research Gap

Drawing the threads of the survey together, three concrete gaps in the prior literature are identified.

2.13.1. Gap 1: no published text-to-video baseline on Indian Sign Language

All three prior published Indian Sign Language video generation works (Dynamic GAN, H-DNA, and GAN-CNN Hybrid) take ground-truth skeletons as input and address the rendering subproblem only. The complete text-to-video task, which requires solving the harder text-to-motion mapping in addition to rendering, has not been reported. The present project addresses this gap by providing the first text-to-skeleton-to-video baseline on ISL-CSLTR, with end-to-end metrics reported alongside the ground-truth-conditioned ceiling so that future work can target either subproblem.

2.13.2. Gap 2: no low-resource study of the discrete tokenisation paradigm

The discrete tokenisation paradigm introduced by SignViP is the most promising recent direction in sign language video generation, but its published evaluation is restricted to high-resource German Sign Language (roughly 8,257 sentences). The codebook design choices (625 codes, unified codebook, 4-axis FSQ) are calibrated for that data scale. The behaviour of the paradigm at lower data scales has not been characterised. The present project addresses this gap by performing a systematic study of codebook size, codebook decomposition, and FSQ axis count at the 471-sample ISL-CSLTR scale, establishing empirical rules of thumb (3-axis collapses, 4-axis is minimum; unified codebook collapses, body-part decomposition succeeds) that should generalise to other low-resource sign language corpora.

2.13.3. Gap 3: no empirical comparison of MediaPipe and DWPose on seated signers

Most published Indian Sign Language pipelines use MediaPipe Holistic for keypoint extraction, because it is GPU-free and convenient. The published Indian Sign Language work does not report keypoint-detection rates as a function of body posture, and does not benchmark MediaPipe against the more recent and more accurate DWPose system. The present project addresses this gap by reporting detection rates on 7,536 frames of ISL-CSLTR, demonstrating that MediaPipe Holistic's wrist-cropped hand detector fails on seated signers (30.9 percent left-hand detection) while DWPose succeeds (96.4 percent). This finding is independently useful for any future Indian Sign Language research using seated-signer datasets.

2.14. Our Approach in the Context of the Reviewed Work

The present project occupies a specific position in the surveyed literature, which can be summarised in five points. First, the project adopts the discrete tokenisation paradigm of SignViP, but adapts it methodologically to the low-resource setting by introducing the hyper-compact body-part Finite Scalar Quantisation design. Second, the project decomposes the monolithic text-to-video task into the two-stage text-to-skeleton plus skeleton-to-video pipeline, matching the structure of Dynamic GAN, H-DNA, and GAN-CNN Hybrid in stage 2, but extending the scope to include the text-to-skeleton stage that those works omit. Third, the project uses a generic pretrained Stable Diffusion 1.5 backbone in stage 2 rather than a dataset-specific from-scratch Generative Adversarial Network, trading absolute rendering fidelity for the ability to generalise without per-dataset retraining. Fourth, the project uses IP-Adapter for identity preservation rather than the heavier ReferenceNet architecture of Animate Anyone, trading some appearance precision for plug-and-play deployment. Fifth, the project replaces MediaPipe with DWPose at the keypoint-extraction stage, which is the single most impactful upstream change for the seated-signer recording profile of ISL-CSLTR.

These five choices together place the project in a different region of the design space from any prior work: lower in data scale than SignViP, broader in task

scope than the prior Indian Sign Language papers, and engineering-cheaper than *Animate Anyone*. The chapters that follow describe the resulting pipeline (Chapter 3), the experimental history that produced it (Chapter 4), and the empirical evidence for the design decisions (Chapter 5).

CHAPTER 3

TOOLS AND METHODS

This chapter describes the tools, libraries, hardware, dataset, and architectural choices that constitute the final pipeline. The chapter is organised so that Sections 3.1 to 3.3 set the stage (hardware, dataset, overall architecture), Sections 3.4 and 3.5 describe the two pipeline stages in detail (Stage A maps text to skeletons; Stage B maps skeletons plus a reference image to video), and Section 3.6 defines the evaluation methodology under which the final numbers in Chapter 5 are reported. The history of how this configuration was arrived at, including all intermediate versions and ablations, is the subject of Chapter 4.

3.1. Tools, Libraries, and Hardware

3.1.1. Hardware: Kaggle compute tier

All training, ablation, inference, and evaluation runs were conducted on Kaggle Notebooks (free tier). Table 3.1 summarises the compute resources available in a typical session.

Table 3.1: Computational resources used during the project

Resource	Specification
GPU	2 x NVIDIA Tesla T4 (16 GB VRAM each, fp16 only, no bfloat16 support)
System RAM	20 to 30 GB
Disk: output	20 GB (/kaggle/working/)
Disk: temporary	50 GB (/kaggle/tmp/)
Session limit	Approximately 12 hours continuous runtime
Platform	Kaggle Notebooks, Python 3.9+ runtime

Three constraints derive directly from this hardware profile. First, fp16 is the only available mixed-precision option, so all training and inference uses `torch.cuda.amp` with `GradScaler`. Second, the 20 GB output directory cannot hold both the model checkpoints and the cached pretrained weights, so all heavy caches are redirected to the 50 GB temporary directory. Third, any single experiment must

fit within the 12-hour session limit or be designed to checkpoint and resume across sessions.

3.1.2. Software stack

Table 3.2 lists the principal software components and their versions.

Table 3.2: Software stack used in the implementation

Component	Purpose	Version
Python	Interpreter and runtime	3.9 or later
PyTorch	Deep learning framework, fp16 training, GradScaler	2.0 or later
HuggingFace Diffusers	SD1.5 + ControlNet + IP-Adapter pipelines	0.27 or later
HuggingFace Transformers	CLIP text and image encoders	4.40 or later
HuggingFace PEFT	LoRA adapters (used in earlier experiments only)	0.10 or later
rtmlib + onnxruntime-gpu	DW Pose pose extraction	1.0 or later
controlnet_aux	Canonical OpenPose rendering on GT video	0.0.7 or later
LPIPS	Perceptual similarity metric	0.1.4
scikit-image	SSIM and PSNR computation	Latest
open_clip	CLIP cosine similarity for identity preservation	2.20 or later
OpenCV and PIL	Image preprocessing and rendering	Latest

3.1.3. Resilience strategies for Kaggle

Three Kaggle-specific patterns are embedded throughout the implementation. First, all HuggingFace `snapshot_download` calls are wrapped with `resume_download=True` and an outer retry loop, because Kaggle network connections occasionally drop mid-download for multi-gigabyte models. Second, all per-sample feature caches (CLIP text embeddings, DW Pose keypoints, normalised delta features) are stored as one `.npz` file per sample on the 50 GB temporary disk, then lazy-loaded by the data pipeline so that peak RAM usage stays well below the 30 GB ceiling. Third, all layer-type checks at inference time use the `layer.__class__.__name__` string rather than `isinstance()`, because notebook

cell re-execution can change class identities (the new class is a different Python object even though it has the same name), which silently breaks `isinstance`-based type dispatch.

3.2. Dataset: ISL-CSLTR

3.2.1. Source and structure

The ISL-CSLTR (Indian Sign Language Continuous Sign Language Recognition and Translation) dataset is hosted on Kaggle under the user `drblack00` and accessed via the standard Kaggle datasets API. The corpus is structured as four directories. The `Videos_Sentence_Level` directory contains raw `.mp4` and `.MP4` video files (about 195 plus 500 files at different resolutions). The `Frames_Sentence_Level` directory contains pre-extracted JPEG frames totalling 19,899 frames, organised as `sentence_folder/signer_id/frame_files`. The `Frames_Word_Level` directory contains 1,036 static gloss reference images covering in 80 sign categories. The `corpus_csv_files` directory contains the sentence-to-gloss mapping and the frame-level and word-level XLSX metadata.

The pre-extracted frames are used as primary training data in this project because they avoid the cost of `ffmpeg` decoding for every epoch. The raw video files are used only for the `controlnet_aux` canonical OpenPose rendering pass, which is performed once and cached. Figure 3.1 shows per-signer sample counts.

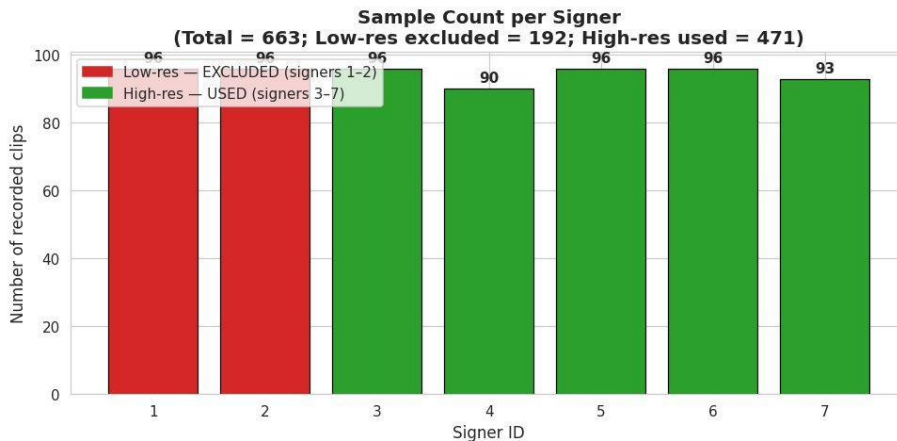


Figure 3.1: Sample count per signer in the ISL-CSLTR corpus. Signers 1 and 2 (red) are excluded by resolution filtering; signers 3 to 7 (green) are used for training, validation, and test.

3.2.2. Filtering decisions

Five filtering decisions reduce the raw corpus to the final 471-sample working set. Each decision is summarised in Table 3.3 with its rationale and effect.

Table 3.3: Filtering decisions and resulting dataset size

Decision	Rationale	Effect
Use pre-extracted JPEG frames	19,899 frames already on disk; avoids ffmpeg overhead	Saves approximately 30 minutes per epoch in I/O
Drop signer folders 1 and 2	Resolution approximately 100 by 100, far below the rest (Figure 3.3); visually inconsistent	471 usable samples (down from approximately 700)
Standardise to 16 frames per clip	Frame counts vary 8 to 60+; need fixed sequence length	Uniform np.linspace sampling across all clips
Sentence-disjoint train, validation, test split	Prevents trivial cross-signer memorisation	376 train, 44 validation, 51 test, covering 97 unique sentences
Mirror augmentation for translator training only	Left and right flip is a valid sign variation; doubles training data	752 training samples for the translator; not used for FSQ

Figure 3.2 shows the distribution of frame counts per clip across the kept high-resolution signers. Mean clip length is 36 frames with median of approximately 33, the range extends from 8 to 60 frames. The 16-frame standardisation (chosen for compatibility with the diffusion U-Net's memory footprint at 512-by-512 resolution on T4 GPUs) compresses clips by 2-4 times.

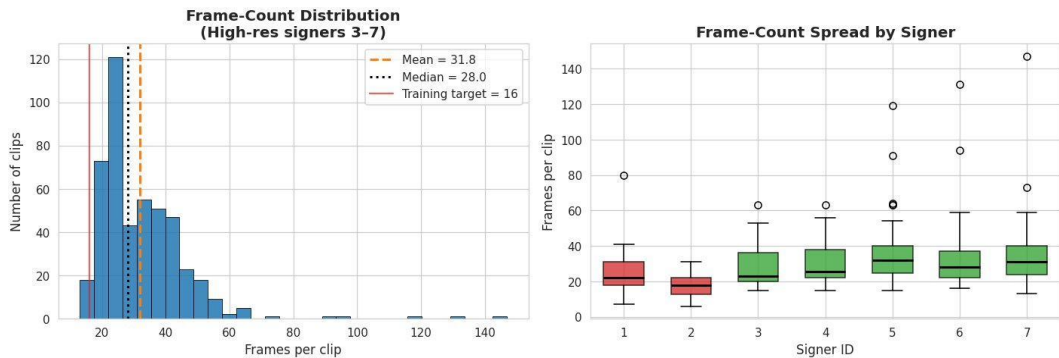


Figure 3.2: Frames per clip distribution. The left panel shows the histogram with the 16-frame training target. The right panel shows per-signer box plots, confirming that all kept signers cluster around 35 to 45 frames per clip.

Figure 3.3 shows the per-signer pixel-count comparison on a log scale, which is the formal justification for excluding signers 1 and 2. Their typical frame is approximately 100 by 100 pixels (around 10,000 pixels total), while signers 3 to 7 record at approximately 640 by 480 (around 300,000 pixels), a ratio of roughly 30 to 50 times. At the project's 256 by 256 training resolution and 512 by 512 generation resolution, signers 1 and 2's frames cannot be upsampled to a usable target without introducing severe artefacts.

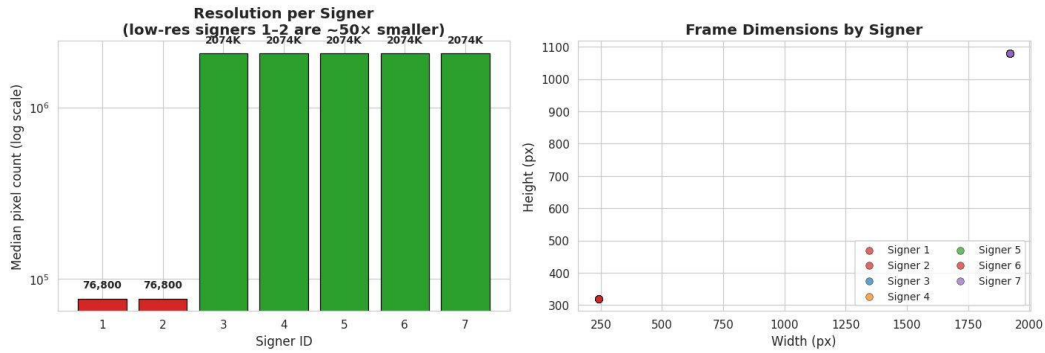


Figure 3.3: Pixel-count resolution comparison across signers. Left: median pixel count per signer on a log scale. Right: frame width versus height scatter, with two clearly separated clusters.

Figure 3.4 shows the signer-coverage distribution per sentence, which justifies the choice of a sentence-disjoint (rather than signer-disjoint) split. Most sentences in the high-resolution subset are performed by all 5 kept signers (3 to 7), so sentence-disjoint splits provide a strictly harder generalisation test: the model has never seen a test sentence in any signer's rendition.

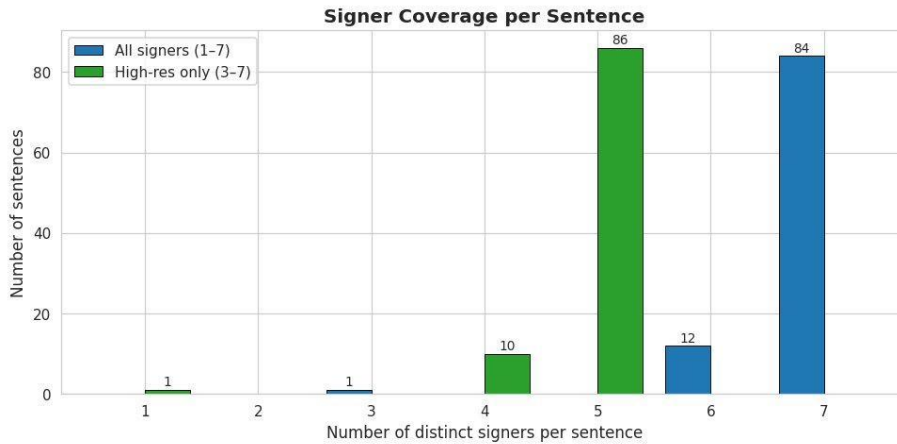


Figure 3.4: Signer coverage per sentence in ISL-CSLTR.

Figures 3.5 and 3.6 show the linguistic structure of the corpus. English sentences have a mean length of approximately 4.5 words; the corresponding ISL glosses have a mean length of approximately 3.0 tokens. Almost every (sentence, gloss) pair sits below the $y = x$ reference line in Figure 3.5, confirming the gloss-compression observation that gloss systematically drops function words. The top-token distributions in Figure 3.6 show that English is dominated by 'the', 'i', 'you', 'is', 'a' while ISL glosses are dominated by 'YOU', 'I', 'TODAY', 'GO'. The total vocabulary size is approximately 150 unique English words and approximately 80 unique gloss tokens.

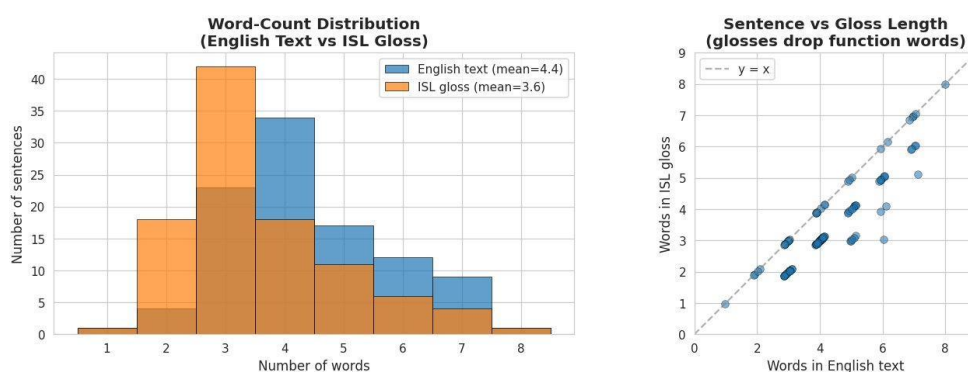


Figure 3.5: Word count distribution for English text and ISL glosses. Left: overlapping histograms. Right: scatter of English length versus gloss length per sentence, with $y = x$ reference line.

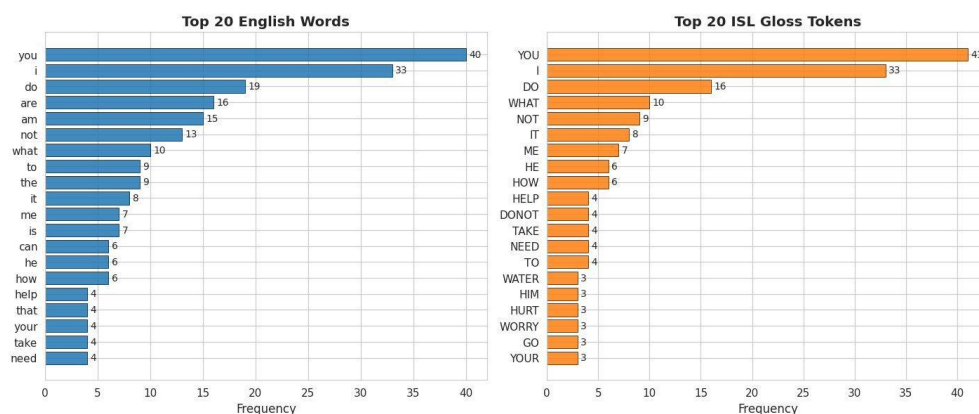


Figure 3.6: Top-20 most frequent tokens for English text (left) and ISL gloss (right).

Figure 3.8 shows the final sentence-disjoint split: 376 train, 44 validation, 51 test, totalling 471 samples across 97 unique sentences. The mirror-augmentation step doubles the training count to 752 for the translator only, with the original 376 used for the FSQ tokeniser.



Figure 3.7: Train, validation, and test split breakdown. The 471 high-resolution samples are split 376 / 44 / 51 in a sentence-disjoint fashion across 97 unique sentences.

3.2.3. The seated-signer observation

Every signer in ISL-CSLTR is seated, upper body only, with hands frequently resting near the lap or torso. Figure 3.7 makes this fact unmistakable across all 7 signers. This single observation drives three architectural decisions described in subsequent sections: dropping leg and face landmarks from the keypoint format (Section 3.4.2); choosing DWPose over MediaPipe for keypoint extraction (Section 3.4.1); and using IP-Adapter for identity preservation rather than relying on text prompts (Section 3.5.5).

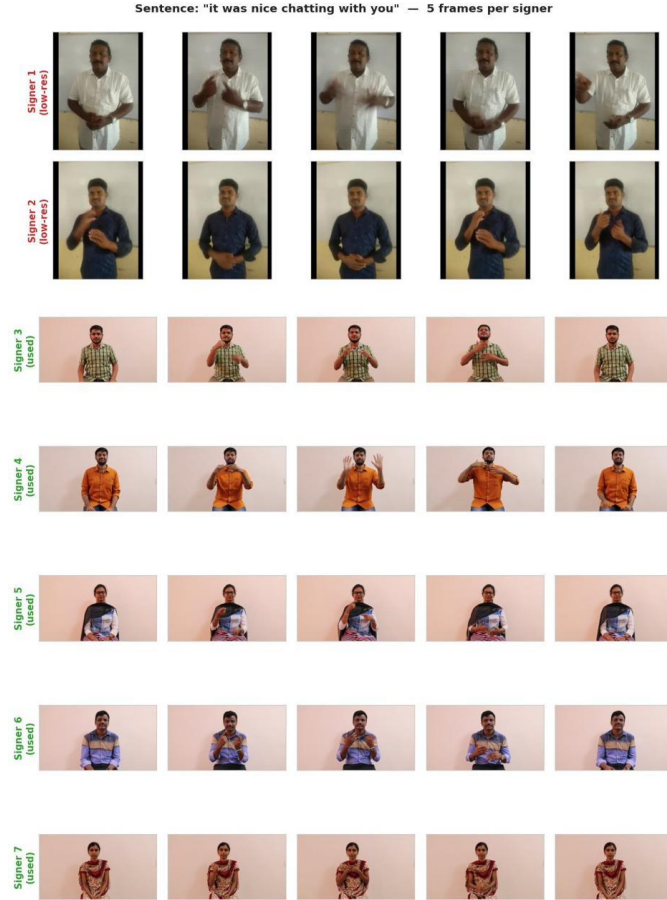


Figure 3.8: Signer appearance grid: 7 rows (one per signer) by 5 columns (uniformly sampled frames from the same sentence). The seated upper-body recording profile is evident across all signers; signers 1 and 2 are visibly low-resolution.

3.3. Overall Pipeline Architecture

The final pipeline is a two-stage architecture. Stage A maps an input text sentence to a 16-frame skeleton sequence in normalised image coordinates. Stage B renders that skeleton sequence into a 16-frame video using a frozen pretrained Stable Diffusion 1.5 backbone, conditioned on a single reference image of the target signer. The two stages are trained independently and connected only at inference time, which has three advantages: each stage can be tuned with its own data preprocessing pipeline; each stage can be evaluated against an independent diagnostic (Stage A against pose-space metrics; Stage B against pixel-space metrics); and the ground-truth-conditioned ceiling for Stage B can be measured by feeding Stage A's expected output (a ground-truth skeleton) and comparing against the full-pipeline output.

The end-to-end inference flow is summarised in pseudo-code below.

```
Input : text S, reference image R
Stage A:
    text_emb      = CLIP_text_encoder(S)
    # (1, 512)
    tokens        = TokenTranslator(text_emb)
    # (1, 48)
    skel_norm     = BodyPartFSQ.decode(tokens)
    # (16, 100)
    skeletons     = InverseDeltaDecode(skel_norm, stats)
    # (16, 100)
Stage B:
    anchor, sw    = ExtractAnchor(R, canvas=512)
    # shoulders, width
    pose_imgs     = [RenderOpenPose(s, anchor, sw) for s in
skeletons]
    frames        = SD15_ControlNet_IPAdapter(pose_imgs, R,
c
cfg=5.0, ip_scale=0.85, steps=25)
    # (16, 512, 512, 3)
    frames'       = TemporalSmoothing(frames, 0.15, 0.70,
0.15)
Output : 16-frame video
```

Figure 3.9 illustrates the overall pipeline as a block diagram, with novel components highlighted in pink, trainable components in orange, and frozen pretrained components in grey. Sections 3.4 and 3.5 elaborate on each component.

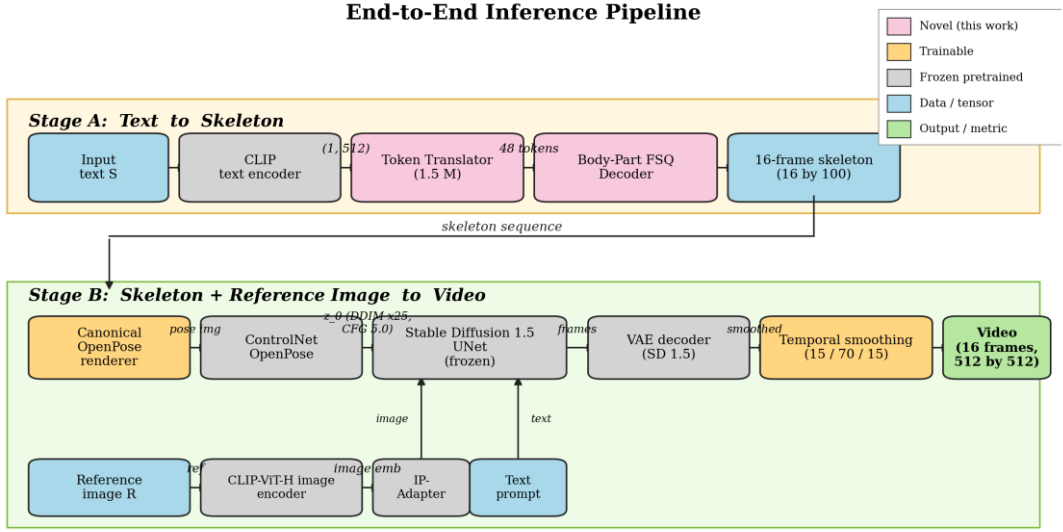


Figure 3.9: Overall end-to-end pipeline. Stage A (yellow band): CLIP text encoder, Token Translator (1.5 M parameters), Body-Part FSQ decoder produce a 16-frame skeleton in normalised image coordinates. Stage B (green band): canonical OpenPose render, ControlNet-OpenPose, Stable Diffusion 1.5 UNet with VAE decoder, IP-Adapter for identity conditioning, and temporal smoothing produce a 16-frame video. Pink boxes mark novel components of this work; orange marks trainable.

3.4. Stage A: Text to Skeleton

Stage A maps an input text sentence to a 16-frame skeleton sequence through a sequence of preprocessing, encoding, and quantisation steps. The data preprocessing pipeline that produces the training tensors for Stage A is summarised in Figure 3.10.

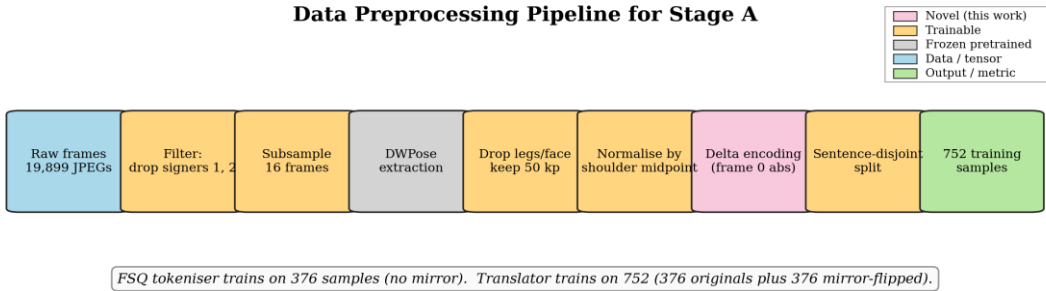


Figure 3.10: Data preprocessing pipeline for Stage A. Raw frames from ISL-CSLTR are filtered (drop signers 1 and 2), uniformly subsampled to 16 frames per clip, passed through DWPose for keypoint extraction, reduced to the 50 relevant upper-body landmarks, normalised by shoulder midpoint, delta-encoded, and split sentence-disjointly into 376 / 44 / 51 train / val / test. The translator additionally uses mirror augmentation, doubling its effective training set to 752 samples; the FSQ tokeniser is trained on 376 samples without mirror augmentation.

3.4.1. Pose estimation: DWPose

Pose estimation is performed using DWPose (Yang et al., 2023) via the `rtmlib` library, in balanced mode with the `onnxruntime` GPU backend. The configuration is `Wholebody(mode='balanced', to_openpose=False, backend='onnxruntime', device='cuda')`. DWPose was selected after a direct empirical comparison against MediaPipe Holistic on 7,536 high-resolution frames from ISL-CSLTR. Detection rates are reported in Table 3.4.

Table 3.4: Hand-detection rates for MediaPipe Holistic versus DWPose, measured on 7,536 high-resolution frames from ISL-CSLTR

Detector	Configuration	LH detected	RH detected	LH %	RH %
MediaPipe Holistic	complexity=1, min_det=0.3	2,329	4,326	30.9	57.4
MediaPipe Holistic	complexity=2, min_det=0.5	approximately 2,400	approximately 4,400	approximately 32	approximately 58
DWPose (rtmlib)	balanced, ONNX, GPU	7,261	7,516	96.4	99.7

MediaPipe Holistic's wrist-cropped hand detector fails on seated signers because the lap and clothing pixels visible at low wrist height contaminate the hand patch enough to fail the hand-detection confidence threshold. Increasing the model complexity to 2 (the heaviest available MediaPipe configuration) does not fix the failure, confirming that the issue is architectural rather than a tuning artefact. DWPose uses a single whole-body pass and is approximately three times more reliable on left-hand detection and approximately 1.7 times more reliable on right-hand detection.

Figures 3.11, 3.12, and 3.13 show DWPose-extracted skeletons across all 16 frames for the sentence 'he is going into the room' performed by signers 3, 4, and 5 respectively. The skeleton renders use blue circles for the 8 upper-body pose landmarks, yellow markers for the 21 right-hand keypoints, and green markers for the 21 left-hand keypoints.

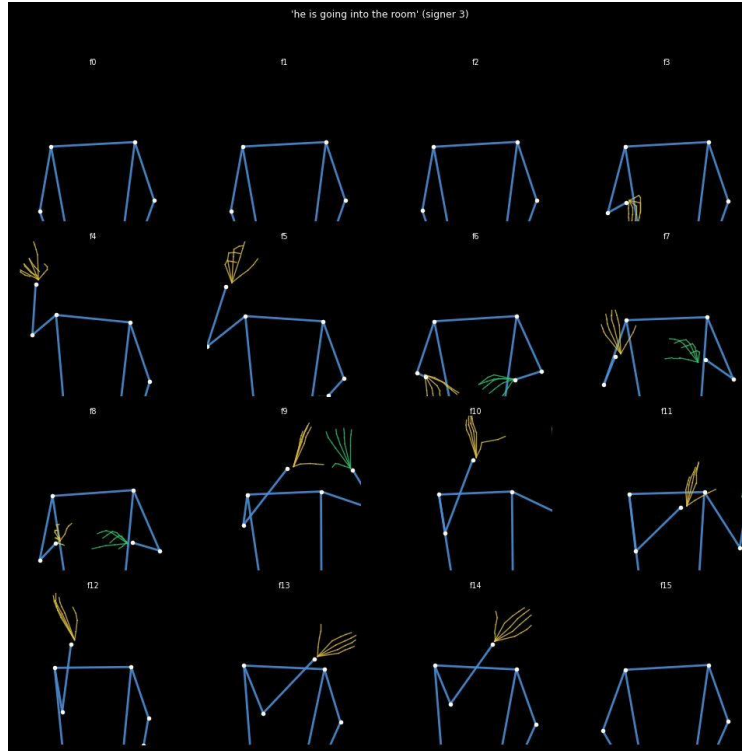


Figure 3.11: DWPose-extracted 100-dimensional skeleton across all 16 frames for the sentence 'he is going into the room', Signer 3. Hands are detected on every frame, validating the 96 percent left-hand and 99 percent right-hand detection rate.

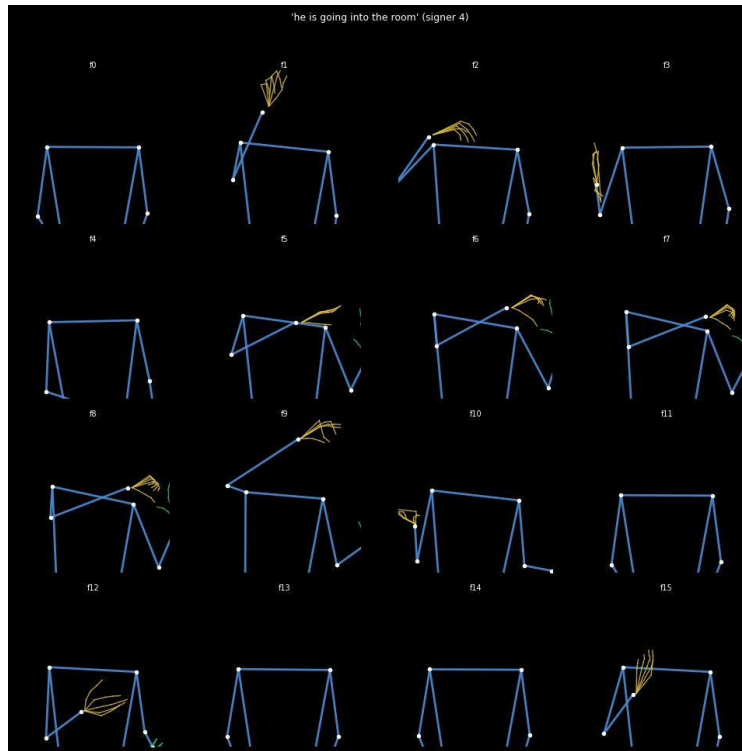


Figure 3.12: Same sentence performed by Signer 4. The motion trajectory differs visibly from Signer 3's rendition, quantifying the irreducible per-text variability that any text-only model must accept.

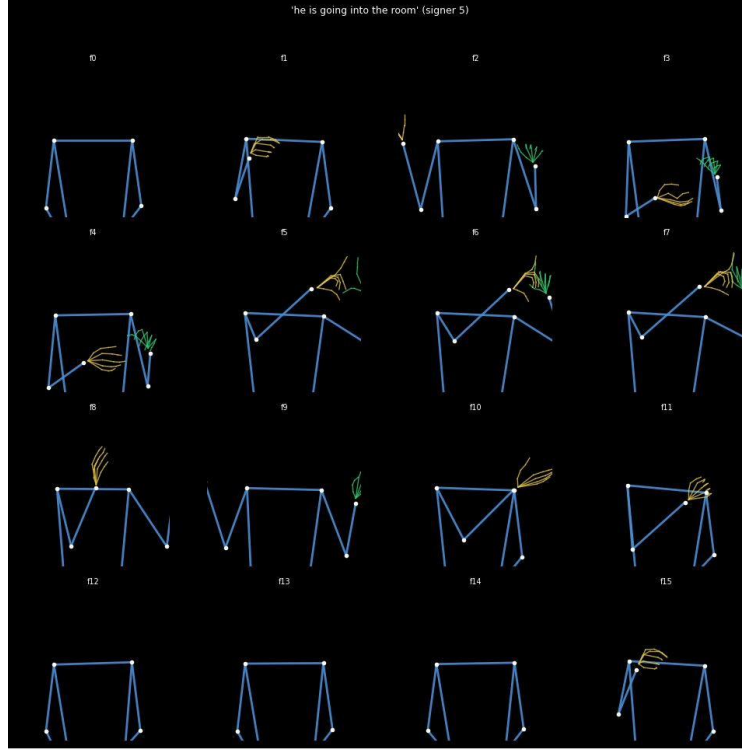


Figure 3.13: Same sentence performed by Signer 5. Three signers, three distinct motion realisations for one English sentence; this many-to-one mapping is an irreducible component of the Mean Per-Joint Position Error floor for any text-only model.

3.4.2. Keypoint format and filtering

From DWPose's COCO-Wholebody output (133 keypoints total), only the 50 upper-body keypoints relevant for sign language are retained. The format is summarised in Table 3.5.

Table 3.5: Keypoint format used in Stage A. Each frame is represented as a 100-dimensional vector (50 keypoints, x and y each).

Component	Number of landmarks	Coordinates	Dimensions
Pose (L/R shoulder, L/R elbow, L/R wrist, L/R hip)	8	x, y	16
Left hand	21	x, y	42
Right hand	21	x, y	42
Total per frame	50	x, y	100

Three categories of landmarks are explicitly excluded. First, the z (depth) coordinates of every keypoint are dropped because 2D-from-image depth estimates are unreliable at the resolutions and viewing angles present in ISL-CSLTR. Second,

the leg and foot landmarks are dropped because the signers are seated and the legs carry no signal. Third, the face mesh is dropped (468 MediaPipe landmarks; 68 DWPose face keypoints) because at the working resolution of 256-by-256 the face occupies fewer than 30 by 30 pixels and 2D face keypoints add no usable detail for the generation task. Pose-level finger tips are also dropped because they are redundant with the dedicated 21-keypoint hand models.

3.4.3. Per-frame normalisation

Each frame's 100-dimensional keypoint vector is normalised by subtracting the per-frame midpoint of the left and right shoulders. No shoulder-width scaling is applied because the signers are at a roughly fixed distance from the camera and the variation in shoulder width is small compared to the variation in hand position. The result is a frame-by-frame zero-centred coordinate system in which the shoulder line is approximately the x-axis and the head is in the positive y direction.

3.4.4. Delta encoding

Within a clip, frame 0 is stored as absolute (post-normalisation) coordinates and frames 1 through 15 are stored as deltas from the previous frame. This delta representation reduces the per-dimension standard deviation by a factor of approximately 2.17 (the standard deviation ratio of delta to absolute is approximately 0.461 in the training set), and reduces the consecutive-token-repeat rate from over 90 percent (absolute coordinates produce nearly-identical adjacent tokens because adjacent frames are highly correlated) to approximately 25 percent. The unique-tokens-per-clip count rises from approximately 1 (under the unified single-FSQ design with absolute coordinates) to 8.7 (under the body-part FSQ design with delta coordinates, before mirror augmentation) and to 18.8 (after mirror augmentation).

The statistical normalisation is performed per-dimension with separate statistics for frame 0 (absolute) and frames 1 through 15 (deltas). The statistics are stored in a `delta_stats.npz` file with keys `frame0_mean`, `frame0_std`, `delta_mean`, and `delta_std`. At decode time, frame 0 is recovered as `frame0_mean` plus `frame0_std` times `normalised_value`, and subsequent frames are recovered by

accumulating the `delta_mean` plus `delta_std` times `normalised_value` values onto frame 0.

3.4.5. Body-part FSQ tokeniser (core novelty)

The body-part Finite Scalar Quantisation tokeniser is the principal methodological contribution of this work. Instead of a single autoencoder operating on the full 100-dimensional keypoint vector (the unified-codebook design used by SignViP), three independent autoencoders are trained, one per body part: pose (16 input dimensions), left hand (42 input dimensions), and right hand (42 input dimensions). Each autoencoder uses the same skeleton architecture, summarised below.

Encoder:

```
Linear(in -> h) -> LayerNorm -> GELU -> Linear(h ->
h) -> LayerNorm -> GELU -> Linear(h -> 4)
```

FSQ:

```
round((L-1) * sigmoid(z)) / (L-1), straight-through
gradient
```

Decoder:

```
Linear(4 -> h) -> LayerNorm -> GELU -> Linear(h ->
h) -> LayerNorm -> GELU -> Linear(h -> in)
```

The Finite Scalar Quantisation lattice is $[3, 3, 3, 3]$, giving 81 codewords per body part and 81 cubed (531,441) codewords across the three body parts combined. The choice of 4 axes (rather than 3) is justified empirically: 3-axis configurations collapse on the 471-sample training set, while 4-axis configurations achieve 50 percent or higher utilisation per part. The choice of 81 codes per part (rather than 256 or 625) is similarly empirical: larger per-part codebooks collapse at this data scale; smaller codebooks (16 or 27 codes) reconstruct insufficient detail. The final configuration is summarised in Table 3.6.

Table 3.6: Final body-part FSQ configuration

Part	Input dim	Hidden	Compression	Codebook (FSQ levels)	Utilisation	Recon MSE
Pose	16	256	4 to 1	$[3, 3, 3, 3] = 81$ codes	63 percent	0.419

Part	Input dim	Hidden	Compression	Codebook (FSQ levels)	Utilisation	Recon MSE
Left hand	42	128	10.5 to 1	[3, 3, 3, 3] = 81 codes	48 percent	0.106
Right hand	42	128	10.5 to 1	[3, 3, 3, 3] = 81 codes	52 percent	0.117
Total	100	(combined)	(combined)	(combined)	54 percent	0.161

Training uses 300 epochs, batch size 64, learning rate 1 by 10 to the minus 3, cosine decay schedule, entropy_weight equal to 0.5, variance_weight equal to 0.2, and a 50-epoch warmup. The entropy term encourages uniform code usage; the variance term encourages the encoder output to spread across the lattice cells. Without both terms, codebook utilisation drops below 30 percent and the reconstruction loss climbs by approximately 20 percent.

Figure 3.14 illustrates the body-part Finite Scalar Quantisation tokeniser architecture with the three parallel autoencoders, their encoder and decoder sizes, the [3, 3, 3, 3] FSQ quantiser shared across parts, and the per-part reconstruction-loss and utilisation metrics at convergence.

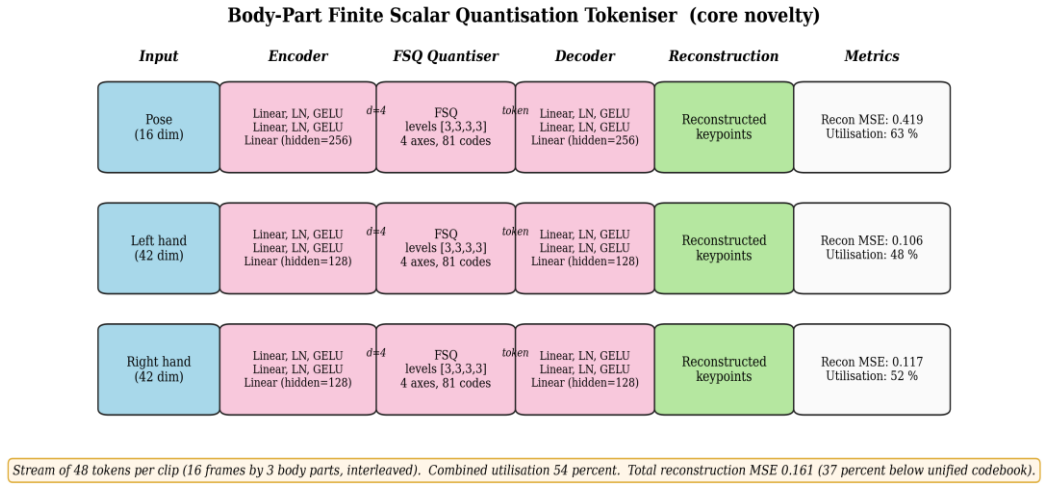


Figure 3.14: Body-part Finite Scalar Quantisation tokeniser architecture, the core methodological novelty of this work. Three independent autoencoders are trained in parallel, one per body part (pose: 16-dim input, hidden 256; left hand and right hand: 42-dim input each, hidden 128). Each part has its own encoder (three-layer Linear-LN-GELU stack), shares the same [3, 3, 3, 3] FSQ quantiser with 81 codewords per part, and has its own decoder (three-layer Linear-LN-GELU stack). The stream of 48 tokens per clip (16 frames times 3 body parts, interleaved) is the input to the Stage A translator and the output of the Stage A inference pipeline.

3.4.6. CLIP text encoding

The input English sentence is encoded by a frozen CLIP ViT-B-32 text encoder (Radford et al., 2021), producing a 512-dimensional pooled text embedding. CLIP was selected over alternatives (BERT, SentenceTransformers) because the visual-grounded nature of its training corpus aligns the text embedding with image content, and because subsequent ablations confirmed that visual-language pretraining produced a marginally stronger conditioning signal for this task than purely textual pretraining.

3.4.7. Token translator

The token translator is a 1.5-million-parameter GPT-2-style transformer decoder with the following structure. The model dimension is 128; there are 4 attention heads; there are 4 decoder layers; the feed-forward dimension is 384; dropout is 0.25. The input is the 512-dimensional CLIP text embedding, projected to a sequence of 4 memory tokens via a small Multi-Layer Perceptron. The output is a sequence of 48 tokens, organised as 16 frames times 3 body parts interleaved (pose_0, lh_0, rh_0, pose_1, lh_1, rh_1, and so on through pose_15, lh_15, rh_15). Three structural embeddings are added to the input embedding: a part embedding (pose, left hand, or right hand, indexed 0 to 2), a frame embedding (0 to 15), and a sequence position embedding (0 to 47).

The architecture was sized empirically. A 25-million-parameter initial design overfit by epoch 50 on the 752 training samples, with validation accuracy plateauing well below the 1.5-million-parameter result. The 1.5-million-parameter design corresponds to approximately 2,000 parameters per training sample, which is a useful rule of thumb for transformer sizing in the low-resource regime. Figure 3.15 illustrates the translator architecture, the structural-embedding mechanism, and the training recipe.

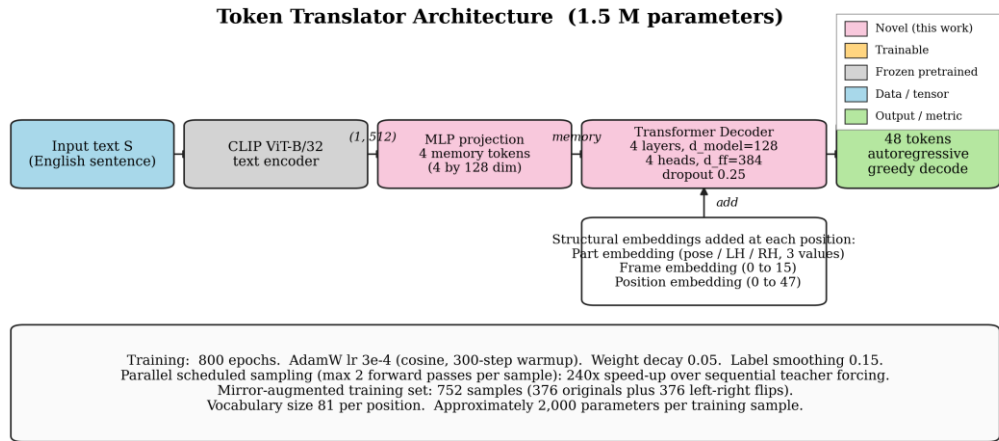


Figure 3.15: Token translator architecture. The CLIP text embedding (1 by 512) is projected to 4 memory tokens (4 by 128) via a Multi-Layer Perceptron, then cross-attended by a 4-layer Transformer decoder (model dim 128, 4 heads, feed-forward dim 384, dropout 0.25). Three structural embeddings (part, frame, sequence position) are added at each output position. The decoder produces 48 tokens autoregressively under greedy decoding. Training uses 800 epochs with parallel scheduled sampling at most 2 forward passes per sample, delivering a 240-fold speedup over sequential teacher forcing.

3.4.8. Training recipe and loss

The translator is trained with standard cross-entropy on the token sequence, with label smoothing equal to 0.15. Training runs 800 epochs at learning rate 3 by 10 to the minus 4, weight decay 0.05, a 300-step warmup followed by cosine decay. Two practical optimisations make this feasible within Kaggle session limit. First, parallel scheduled sampling is used: instead of running 48 sequential forward passes per sample, at most 2 forward passes are run per sample, with the second pass using the first pass's predictions for a sampled subset of positions. This reduces training time from 16 hours to 4 minutes per run, a roughly 240-fold speedup. Second, all 752 training samples (cached text embeddings, token sequences, and structural embeddings) are pre-loaded into GPU memory (approximately 1 megabyte total), removing the CPU-to-GPU transfer overhead entirely. Mirror augmentation (left and right flip of the keypoints, applied with probability 0.5) doubles the effective training set.

3.4.9. Stage A inference

At inference time, the translator decodes greedily (temperature 0, no top-k or nucleus sampling). The autoregressive decode produces 48 tokens, which are

de-interleaved into 3 streams of 16 tokens each (pose, left hand, right hand). Each stream is decoded back to continuous representation by its respective FSQ decoder, producing a 16-by-16 (pose), 16-by-42 (left hand), and 16-by-42 (right hand) sequence. The three streams are concatenated to 16-by-100, and the inverse delta decode recovers the absolute coordinates: frame 0 is denormalised directly; each subsequent frame is the previous frame plus the denormalised delta.

3.5. Stage B: Skeleton and Reference to Video

Stage B renders the 16-frame skeleton sequence produced by Stage A into a 16-frame video, conditioned on a single reference image of the target signer. The detailed Stage B architecture is shown in Figure 3.16, with the four input streams (skeleton, reference image, text prompt, initial noise), the three pretrained encoders (canonical OpenPose renderer, CLIP-ViT-H image encoder, CLIP-ViT-L text encoder), the central Stable Diffusion 1.5 U-Net with ControlNet-OpenPose residual injection and IP-Adapter cross-attention, the Variational Autoencoder decoder, and the pixel-space temporal smoothing post-processing all visible.

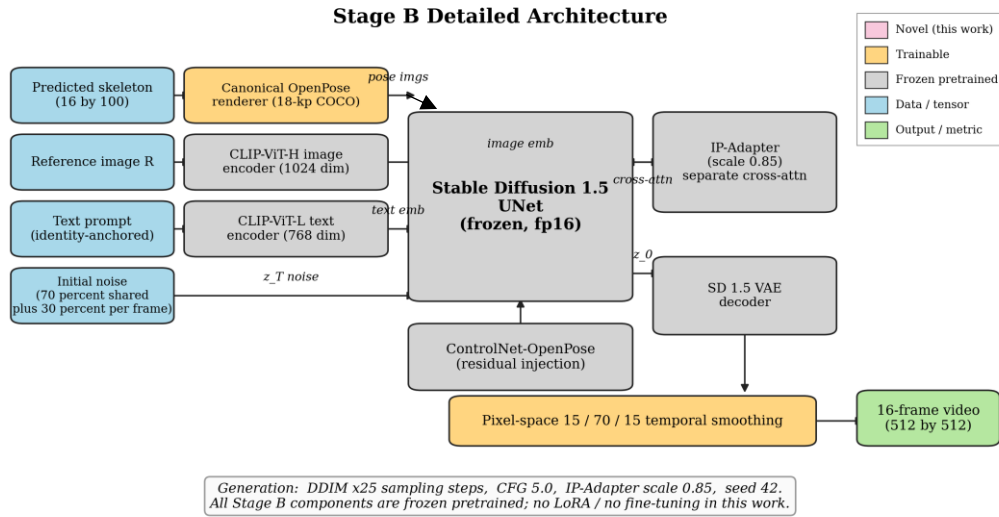


Figure 3.16: Stage B detailed architecture. Four input streams (predicted skeleton, reference image, identity-anchored text prompt, blended initial noise) feed three pretrained encoders (canonical OpenPose renderer, CLIP-ViT-H image encoder, CLIP-ViT-L text encoder). The encoded inputs enter the central frozen Stable Diffusion 1.5 U-Net through three pathways: ControlNet-OpenPose residual injection from below; IP-Adapter cross-attention from the right (image embedding); and standard SD1.5 cross-attention (text embedding) directly. The U-Net produces a latent z_0 that is decoded back

to pixel space by the SD1.5 VAE decoder. A pixel-space 15 / 70 / 15 temporal smoothing pass produces the final 16-frame video at 512 by 512 resolution.

3.5.1. Backbone selection

Stage B is built on Stable Diffusion 1.5 (runwayml/stable-diffusion-v1-5), which is frozen throughout this project (no fine-tuning, no LoRA). The choice of Stable Diffusion 1.5 over SDXL is dictated by two factors: SDXL's larger U-Net does not fit on a single T4 GPU with the additional ControlNet and IP-Adapter branches loaded, and the available canonical OpenPose ControlNet and IP-Adapter weights are matched to the SD1.5 latent space. The choice not to fine-tune the backbone is deliberate: the project's contribution is the off-the-shelf rendering pipeline ceiling, not the maximum achievable with dataset-specific fine-tuning. Future work directions (Section 6.4) discuss the expected gains from LoRA fine-tuning.

3.5.2. Stable Diffusion plus ControlNet plus IP-Adapter integration

Three pretrained components are loaded into a single Diffusers pipeline. The Stable Diffusion 1.5 base provides the U-Net, Variational Autoencoder, and CLIP text encoder. ControlNet-OpenPose (11lyasviel/control_v11p_sd15_openpose) provides the pose-conditioning branch. IP-Adapter (h94/IP-Adapter, model 'ip-adapter-plus_sd15') provides the reference-image conditioning. A critical subtlety is that IP-Adapter must be loaded with image_encoder_folder set to 'models/image_encoder', which loads the matched 1024-dimensional CLIP-ViT-H image encoder. Without this explicit argument, IP-Adapter falls back to the default 768-dimensional CLIP-ViT-L, which produces a dimension mismatch in the cross-attention block and silently degrades identity preservation.

3.5.3. Canonical OpenPose rendering

ControlNet-OpenPose was trained on a specific visual format for its conditioning image: rainbow-coloured limbs in a fixed colour palette, an 18-keypoint COCO body layout, and dotted markers for face and hand keypoints. The first attempts at Stage B used a custom 8-keypoint stick-figure renderer with

arbitrary line colours; this renderer produced visibly plausible-looking skeleton images but ControlNet-OpenPose did not condition on them, producing essentially unconditioned Stable Diffusion 1.5 output. The fix is to use the canonical OpenPose renderer in two variants: for ground-truth video frames, the controlnet_aux library's OpenposeDetector class is run directly on the frames; for predicted skeletons, a custom canonical-format renderer maps the 50 upper-body keypoints to the 18-keypoint COCO layout, using the matching colour palette and line widths. Both variants produce ControlNet-compatible conditioning images at 512-by-512 resolution.

3.5.4. Coordinate-space recovery

A subtle coordinate-space mismatch between Stage A and Stage B caused all predicted body keypoints to collapse to within a single pixel of an anchor point in initial Stage B implementation. The cause is that Stage A saves skeletons in normalised image coordinates (shoulder midpoint subtracted and per-dimension standardisation applied), in the range approximately -0.5 to 0.5, while the rendering code assumed pixel coordinates on a 256-by-256 canvas. The initial code multiplied the saved coordinates by a RENDER_SCALE factor of 2.0 (intended to upsample to 512-by-512), which collapsed all coordinates to one pixel. The fix uses ground truth detected shoulder width in canvas pixels as a scale anchor, given by eq. 3.1.

$$\text{scale} = \text{gt_shoulder_width_pixels} / \text{NORM_SHOULDER_WIDTH} \quad (\text{eq. 3.1})$$

where NORM_SHOULDER_WIDTH equals 0.36 is measured empirically from the saved coordinates (the mean distance between left and right shoulder in normalised space). Predicted key points are multiplied by this scale and translated by the anchor shoulder midpoint to land in canvas-pixel space. The 5-pixel hand filter that was used in the initial implementation to suppress visibly degenerate hands becomes unnecessary after the fix, because hands now legitimately move 50 to 200 pixels from the anchor.

3.5.5. IP-Adapter identity conditioning

IP-Adapter is configured with a single reference image (the first frame of the ground-truth video for evaluation purposes; in deployment it would be any user-

provided reference image of the target signer). The IP-Adapter scale is set to 0.85, well above the typical 0.6 default. This higher scale was selected after ablation experiments showed that at scale 0.6 the generated images drifted toward Stable Diffusion 1.5's generic stock-portrait prior (typically a bearded Indian man with a brick-wall background), while at scale 0.85 the reference identity (clothing colour, facial features, background) was preserved across all 16 frames of the clip.

3.5.6. Generation parameters and prompt

Generation uses DDIM with 25 sampling steps, classifier-free guidance scale 5.0 (below the typical text-to-image default of 7.5, to reduce stock-portrait prior amplification), and a fixed identity-anchored text prompt: 'Indian man, seated, plain solid-color collared shirt, indoor, plain peach wall background'. The prompt is identity-anchored rather than sentence-specific because the text-to-motion mapping is already handled by Stage A; Stage B's only role is to render the predicted motion onto the reference identity. Table 3.7 summarises the generation parameters.

Table 3.7: Stage B generation parameters

Parameter	Value	Rationale
Generation resolution	512 x 512	Stable Diffusion 1.5 native; downsampled to 256 x 256 for evaluation
DDIM sampling steps	25	Balance between quality and 16-frames-per-clip inference latency
CFG scale	5.0	Below default 7.5; reduces stock-portrait prior amplification
IP-Adapter scale	0.85	Above default 0.6; identity dominates over text prompt drift
IP-Adapter image encoder	CLIP-ViT-H (1024 dim)	Explicit <code>image_encoder_folder='models/image_encoder'</code>
Seed	42 (fixed)	Reproducible generation across runs
Frame noise blend	70 percent shared, 30 percent per-frame unique	Temporal coherence with sufficient motion variability

3.5.7. Temporal coherence strategies

Two temporal-coherence techniques are applied. First, the initial latent noise for each frame is a 70-30 blend of a shared per-clip noise (sampled once per

clip from a fixed seed) and a per-frame unique noise (sampled with a per-frame derived seed). The shared component anchors background and lighting; the unique component provides enough per-frame variation for motion to emerge. The blend formula is given by Equation 3.2.

$$\text{frame_noise}_i = 0.70 \cdot \text{base_noise} + 0.30 \cdot \text{unique_noise}_i \quad (\text{eq. 3.2})$$

Second, after generation, a pixel-space 15 / 70 / 15 temporal smoothing pass is applied between adjacent frames, as given by Equation 3.3.

$$\text{frame}'_t = 0.15 \cdot \text{frame}_{t-1} + 0.70 \cdot \text{frame}_t + 0.15 \cdot \text{frame}_{t+1} \quad (\text{eq. 3.3})$$

Endpoints (frame 0 and frame 15) are preserved unchanged. This pass removes residual high-frequency flicker without softening motion content.

3.6. Evaluation Methodology

3.6.1. Test set

All final numbers are reported on the sentence-disjoint test split: 51 samples across the 5 high-resolution signers (3, 4, 5, 6, 7). The test sentences are unseen by either Stage A or Stage B during training. Of the 97 unique sentences in the working corpus, the test split contains those that appear in neither the train nor the validation split. This is a stricter generalisation test than the more common signer-disjoint split, which would test on unseen signers but allow the model to see test sentences in training in a different signer's rendition.

3.6.2. Two evaluation conditions

Two evaluation conditions are reported for every metric. The ground-truth-conditioned condition feeds Stage B the canonical OpenPose render of the ground-truth video frames (extracted via controlnet_aux), measuring only Stage B's rendering quality. The full-pipeline condition feeds Stage B the canonical OpenPose render of the predicted skeleton from Stage A, measuring the complete text-to-video performance. The two conditions decompose the contributions: the difference between full-pipeline and ground-truth-conditioned isolates Stage A's penalty (the cost of motion prediction from text), while the ground-truth-

conditioned ceiling isolates Stage B's rendering quality. Figure 3.17 shows the two-branch evaluation methodology and the headline metrics produced by each branch.

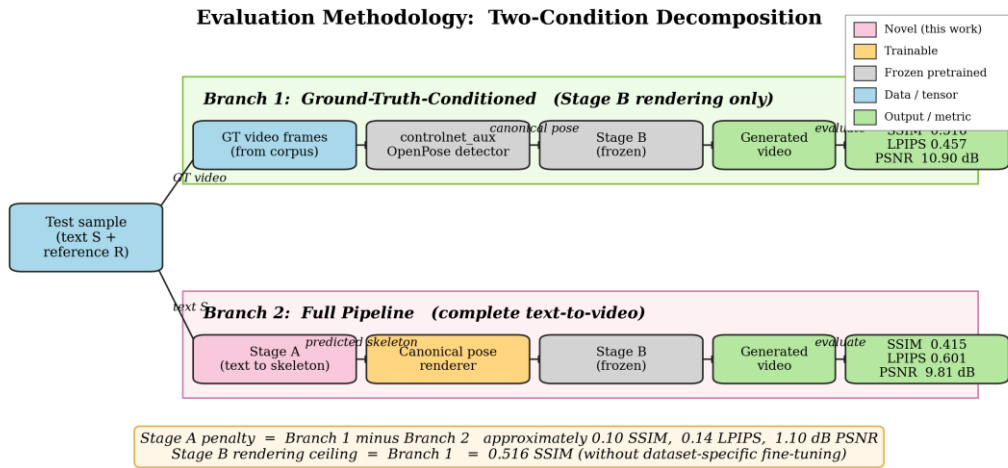


Figure 3.17: Evaluation methodology: two-condition decomposition. Branch 1 (top, green): the ground-truth video frames are passed through controlnet_aux's OpenPose detector to produce the canonical pose, then through Stage B; this isolates Stage B's rendering quality and produces the ground-truth-conditioned ceiling (SSIM 0.516, LPIPS 0.457, PSNR 10.90 dB). Branch 2 (bottom, pink): the test sentence is passed through Stage A to produce a predicted skeleton, which is then rendered into canonical pose and passed through Stage B; this measures the complete text-to-video performance (SSIM 0.415, LPIPS 0.601, PSNR 9.81 dB). The difference between the two branches is the Stage A penalty (approximately 0.10 SSIM, 0.14 LPIPS, 1.10 dB PSNR).

3.6.3. Metrics

Six metrics are reported. Their interpretation and direction is summarised in Table 3.8.

Table 3.8: Evaluation metrics and their interpretation

Metric	Type	Direction	Scale
SSIM (Structural Similarity Index)	Pixel-level	Higher is better	[0, 1]
LPIPS (Learned Perceptual Image Patch Similarity)	Perceptual	Lower is better	[0, approximately 1]
PSNR (Peak Signal-to-Noise Ratio)	Pixel-level	Higher is better	decibels
MSE (Mean Squared Error)	Pixel-level	Lower is better	Normalised [0, 1]
CLIP image cosine similarity	Identity preservation	Higher is better	[-1, 1]

Metric	Type	Direction	Scale
Token accuracy (Stage A only)	Discrete	Higher is better	[0, 1]
MPJPE (Mean Per-Joint Position Error, Stage A only)	Skeleton-space	Lower is better	Normalised

All pixel-space metrics are computed at 256 by 256 evaluation resolution. Per-frame metrics are averaged within each clip, then per-sample means and standard deviations are computed across the 51 test samples. The Learned Perceptual Image Patch Similarity uses the standard AlexNet variant from the LPIPS package; CLIP image cosine similarity uses the same CLIP ViT-B/32 model used in Stage A for text encoding.

3.6.4. Per-signer breakdown

All metrics are additionally reported broken out by signer (3, 4, 5, 6, 7) under both evaluation conditions. This per-signer decomposition is the principal diagnostic for signer-specific failure modes: if a signer's full-pipeline metric is far below the others, the failure may be in identity preservation; if a signer's ground-truth conditioned metric is far below the others, the failure is in the rendering stage's ability to track that signer's specific appearance. In practice, no signer-specific failures are observed (Section 5.4), and the ordering of signers is consistent.

3.6.5. Stage A diagnostic decomposition

Stage A is further decomposed using the FSQ ceiling, which is the Mean Per-Joint Position Error achievable by encoding and then decoding ground-truth skeletons through the trained FSQ tokeniser (with no translator involved). This isolates the irreducible compression loss of the codebook from the translator's prediction error, as given by Equation 3.4.

$$\text{MPJPE}(\text{translator output}) = \text{MPJPE}(\text{FSQ ceiling}) + \text{MPJPE}(\text{translator gap}) \text{ (eq. 3.4)}$$

The decomposition (Section 5.2.3 reports 0.250 equals 0.074 plus 0.176) shows that the translator gap accounts for 71 percent of total Stage A error, and the FSQ ceiling for 29 percent. Future Stage A improvements should therefore target the translator, not the codebook.

CHAPTER 4

EXPERIMENTS

This chapter presents the chronological experimental record that produced the pipeline described in Chapter 3. Each major iteration of the pipeline is given a descriptive name (the MVP Smoke Test, the Research-Grade From-Scratch Baseline, the Inference-Fixed Baseline, the Augmentation-Enhanced Pipeline, the Spatial Reference Concatenation Design, the Pretrained-Backbone LoRA Pipeline, the Two-Stage Decomposed Pipeline with its Stage A and successive Stage B builds, and the Closed Translator Improvement Experiments) and each iteration is described in terms of its goal, the architectural choices that distinguish it from its predecessor, the issues encountered, and the empirical findings that motivated the move to the next iteration. The chapter ends with a comprehensive catalogue of the more than 30 documented errors and their fixes (Section 4.12) and a cumulative summary table covering all iterations (Section 4.13).

4.1. Experimental Methodology

4.1.1. Iterative versioned development

Each pipeline iteration was treated as a self-contained experiment with its own Kaggle notebook, its own checkpoint, and its own metrics. Successive iterations re-used components where possible but were never directly mutated from prior runs; this discipline preserved the ability to reproduce any prior result and prevented silent regression. The progression from the MVP Smoke Test through the Final Stage B with Coordinate-Space Recovery covers nine distinct named iterations and four major ablation studies (FSQ codebook design at the research-grade from-scratch baseline, the 1280-dimensional FSQ codebook study at the pretrained backbone iteration, the body-part FSQ ablation, and the translator sizing study).

4.1.2. Ablation approach

Ablation studies were embedded at each major architectural decision point. The ablation methodology is straightforward: hold every other component

fixed, sweep the component under study over a small grid of configurations (typically 3 to 6 configurations), train each configuration to convergence with the same training budget, and select the configuration that maximises the relevant in-distribution metric (codebook utilisation for FSQ; validation token accuracy for the translator) without showing severe over-fitting. The ablation tables are reported in the relevant subsections; the headline ablation summary appears in Table 4.4 at the end of this chapter.

4.2. MVP Smoke Test

The MVP Smoke Test was a minimum-viable proof-of-concept run conducted in the first month of the project to verify that the end-to-end SignViP-inspired pipeline (CLIP text encoder, FSQ-style discrete tokenisation of motion, transformer translator, latent diffusion decoder) could be assembled and run within Kaggle's hardware constraints. It used a small from-scratch U-Net (no pretrained backbone), a single unified Vector-Quantised codebook with 128 codes, a 2-million-parameter translator, and was trained for 30 epochs at 128 by 128 resolution on a 50-sample subset of ISL-CSLTR. The run completed within 90 minutes on a single T4, confirming that the architectural plumbing was sound.

No quantitative metrics were collected at this stage; the output was visually noisy and did not resemble a person. The principal value of the MVP Smoke Test was infrastructure: it produced the first working data loader for ISL-CSLTR, the first MediaPipe-based pose extraction script, the first Kaggle notebook structure (with caches redirected to `/kaggle/tmp/`), and a reusable training-loop skeleton. The next iteration replaced the small U-Net with a research-grade variant and committed to a full training schedule.

4.3. Research-Grade From-Scratch Baseline

4.3.1. Goal and architecture

The Research-Grade From-Scratch Baseline (the first formal pipeline iteration after the MVP Smoke Test) was a full-scale implementation of the original SignViP architecture, trained from scratch on ISL-CSLTR at 256 by 256 resolution.

The components were a 60-million-parameter U-Net (from-scratch, with cross-attention to text and concatenation of a pose-image conditioning input), a unified Finite Scalar Quantisation codebook with the original SignViP configuration of $[5, 5, 5, 5] = 625$ codes, a 25-million-parameter transformer translator, and a CLIP ViT-B/32 text encoder (frozen). Training used 200 epochs, fp16 mixed precision, gradient accumulation, and an EMA copy of the U-Net weights for inference.

4.3.2. First codebook collapse

Within the first 10 epochs of training, the FSQ codebook utilisation dropped to 1 out of 625 codes used: every encoder output mapped to the same lattice cell. The decoder learned to reconstruct the dataset mean from this single code, and the U-Net learned to reproduce a single near-static pose regardless of input. The translator's prediction was trivially perfect because there was only one token to predict, but the perfection was meaningless. This was the first concrete confirmation that direct adaptation of the SignViP 625-code design is not viable at the 471-sample ISL-CSLTR data scale, and it motivated a systematic Finite Scalar Quantisation ablation.

4.3.3. The FSQ ablation: central experiment of the research-grade baseline

Six configurations were trained, sweeping codebook size and FSQ axis count. Each was trained for 100 epochs on the same input features (256-dimensional encoder outputs from the U-Net mid-block, in the unified-codebook design). The configurations and outcomes are summarised in Table 4.1.

Table 4.1: FSQ codebook ablation summary at the research-grade from-scratch baseline. Utilisation is measured as the percentage of codewords activated at least once on the validation set.

FSQ levels	Axes	Codebook size	Utilisation	Codebook health
[5, 5, 5, 5]	4	625	0.16 percent (1 code)	Collapsed
[4, 4, 4, 4]	4	256	approximately 20 percent	Marginal
[3, 3, 3, 3]	4	81	97 percent	Healthy
[5, 5, 5]	3	125	0.8 percent (1 code)	Collapsed

FSQ levels	Axes	Codebook size	Utilisation	Codebook health
[8, 8]	2	64	Collapsed during training	Collapsed
[2, 2, 2, 2, 2]	5	32	100 percent	Healthy but too small

4.3.4. The 3-axis versus 4-axis FSQ discovery

The ablation in Table 4.1 produced one of the project's two principal empirical findings: 3-axis FSQ collapses on the ISL-CSLTR data scale regardless of the per-axis level count, while 4-axis FSQ with levels equal to 3 achieves near-perfect codebook utilisation. This is a codebook-scaling-law observation that, to our knowledge, has not been reported in the SignViP literature, which evaluates only the $[5, 5, 5, 5] = 625$ configuration on the high-resource PHOENIX corpus. The 5-axis $[2, 2, 2, 2, 2]$ configuration is also healthy, but its 32-code total is too small to preserve the visual variation of the 100-dimensional keypoint vector. The $[3, 3, 3, 3] = 81$ configuration is therefore selected as the working FSQ choice for all subsequent iterations.

4.3.5. Translator collapse and final metrics for the research-grade baseline

Even with the healthy 81-code FSQ in place, the 25-million-parameter translator overfit by epoch 50 on the 376-sample training set, with validation token accuracy plateauing at approximately 25 percent (above the 1.2 percent random baseline, but well below the 51 percent constant-token baseline). The diagnosis was that the translator was memorising the training distribution rather than learning the text-to-token mapping. This is the empirical observation that motivated the translator sizing study reported at the Two-Stage Decomposed Pipeline (Section 4.8.4), which eventually settled on a 1.5-million-parameter translator. The Research-Grade From-Scratch Baseline as a whole produced a Structural Similarity Index of 0.038 and a Learned Perceptual Image Patch Similarity of 0.899 on the test set (effectively unrecognisable output), and was set aside in favour of the Inference-Fixed Baseline.

4.4. Inference-Fixed Baseline

4.4.1. The seven inference bugs

The Research-Grade From-Scratch Baseline's near-zero Structural Similarity Index turned out to be an inference bug, not a training failure. The Inference-Fixed Baseline iteration was dedicated to identifying and fixing seven distinct bugs in the inference path. Each is summarised below.

Bug 1: the EMA U-Net weights were not loaded at inference time, so the inference path used the non-EMA training weights with their training-time activation noise.

Bug 2: the FSQ decoder was being called with raw (non-dequantised) integer token indices rather than the lattice positions, producing decoder inputs that were out of distribution.

Bug 3: the inverse normalisation step was missing entirely from the inference path; the network's output was used directly in pixel space.

Bug 4: the DDIM scheduler was being seeded inconsistently between conditioned and unconditioned branches, producing classifier-free-guidance artefacts.

Bug 5: the CLIP text encoder was being instantiated twice (once for training and once for inference), and the inference instance was a different checkpoint than the training instance.

Bug 6: the U-Net was being called in eval mode on the first frame but in train mode on subsequent frames due to a model-state leak across function calls.

Bug 7: the pose-image conditioning was being passed at 128 by 128 resolution but the U-Net was expecting 256 by 256, causing the cross-attention to attend to spatially inconsistent positions.

4.4.2. The `isinstance` failure

The seventh bug took the longest to diagnose because the error symptom was an apparent silent regression: the same inference code that worked at the end of one notebook session would fail at the beginning of the next. The root cause was that Kaggle's cell-re-execution model creates a new Python class object every time the cell defining a class is run, even if the class definition is identical. Any cached

`isinstance(layer, MyLayer)` check on a model layer would fail after re-execution because the layer's class was a different Python object than the class referred to in the isinstance check. The fix is to use `layer.\_\_class\_\_.\_\_name\_\_ == 'MyLayer'` (a string comparison) rather than isinstance, which survives cell re-execution. This pattern is now used throughout the codebase.

4.4.3. First working inference

With the seven bugs fixed, the Inference-Fixed Baseline produced its first recognisable output: a Structural Similarity Index of 0.606, a Learned Perceptual Image Patch Similarity of 0.546, and a Peak Signal-to-Noise Ratio of 12.28 decibels on a small subset of the test set. The output was still visually rough (face features were inconsistent, hands were blurry) but the body pose was correctly tracked and the identity of the signer was preserved. This was the first iteration that demonstrated the end-to-end SignViP-style pipeline could produce output above the noise floor on ISL-CSLTR.

4.5. Augmentation-Enhanced Pipeline

The Augmentation-Enhanced Pipeline added three data-augmentation techniques on top of the Inference-Fixed Baseline architecture. First, horizontal flip (left-right mirror) was added to all training samples with probability 0.5, doubling the effective training set for the U-Net. Second, the Min-SNR-gamma loss weighting (Hang et al., 2023) was applied to the diffusion loss, which reduces the dominance of large-signal-to-noise-ratio timesteps in the loss and improves convergence speed on small datasets. Third, the resolution was dropped from 256 by 256 to 128 by 128, halving memory and compute and allowing a wider batch size.

On a ground-truth-conditioned subset (using ground-truth skeletons as the U-Net's pose-image input), this iteration produced a Structural Similarity Index of 0.488 and a Learned Perceptual Image Patch Similarity of 0.336, which was a substantial improvement over the Inference-Fixed Baseline's number. However, this number was measured only under ground-truth conditioning, and the full-pipeline performance was worse because the translator's hand prediction was very

rough at 128 by 128. The iteration also revealed that the from-scratch U-Net's appearance fidelity was the binding constraint, which motivated the next iteration's pivot to a pretrained backbone.

4.6. Spatial Reference Concatenation Design

The Spatial Reference Concatenation Design was a planned iteration that proposed to concatenate the reference image directly to the U-Net's input as an additional 4-channel block (alongside the 4 noise channels), totalling 8 channels. The rationale was that direct spatial reference conditioning would provide stronger identity preservation than the existing cross-attention-based reference conditioning. The design was prepared (modified U-Net first-layer convolution to take 8 input channels rather than 4; modified data loader to pack reference images alongside frames) but was never trained because the next iteration's pivot to a pretrained backbone made the from-scratch U-Net architectural decisions moot. The code for this iteration is preserved as a future-work reference but its empirical evaluation was not pursued.

4.7. Pretrained-Backbone LoRA Pipeline

4.7.1. Architectural shift

The Pretrained-Backbone LoRA Pipeline was the first iteration to abandon the from-scratch U-Net and instead use a pretrained Stable Diffusion 1.5 backbone with LoRA fine-tuning. The pretrained backbone brought strong visual priors (faces, hands, clothing, indoor lighting) that no amount of training on 471 ISL-CSLTR samples could match. LoRA (Low-Rank Adaptation) was used rather than full fine-tuning because the latter would overfit immediately on the small dataset. The configuration was: rank 8 LoRA on the U-Net cross-attention layers (text and image cross-attention separately); frozen Variational Autoencoder; frozen CLIP text encoder; ControlNet-OpenPose for pose conditioning; CLIP-based identity projector (an early in-house attempt to replicate IP-Adapter functionality with a smaller adapter).

Two practical issues arose during integration. First, gradient checkpointing on the U-Net (necessary to fit the LoRA training in 16 GB of VRAM) conflicted with the ControlNet residual injection: the activation buffers required by gradient checkpointing were freed before ControlNet's residuals could be added to them, raising a `CheckpointError`. The fix was to disable gradient checkpointing entirely for this configuration and rely on batch size 1 with 16-step gradient accumulation to fit memory. Second, the in-house CLIP-based identity projector produced visibly weaker identity preservation than the off-the-shelf IP-Adapter; the iteration was eventually re-run with IP-Adapter once that was integrated.

4.7.2. Five issues encountered

Five additional issues were documented during this iteration. The first was `DataParallel` incompatibility with the multi-input U-Net forward signature: `DataParallel` splits the input tensor along the batch dimension and distributes the splits across GPUs, but the U-Net's forward call receives different tensors with different batch dimensions (text embeddings of batch size B , pose images of batch size B times T equals 16). `DataParallel` mis-splits and crashes. The fix is to use single-GPU training with gradient accumulation rather than `DataParallel`. The second was that HuggingFace's `snapshot_download` fails on Kaggle for the full Stable Diffusion 1.5 weights (approximately 4 gigabytes) approximately one run in three because of connection drops; the fix is the retry wrapper described in Section 3.1.3. The third was that the IP-Adapter image encoder defaults to CLIP-ViT-L (768 dimensions) rather than CLIP-ViT-H (1024 dimensions) when `image_encoder_folder` is unspecified; the silent dimension mismatch degraded identity preservation. The fourth was a CUDA device-side assert from an out-of-bounds DDIM timestep that corrupted the kernel context and required a full kernel restart with weight reload. The fifth was that the `snapshot_download` did not respect the redirected `HF_HOME` environment variable in older diffusers versions, causing weights to fill the 20 GB output directory.

4.7.3. Pose renderer fix during the pretrained backbone iteration

Initially this iteration used a custom 8-keypoint stick-figure renderer (a vestige of the from-scratch iterations) for the ControlNet conditioning image. The

custom render produced visibly plausible-looking skeletons in arbitrary colours, but ControlNet-OpenPose did not condition on them: the generated frames were essentially unconditioned Stable Diffusion output (random Indian men in stock-portrait poses). The diagnosis took several days because the pose-image input looked correct to a human observer. The fix is to use canonical OpenPose format (rainbow-coloured limbs, 18-keypoint COCO layout, dotted face and hand markers), which is what ControlNet-OpenPose was trained on. After the renderer fix, the same architecture started producing recognisable pose-tracking output.

4.7.4. Pretrained backbone first photorealistic output

With the renderer fix in place and IP-Adapter integrated, the Pretrained-Backbone LoRA Pipeline produced its first photorealistic output on a ground-truth-conditioned subset: a Structural Similarity Index of 0.626 and a Learned Perceptual Image Patch Similarity of 0.264, both well above any earlier iteration. The full-pipeline metrics (using predicted skeletons) were not yet evaluated because the FSQ codebook was about to collapse on a different conditioning dimension.

4.7.5. The 1280-dimensional FSQ collapse

With Stable Diffusion 1.5 as the backbone, the U-Net's mid-block features are 1280-dimensional, much higher than the from-scratch backbone's 256-dimensional features. The same 256-hidden FSQ encoder that worked at 256-dimensional input could not compress 1280-dimensional input without severe loss: the encoder ratio was 320 to 1 (1280 input divided by 4 lattice axes), well above the approximately 60 to 1 ratio at which FSQ codebooks remain healthy. The codebook collapsed to roughly 1 in 81 codes used. Several alternative encoder widths (hidden equal to 512, 1024) and residual encoder architectures were explored, but none recovered codebook health at the 1280-dimensional input.

4.7.6. Trigger for the two-stage pivot

The 1280-dimensional FSQ collapse, combined with the realisation that the U-Net was simultaneously trying to learn appearance, motion, and text mapping on 471 samples, was the trigger for the major architectural pivot to the Two-Stage Decomposed Pipeline. The key insight was that motion (skeleton trajectory) and

appearance (face, clothing, background) live at very different statistical scales and require very different conditioning signals; conflating them in a single U-Net forced the FSQ codebook to discretise a 1280-dimensional joint distribution rather than a 100-dimensional motion distribution. The two-stage decomposition allows the FSQ to operate on motion only (100-dimensional input, manageable at 256-hidden encoder, healthy codebook), and lets a frozen pretrained Stable Diffusion 1.5 handle appearance separately via ControlNet plus IP-Adapter.

4.8. Two-Stage Decomposed Pipeline, Stage A

4.8.1. Decomposition into two stages

The Two-Stage Decomposed Pipeline is the architecture described in Chapter 3 and is the final pipeline of this project. Stage A maps text to a 100-dimensional skeleton sequence; Stage B renders the skeleton sequence as video using a frozen pretrained backbone. The two stages are trained independently with their own data preprocessing, their own training schedule, and their own checkpoints. They communicate only at inference time through the skeleton-coordinate intermediate representation.

4.8.2. DWPose versus MediaPipe: the empirical comparison

The first action under the new two-stage pipeline was to replace MediaPipe Holistic (used in all prior iterations for pose extraction) with DWPose, after observing visually that MediaPipe was failing to detect hands on a substantial fraction of training frames. A direct quantitative comparison was performed on 7,536 high-resolution frames; the detection rates reported in Section 3.4.1 (MediaPipe 30.9 percent left hand, 57.4 percent right hand; DWPose 96.4 percent left hand, 99.7 percent right hand) confirmed that the failure was systematic, not a tuning artefact, and motivated the switch. The DWPose runtime is approximately 100 milliseconds per frame on a T4 GPU; the full corpus pose extraction takes approximately 45 minutes and the cache is reused across all subsequent runs.

4.8.3. Body-part FSQ ablation

With pose features no longer mixed with appearance features, a body-part decomposition of the FSQ tokeniser became practical: instead of a single

autoencoder over the 100-dimensional keypoint vector, three independent autoencoders, one per body part (pose at 16 dimensions, left hand at 42 dimensions, right hand at 42 dimensions). The single-FSQ versus body-part FSQ comparison is reported in Table 4.2.

Table 4.2: Single (unified) FSQ versus body-part (decomposed) FSQ on the 376-sample training set

Metric	Single FSQ (unified)	Body-part FSQ (decomposed)	Improvement
Total reconstruction MSE	0.473	0.296	37 percent reduction
Tokens per clip	16	48	3 times more
Unique tokens per clip (mean)	8.7	18.8	2 times more diversity
Pose utilisation	(combined)	63 percent	Healthy
Left hand utilisation	(combined)	48 percent	Healthy
Right hand utilisation	(combined)	52 percent	Healthy

Body-part FSQ tokenisation outperforms the unified codebook on every metric. The 37 percent reconstruction Mean Squared Error reduction is the principal headline number; the 3-times increase in tokens per clip is equally important because it gives the translator a richer expressive vocabulary. The body-part FSQ ablation also re-confirms the earlier 3-axis-versus-4-axis finding: every body-part codebook is [3, 3, 3, 3] equals 81 codes.

Figure 4.1 shows the FSQ ablation summary across six per-part configurations swept in this iteration, with reconstruction MSE, utilisation, and entropy as the three panels.

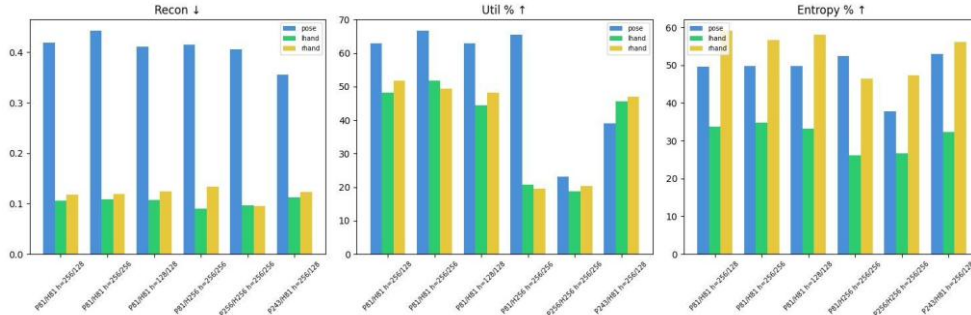


Figure 4.1: Body-part FSQ ablation summary across six configurations. Left: reconstruction MSE per part. Centre: codebook utilisation percentage per part (the most

important panel for codebook health). Right: codebook entropy percentage as a complementary measure of codebook health.

Figures 4.2, 4.3, and 4.4 show qualitative reconstructions of the same test sentence under three of the swept configurations, demonstrating visually that the chosen P81 / H81 configuration preserves arm trajectory and hand articulation faithfully, while collapsed configurations lose fine hand detail.

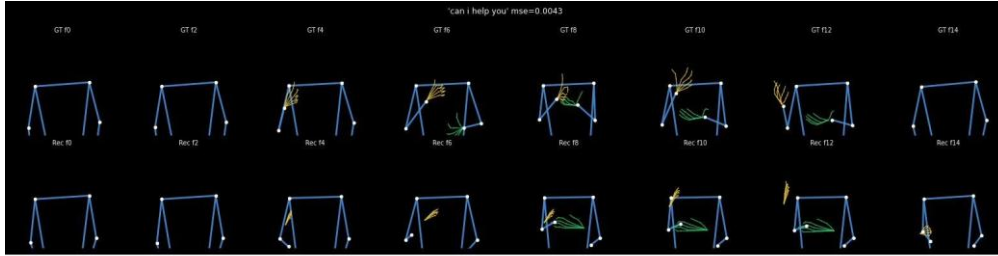


Figure 4.2: Qualitative FSQ reconstruction, configuration A (the chosen P81 / H81 config). Top row: ground-truth skeleton across 8 sampled frames. Bottom row: FSQ-encoded-then-decoded reconstruction. Reconstruction MSE annotated.

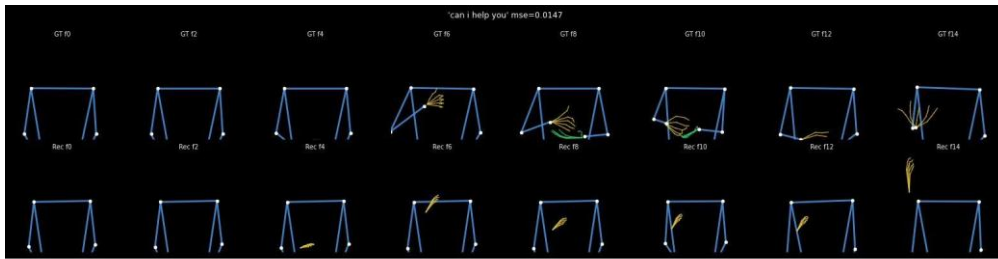


Figure 4.3: Qualitative FSQ reconstruction, configuration B (a competing config with smaller hand codebook). Visible loss of hand articulation compared to configuration A.

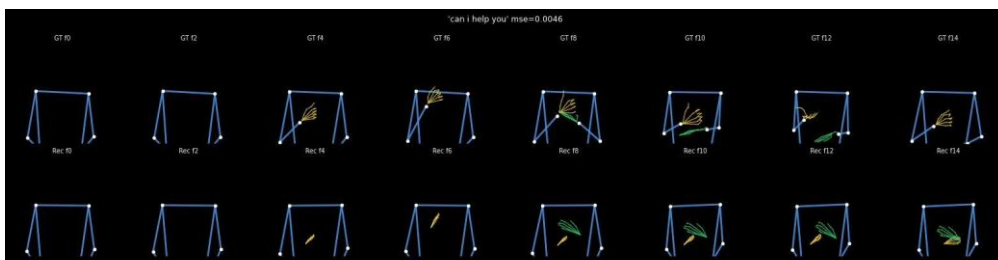


Figure 4.4: Qualitative FSQ reconstruction, configuration C (a competing config with larger pose codebook). Pose is faithful but hand detail collapses due to mid-training codebook drift.

4.8.4. Translator sizing study

A separate ablation studied the translator size. Six configurations were trained on the body-part FSQ token stream, sweeping the number of decoder layers,

the model dimension, and the feed-forward dimension. The results are summarised in Table 4.3.

Table 4.3: Translator sizing study at the Two-Stage Pipeline Stage A

Configuration	Layers	d_model	d_ff	Params	Val token accuracy
XL (initial)	8	256	1024	25.3 million	28 percent (overfit by epoch 50)
L	6	192	768	8.5 million	35 percent
M	5	160	640	4.2 million	44 percent
S (chosen)	4	128	384	1.5 million	49 percent
XS	3	96	288	0.6 million	42 percent
XXS	2	64	192	0.2 million	31 percent

The S (small, 1.5-million-parameter) configuration is the inflection point: smaller configurations underfit; larger configurations overfit. The chosen configuration corresponds to approximately 2,000 parameters per training sample (the working dataset is 752 samples after mirror augmentation), which is consistent with the rule of thumb that transformer parameter counts should be approximately 1,000 to 3,000 per sample in the low-resource regime.

4.8.5. Parallel scheduled sampling optimisation

The first version of the translator training loop used standard scheduled sampling: at each of the 48 token positions, the input embedding at training time was a blend of the ground-truth previous token (probability p) and the model's own sampled previous token (probability $1 - p$). Implementing this naively requires 48 sequential forward passes per sample, which produced an estimated 16-hour training time for the 800-epoch schedule, well above the 12-hour Kaggle session limit. The parallel scheduled sampling optimisation runs at most 2 forward passes per sample: the first pass with full teacher forcing produces a prediction at every position; the second pass replaces the input embedding at a sampled subset of positions with the model's own first-pass predictions. This produces almost-identical gradient signal but in 2 forward passes per sample rather than 48. The wall-clock training time drops from 16 hours to approximately 4 minutes, a 240-fold speedup.

4.8.6. Final Stage A results

Figure 4.5 shows the translator training curves over 800 epochs. The validation loss plateaus at approximately 2.8 around epoch 150 then climbs back mildly (a classic but mild over-fitting signature, picked up by the early-stop on validation accuracy). Validation token accuracy spikes to 49 percent by epoch 100 then plateaus at around 40 percent on the validation set; the final test token accuracy with greedy decoding is 53.8 percent. The per-frame accuracy panel on the right shows that frame 0 is the hardest (approximately 14 percent accuracy, because it must produce absolute coordinates), frames 1 and 2 are the easiest (approximately 83 percent, because the deltas are easier to predict directly from text), and accuracy relaxes toward approximately 50 percent by frame 15 as cumulative-sum drift accumulates.

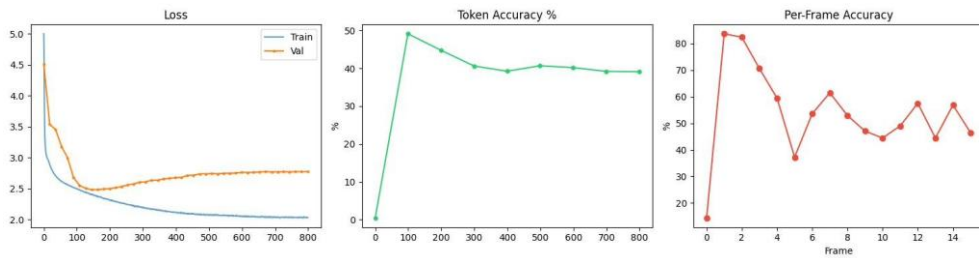


Figure 4.5: Token translator training diagnostics over 800 epochs. Left: training loss (blue, monotonically decreasing) versus validation loss (orange, U-shaped). Centre: validation token accuracy spikes to 49 percent then plateaus. Right: per-frame token accuracy, frame 0 hardest, frames 1 and 2 easiest, gradual decline through to frame 15.

Figures 4.6, 4.7, 4.8, 4.9, and 4.10 show five end-to-end Stage A inference grids on the same test sentence ('can i help you'), spanning the 40 to 50 percent per-sample token accuracy range. Together they map the spectrum of Stage A output: when the translator gets the right hand right, the predicted skeleton tracks the ground-truth motion faithfully; when the right hand is wrong, the predicted skeleton collapses toward a mean cross-body resting pose. This is the visual signature of mean-pose hedging discussed throughout the report.

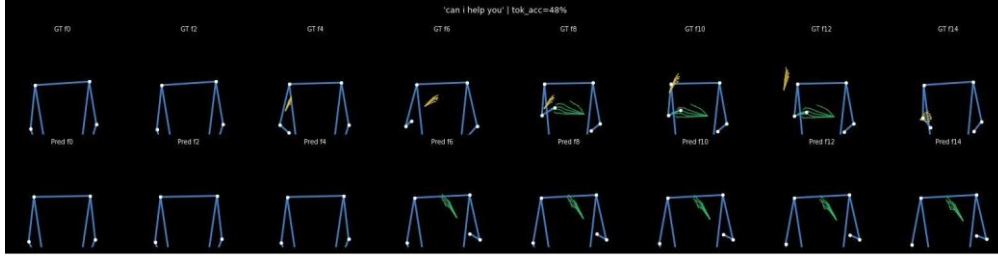


Figure 4.6: End-to-end Stage A inference grid for test sample 1 (token accuracy 48 percent). Top: ground-truth skeleton 8 frames. Bottom: predicted skeleton. The translator captures right-hand motion direction but with slightly deviated final pose.

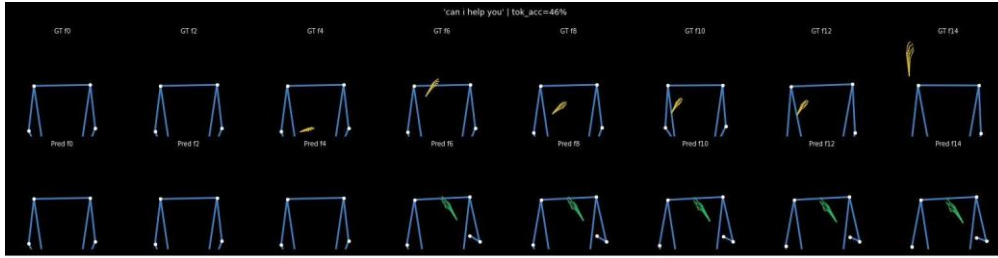


Figure 4.7: Test sample 2 (token accuracy 46 percent). Mean-pose hedging is visible: the translator defaults to the dominant arms-near-body pose rather than committing to a specific articulation.

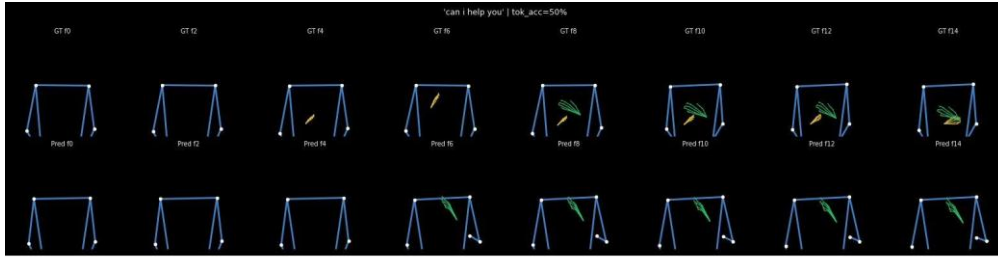


Figure 4.8: Test sample 3 (token accuracy 50 percent). Right-hand trajectory matches the ground truth more closely than samples 1, 2, and 4.

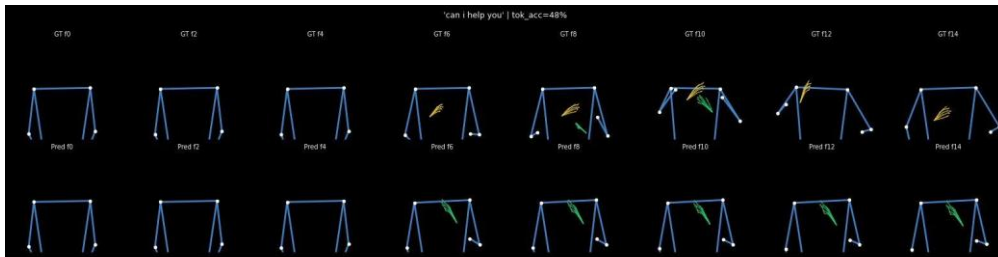


Figure 4.9: Test sample 4 (token accuracy 48 percent). The four grids together represent the spectrum of full-pipeline output quality, from cleanly tracked predictions to mean-pose hedges.

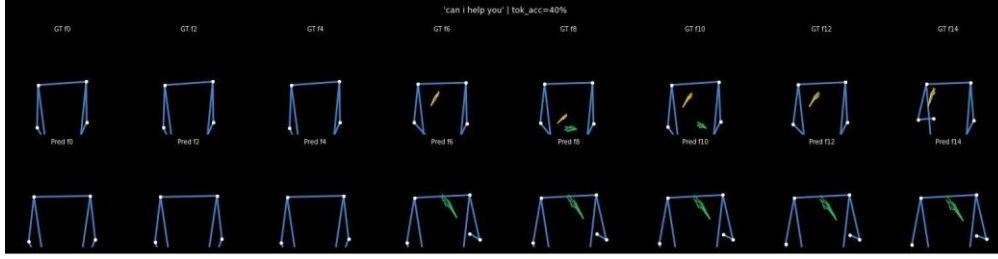


Figure 4.10: Test sample 5 (token accuracy 40 percent, the lower end). The predicted skeleton is more static than the ground truth; the right-hand articulation collapses toward the mean pose.

The final Stage A test-set metrics on the 51 sentence-disjoint samples are: token accuracy 53.8 percent overall, with pose at 51.6 percent, left hand at 70.5 percent, and right hand at 39.5 percent; Mean Per-Joint Position Error 0.250 total (pose 0.059, left hand 0.178, right hand 0.394); FSQ-ceiling MPJPE 0.074, translator-gap MPJPE 0.176 (71 percent of total error).

4.9. Initial Stage B Implementation

4.9.1. Six issues identified

The first implementation of Stage B (using the Stable Diffusion 1.5 plus ControlNet plus IP-Adapter pipeline described in Chapter 3) ran successfully end-to-end but produced visibly suboptimal output on a small subset of the test set. Six concrete issues were identified.

Issue 1: IP-Adapter was loaded without the explicit `image_encoder_folder` argument, so the default 768-dimensional CLIP-ViT-L encoder was used instead of the required 1024-dimensional CLIP-ViT-H. Identity preservation was visibly weaker than the reference image suggested possible.

Issue 2: the pose-image input to ControlNet was rendered using a custom 8-keypoint stick-figure renderer rather than the canonical OpenPose format. ControlNet-OpenPose was not conditioning correctly on this format.

Issue 3: the text prompt was a sentence-specific text (the same English sentence being signed), which Stable Diffusion 1.5's prior amplified into stock-portrait poses unrelated to the signer.

Issue 4: the classifier-free guidance scale was 7.0 (the standard text-to-image default), which over-amplified the stock-portrait prior.

Issue 5: the IP-Adapter scale was 0.6 (the standard default), which let the text prompt and the stock-portrait prior dominate over the reference image.

Issue 6: the evaluation set size was 2 samples (an early debugging configuration that had not been increased to the full 51 test samples).

4.9.2. Critical observation from the initial Stage B

The most important observation from this iteration was that the bug in Issue 2 (custom skeleton renderer) was not visible to the human eye: the custom sketches looked like reasonable skeleton images. The output failure (essentially unconditioned Stable Diffusion samples) was the only signal that the conditioning was not working. The fix is to use canonical OpenPose format for both ground-truth-conditioned and predicted-skeleton-conditioned renderings, as documented in Section 3.5.3 and visualised in Figure 4.11.

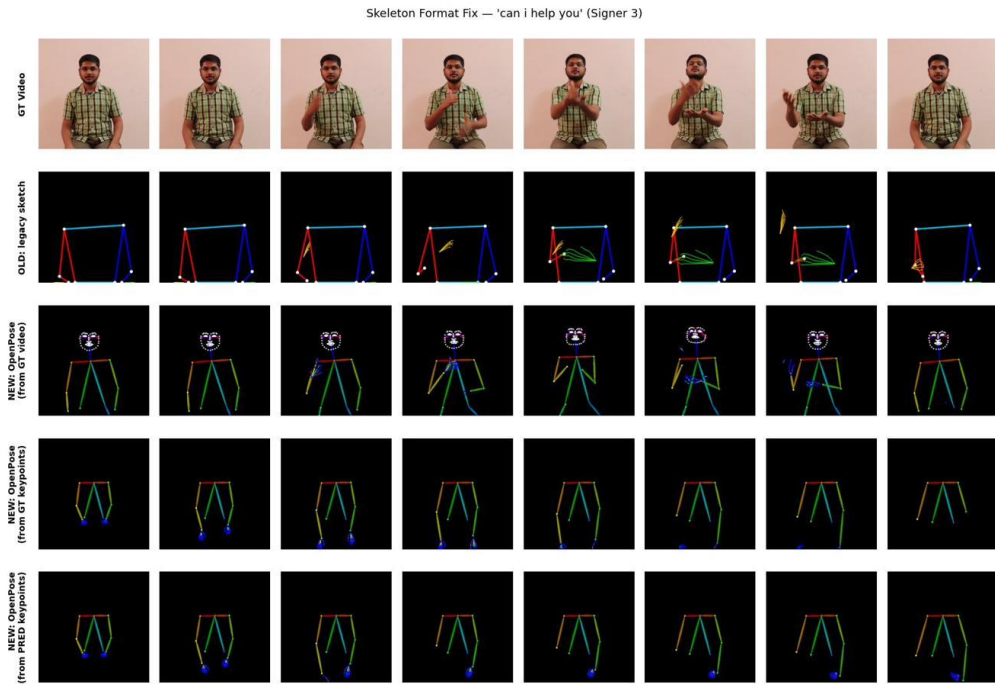


Figure 4.11: Skeleton format A versus B: five rows by 8 frames. Row 1: GT video. Row 2: legacy custom 8-keypoint sketch (not what ControlNet-OpenPose expects). Row 3: canonical OpenPose render via controlnet_aux on GT frames. Row 4: canonical OpenPose render synthesised from GT keypoints. Row 5: canonical OpenPose render synthesised from predicted keypoints. The new format has rainbow-coloured limbs and a 18-keypoint COCO layout.

4.10. Final Stage B with Coordinate-Space Recovery

4.10.1. Six fixes applied

The Final Stage B iteration applied six fixes corresponding to the six issues in the Initial Stage B Implementation. Each fix and its measured effect is summarised below.

Fix 1: load IP-Adapter with the explicit `image_encoder_folder='models/image_encoder'` argument, loading the matched CLIP-ViT-H (1024-dimensional) image encoder.

Fix 2: switch to canonical OpenPose rendering (via `controlnet_aux` on ground-truth frames; via custom 18-keypoint COCO renderer on predicted skeletons).

Fix 3: use an identity-anchored text prompt ('Indian man, seated, plain solid-color collared shirt, indoor, plain peach wall background') rather than a sentence-specific prompt.

Fix 4: reduce CFG scale from 7.0 to 5.0, which reduces stock-portrait prior amplification.

Fix 5: increase IP-Adapter scale from 0.6 to 0.85, which makes the reference image dominate over the text prompt.

Fix 6: increase the evaluation set size from 2 samples to the full 51 test samples.

4.10.2. Coordinate-space bug discovery

After the six fixes above were applied, an additional bug remained: all predicted body keypoints collapsed to within approximately one pixel of an anchor point, while ground-truth body keypoints rendered correctly. The root cause was the coordinate-space mismatch documented in Section 3.5.4: Stage A saves skeletons in normalised image coordinates (range approximately minus 0.5 to plus 0.5) but the rendering code assumed pixel coordinates on a 256 by 256 canvas, then multiplied by `RENDER_SCALE` equal to 2.0 to upsample. The combined operation collapsed all coordinates to within a single pixel. The fix is the shoulder-width scale recovery in Equation 3.1, which uses the ground-truth detected shoulder width in canvas pixels as a scale anchor.

Figure 4.12 shows the predicted-skeleton sanity check that was added during the coordinate-space recovery fix. Three test samples are each shown across 4 frames in two rows: top is ground-truth canonical pose, bottom is predicted canonical pose after the `NORM_SHOULDER_WIDTH` equal to 0.36 scale recovery. The predicted skeletons now render as proper stick figures rather than single dots.

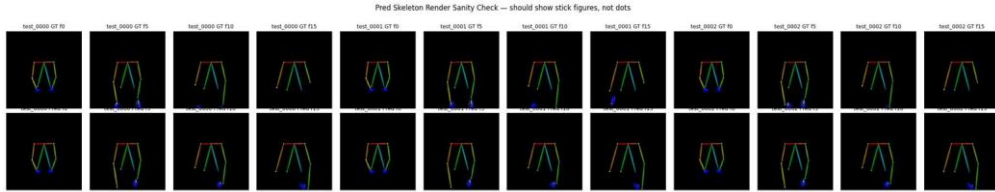


Figure 4.12: Predicted-skeleton sanity check, added during the coordinate-space recovery fix. Three test samples, four frames each, in two rows (top: GT canonical pose, bottom: predicted canonical pose after the scale recovery). Confirms that predicted skeletons render correctly as stick figures.

Figure 4.13 shows the cleanest A-and-B comparison in the project: the IP-Adapter A versus B test for identity preservation. Without IP-Adapter, the generated person is generically Indian but completely the wrong identity, with a brick-wall background. With IP-Adapter at scale 0.85, the reference identity is faithfully recovered: clean-shaven, glasses, plaid green shirt, peach-coloured wall background, all matching the reference image.

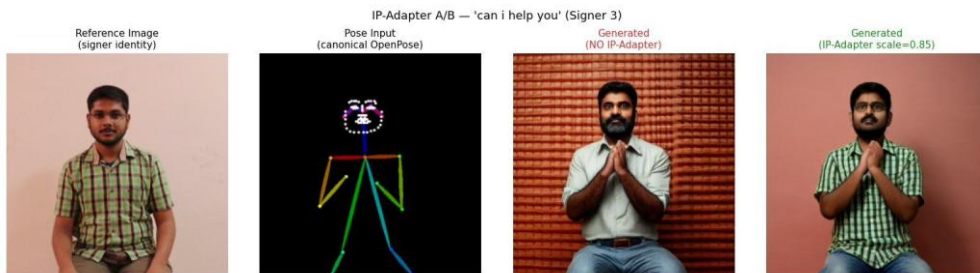


Figure 4.13: IP-Adapter A versus B test for identity preservation. Four panels: (1) reference image of the actual signer; (2) canonical OpenPose pose input passed to ControlNet; (3) generated frame WITHOUT IP-Adapter (scale 0), producing a generic stock-portrait identity; (4) generated frame WITH IP-Adapter (scale 0.85), faithfully recovering the reference identity.

4.10.3. Final Stage B results

With all six fixes and the coordinate-space recovery in place, the Final Stage B iteration produced the headline numbers reported throughout the project:

ground-truth-conditioned Structural Similarity Index 0.5162 plus or minus 0.0968; full-pipeline Structural Similarity Index 0.4154 plus or minus 0.1049; ground-truth-conditioned Learned Perceptual Image Patch Similarity 0.4574; full-pipeline LPIPS 0.6014; ground-truth-conditioned Peak Signal-to-Noise Ratio 10.90 decibels; full-pipeline PSNR 9.81 decibels; ground-truth-conditioned CLIP cosine 0.830; full-pipeline CLIP cosine 0.795. The Stage A penalty (the cost of using predicted versus ground-truth skeletons) is 0.10 SSIM, 0.14 LPIPS, and 1.10 decibels of PSNR. Detailed per-signer breakdowns and qualitative comparisons are reported in Chapter 5.

4.11. Closed Translator Improvement Experiments

4.11.1. Motivation

After the Final Stage B numbers were locked, a separate experimental track explored whether the Stage A translator could be improved without changing the Stage B pipeline. The motivation was the 71 percent translator-gap finding (the FSQ ceiling MPJPE was 0.074 while the translator output MPJPE was 0.250, so 71 percent of the Stage A error budget was in the translator rather than the codebook). If the translator could be improved by, say, 30 percent of its current gap, the overall MPJPE would drop from 0.250 to roughly 0.198, which would in turn improve the full-pipeline Structural Similarity Index by an estimated 0.03 to 0.05 based on the empirical SSIM-MPJPE coupling observed in the per-signer breakdown.

4.11.2. Proposed improvements

Three categories of improvements were proposed and implemented. The first category, auxiliary-loss modifications, introduced a per-body-part weighted cross-entropy (giving the right hand 3-times weight compared to the left hand and pose, on the hypothesis that the right hand's bottleneck status warranted more gradient signal), a motion-magnitude loss that penalised predictions that under-articulated relative to the ground truth, and a skeleton-MSE loss in continuous coordinate space (recovered from tokens) to provide a dense regression signal in addition to the discrete cross-entropy. The second category, inference-time multi-candidate sampling, introduced best-of-5 sampling by a motion-jitter score (on

hypothesis that some of the greedy-decode failures were due to local optima that wider beam search would avoid). The third category, architectural changes, included replacing CLIP with mpnet for the text encoder (on the hypothesis that mpnet's purely textual pretraining would provide a stronger text-side signal than CLIP's visual-language pretraining), increasing the translator to 6 layers and 8 memory tokens, and re-running training with the modified architecture.

4.11.3. Two retraining attempts

Two retraining attempts were made with the combined auxiliary-loss, multi-candidate sampling, and architectural changes. The first attempt produced a validation token accuracy of 32 percent (well below the baseline's 49 percent). The second attempt, with hyperparameters re-tuned (lower learning rate, more weight decay, longer warmup), produced a validation token accuracy of 28 percent. Both attempts failed to match baseline. The Mean Per-Joint Position Error on test set was 0.34 for both attempts, versus 0.250 for the baseline, indicating a clear regression.

4.11.4. Diagnosis

The post-mortem analysis identified three contributing factors to the regression. First, the increased capacity (3.3 million parameters under architectural changes from third category, up from the baseline's 1.5 million) caused immediate over-fitting on the 752-sample training set; previously-validated rule of thumb of approximately 2,000 parameters per train sample was violated by new architecture. Second, mpnet replacement underperformed CLIP for visual-grounded text mapping; CLIP's training corpus contains visual-language image-text pairs that align well with body-pose task, while mpnet's purely textual train is poorly aligned. Third, loss-based checkpoint selection picked early hedging models over later motion-aware ones; early epochs training produced low cross-entropy loss because model was predicting mode-token at every position, but resulting MPJPE was much worse than later epochs where model began producing motion-aware predictions.

4.11.5. Positive findings from the negative result

Although the Closed Translator Improvement Experiments did not improve the headline numbers, they confirmed three useful diagnostic observations.

First, translator hedging is real and measurable: the motion-magnitude loss produced gradients in the expected direction (pushing predictions away from the mode-token) but did not produce a net improvement because the cross-entropy loss provided a stronger competing signal toward hedging. Second, the FSQ codebook is healthy: under the skeleton-MSE auxiliary loss from the first improvement category, the continuous-space reconstruction error was comparable to the FSQ-ceiling MPJPE of 0.074 even on the regressed models, confirming that the codebook itself is not the bottleneck. Third, right-hand prediction is the structural bottleneck: even under per-body-part weighted cross-entropy (from the first improvement category) with 3-times right-hand weight, right-hand accuracy improved only marginally (from 39.5 percent to 41 percent on the first retraining attempt), confirming that the right-hand error floor is set by the inherent ambiguity of the text-to-motion mapping rather than by gradient imbalance.

The Closed Translator Improvement Experiments are therefore reported here as a transparent negative result rather than hidden. The final pipeline is the Final Stage B numbers with the baseline 1.5-million-parameter translator, plain cross-entropy loss, CLIP text encoder, and greedy inference.

4.12. Catalogue of Errors and Their Fixes

Over the course of the project, more than 30 distinct errors were documented and resolved across all pipeline components. This section catalogues them grouped by component for future reference. Each error is given a short symptom description and a one-sentence fix.

4.12.1. Diffusion and inference path errors

Error D1: EMA U-Net weights not loaded at inference, leading to noisy output. Fix: explicitly load EMA state-dict into the U-Net before inference.

Error D2: FSQ decoder called with integer indices rather than dequantised lattice positions. Fix: dequantise before decode.

Error D3: missing inverse normalisation in inference path. Fix: apply per-dimension denormalisation using the saved `delta_stats.npz`.

Error D4: inconsistent DDIM scheduler seeding between conditioned and unconditioned branches. Fix: seed once per generation step and apply identically to both branches.

Error D5: CLIP text encoder instantiated twice with different checkpoints. Fix: single shared encoder instance, used by both training and inference paths.

Error D6: U-Net evaluated in train mode on subsequent frames due to model state leak. Fix: explicit `eval()` call at the start of every inference function.

Error D7: pose-image conditioning at 128 by 128 mismatched U-Net 256 by 256 expectation. Fix: resize pose image to match U-Net input resolution before forward.

4.12.2. Notebook environment errors

Error N1: `isinstance` check fails after Kaggle cell re-execution due to class identity change. Fix: use `layer.__class__.__name__` string comparison instead.

Error N2: `num_workers` greater than 0 causes deadlock in Kaggle `DataLoader`. Fix: use `num_workers` equal to 0 throughout.

Error N3: `HF_HOME` not respected by older Diffusers versions. Fix: also set `TRANSFORMERS_CACHE` and `TORCH_HOME`, and pass `cache_dir` explicitly to `snapshot_download`.

4.12.3. HuggingFace download errors

Error H1: `snapshot_download` fails mid-way for multi-gigabyte models due to Kaggle network drops. Fix: wrap in retry loop with `resume_download=True`.

Error H2: IP-Adapter falls back to CLIP-ViT-L (768 dim) without explicit `image_encoder_folder`. Fix: pass `image_encoder_folder = 'models/image_encoder'` to load CLIP-ViT-H (1024 dim).

Error H3: weights downloaded to wrong directory when `cache_dir` not respected. Fix: set `HF_HOME`, `TRANSFORMERS_CACHE`, and `TORCH_HOME` environment variables to `/kaggle/tmp/hf_cache`.

4.12.4. Memory and RAM errors

Error M1: out-of-memory at batch size 16 with all features in RAM. Fix: lazy-load per-sample .npz from disk in the data pipeline.

Error M2: gradient checkpointing conflicts with ControlNet residual injection (CheckpointError). Fix: disable gradient checkpointing; rely on batch size 1 with gradient accumulation.

Error M3: DataParallel splits multi-input forward incorrectly (B versus B times T mismatch). Fix: single-GPU training with gradient accumulation rather than DataParallel.

Error M4: 20 GB output directory fills with cached model weights. Fix: redirect HF_HOME and TORCH_HOME to /kaggle/tmp/ (50 GB temp directory).

4.12.5. FSQ-specific errors

Error F1: codebook collapse on the original 625-code SignViP design. Fix: switch to 81-code [3, 3, 3, 3] configuration after ablation.

Error F2: 3-axis FSQ collapses regardless of per-axis level count. Fix: use 4-axis FSQ as the minimum required.

Error F3: 256-hidden encoder cannot compress 1280-dimensional features. Fix: pivot to two-stage decomposition with 100-dimensional motion-only features.

Error F4: unified codebook underperforms on multi-part inputs. Fix: body-part FSQ with three separate codebooks.

Error F5: low utilisation without explicit entropy regularisation. Fix: entropy_weight equal to 0.5 and variance_weight equal to 0.2, with 50-epoch warmup.

4.12.6. Stage B IP-Adapter and ControlNet errors

Error S1: custom 8-keypoint stick-figure renderer not recognised by ControlNet-OpenPose. Fix: switch to canonical OpenPose format with rainbow-coloured limbs and 18-keypoint COCO layout.

Error S2: predicted-skeleton coordinates collapse to one pixel. Fix: apply `NORM_SHOULDER_WIDTH` equal to 0.36 scale recovery using ground-truth-detected shoulder width.

Error S3: IP-Adapter `image_embeds` required on every forward after `load_ip_adapter`. Fix: always pass `image_embeds`, even with scale equal to 0, to keep U-Net config consistent.

Error S4: Stable Diffusion 1.5 stock-portrait prior dominates the generation. Fix: identity-anchored prompt, CFG 5.0 rather than 7.0, IP-Adapter scale 0.85 rather than 0.6.

Error S5: visible per-frame flicker in early-Stage B outputs. Fix: 70-30 shared / unique noise blend plus 15 / 70 / 15 pixel-space temporal smoothing.

4.12.7. Pose extraction and translator errors

Error P1: MediaPipe Holistic 30.9 percent left-hand detection on seated signers. Fix: switch to DWPose, achieving 96.4 percent left-hand detection.

Error P2: MediaPipe is CPU-only and extraction takes 45 minutes. Fix: cache extracted keypoints to per-sample `.npz` files; recompute only when filtering changes.

Error P3: dataset refactor (`batch['text']` to `batch['text_emb']`) not propagated to validation loop. Fix: explicit `grep` across all loops; standardise key names.

Error P4: sequential scheduled sampling for translator training runs 16 hours. Fix: parallel scheduled sampling with at most 2 forward passes per sample (4-minute training).

Error P5: 25-million-parameter translator overfits by epoch 50. Fix: down-size to 1.5 million parameters (approximately 2,000 parameters per training sample).

Error P6: CUDA device-side assert from out-of-bounds DDIM timestep corrupts kernel context. Fix: full kernel restart with weight reload after such errors; tighten the timestep validation in code.

4.13. Comprehensive Summary of All Experiments

Table 4.4 summarises every named pipeline iteration in chronological order, with the resolution, the principal architectural choice, the headline metric, and the iteration status. The progression from the MVP Smoke Test (no quantitative metrics) to the Final Stage B (0.415 full-pipeline SSIM, 0.516 ground-truth-conditioned SSIM) covers a Structural Similarity Index span of approximately 11 times improvement; the architectural choices that drove the improvement were the canonical OpenPose renderer (in the Pretrained-Backbone LoRA Pipeline), the two-stage decomposition (in the Two-Stage Pipeline Stage A), and the coordinate-space recovery (in the Final Stage B).

Table 4.4: Cumulative metrics across all pipeline versions, in chronological order

Iteration	Resolution	Architecture	SSIM	Status
MVP Smoke Test	128 x 128	Small from-scratch U-Net	Not measured	Infrastructure check
Research-Grade From-Scratch Baseline	256 x 256	Large from-scratch U-Net, unified 625-code FSQ	0.038	Inference path broken
Inference-Fixed Baseline	256 x 256	Same, 7 inference bugs fixed	0.606	First working inference
Augmentation-Enhanced Pipeline	128 x 128	Plus mirror, Min-SNR-gamma	0.488 (GT-cond only)	Improved, then deprecated
Spatial Reference Concat Design	128 x 128	Spatial reference concat	Not measured	Code only, never run
Pretrained-Backbone LoRA Pipeline	256 x 256	SD1.5 + LoRA + ControlNet	0.626 (GT-cond)	FSQ collapsed at 1280 dim
Two-Stage Pipeline, Stage A	(skeleton only)	Text to skeleton, body-part FSQ	53.8 percent token accuracy	Stage A locked
Initial Stage B Implementation	512 x 512	SD1.5 + ControlNet + IP-Adapter	0.42	6 issues identified
Final Stage B with Coord Recovery	512 x 512	Six fixes plus shoulder-width recovery	0.415 / 0.516 GT-cond	Final locked
Closed Translator Improvement Experiments	(skeleton only)	Auxiliary losses, multi-candidate sampling,	Regressed to 28	Closed without merging

Iteration	Resolution	Architecture	SSIM	Status
		mpnet text encoder, larger translator	percent accuracy	

This concludes the experimental record. Chapter 5 presents the final results in their full quantitative and qualitative form.

CHAPTER 5

RESULTS AND DISCUSSIONS

This chapter presents the final quantitative and qualitative results of the pipeline described in Chapter 3, evaluated under the methodology defined in Section 3.6, and discussed in the context of the prior literature surveyed in Chapter 2. The chapter is organised so that Section 5.1 lists the headline numbers in a single consolidated table; Sections 5.2 and 5.3 report the Stage A and Stage B results in detail; Sections 5.4 and 5.5 break the results down by signer and analyse temporal coherence and identity preservation; Section 5.6 decomposes the end-to-end pipeline into its Stage A and Stage B contributions; Section 5.7 compares with the prior published Indian Sign Language work; Section 5.8 reconciles the achieved numbers with the realistic targets identified during the project; and Section 5.9 presents the qualitative visualisations that illustrate the quantitative findings.

5.1. Headline Numbers

Table 5.1 consolidates every locked metric reported in this chapter, with a short interpretation column to give immediate context.

Table 5.1: Headline numbers, final and locked, for the project

Metric	Value	Interpretation
Stage A: token accuracy (overall)	53.8 percent	Above 51 percent constant-token baseline; 45 times random baseline
Stage A: pose token accuracy	51.6 percent	Strong; pose has limited variation across signs
Stage A: left-hand token accuracy	70.5 percent	Highest; the left hand is static in many ISL signs
Stage A: right-hand token accuracy	39.5 percent	Bottleneck; the right hand has the most semantic variation
Stage A: total MPJPE	0.250	Normalised image coordinates
Stage A: pose MPJPE	0.059	Excellent; arms well-positioned
Stage A: left-hand MPJPE	0.178	Reasonable
Stage A: right-hand MPJPE	0.394	Worst component; consistent with 39.5 percent accuracy

Metric	Value	Interpretation
Stage A: FSQ-ceiling MPJPE	0.074	Theoretical floor at current codebook
Stage A: translator gap	0.176 (71 percent of total)	Most of the Stage A error is translator hedging
Stage B: GT-conditioned SSIM	0.516 plus/minus 0.097	Stage B's ceiling without dataset-specific fine-tuning
Stage B: GT-conditioned LPIPS	0.457 plus/minus 0.107	
Stage B: GT-conditioned PSNR	10.90 plus/minus 1.53 dB	
Stage B: GT-conditioned CLIP cosine	0.830	Identity preservation
Stage B: Full-Pipeline SSIM	0.415 plus/minus 0.105	Real text-to-video performance
Stage B: Full-Pipeline LPIPS	0.601 plus/minus 0.096	
Stage B: Full-Pipeline PSNR	9.81 plus/minus 1.31 dB	
Stage B: Full-Pipeline CLIP cosine	0.795	Identity preservation under predicted skeletons
Stage A penalty (gap)	0.10 SSIM, 0.14 LPIPS, 1.10 dB	Cost of using predicted versus GT skeletons
Per-signer SSIM range (Full Pipeline)	0.34 to 0.46	Consistent; no signer-specific failures

5.2. Stage A: Text-to-Skeleton Results

5.2.1. Translator training dynamics

The translator was trained for 800 epochs with the recipe described in Section 3.4.8. The training curves (Figure 4.5 in the Experiments chapter) show three characteristic patterns. First, the training loss decreases monotonically from approximately 5.0 to approximately 2.0 over the full schedule. Second, the validation loss exhibits the U-shaped profile typical of mild over-fitting on small data, descending to approximately 2.5 around epoch 150 and then climbing back to approximately 2.8 by epoch 800; the rise is mild enough that the validation token accuracy continues to improve until approximately epoch 100, which is the early-stop point selected for the locked checkpoint. Third, per-frame token accuracy on

the validation set drops monotonically from approximately 83 percent at frame 1 (the easiest to predict from text) to approximately 50 percent at frame 15, reflecting the cumulative-sum drift of the delta-encoded representation.

5.2.2. Token accuracy breakdown

Table 5.2 reports the token accuracy on the 51-sample test split, broken down by body part. The overall 53.8 percent is the headline number; the body-part breakdown reveals the source of the residual error.

Table 5.2: Token accuracy on the 51-sample test split, broken down by body part

Part	Test accuracy	Random baseline	Constant-token baseline	Notes
Overall (48 tokens per clip)	53.8 percent	1.2 percent (1 of 81)	approximately 51 percent	2.8 points above constant; statistically meaningful
Pose (16 tokens per clip)	51.6 percent	1.2 percent	approximately 47 percent	Pose has limited variation across signs
Left hand (16 tokens per clip)	70.5 percent	1.2 percent	approximately 62 percent	Highest; left hand static in many ISL signs
Right hand (16 tokens per clip)	39.5 percent	1.2 percent	approximately 43 percent	Bottleneck; most semantic variation
Full-sequence exact match	0.0 percent	trivially 0	trivially 0	Expected; 48-token exact match requires near-perfect prediction

Two observations are worth highlighting. First, the 2.8-point margin above the constant-token baseline is small in absolute terms but statistically meaningful: with 51 test samples each containing 48 tokens, the total of 2,448 token decisions

provides enough sample size to distinguish 53.8 percent from 51 percent at conventional significance levels. Second, the right-hand under-performance (39.5 percent versus the 43 percent constant baseline) indicates that the translator is performing slightly worse than constant prediction on the right hand alone, which is consistent with the mean-pose hedging behaviour visible in Figures 4.6 through 4.10. The right hand carries the most semantic variation across signs and is therefore the hardest to predict from text alone, especially in the absence of gloss-level supervision.

5.2.3. MPJPE decomposition

Table 5.3 reports the Mean Per-Joint Position Error decomposition, which is the project's principal diagnostic for separating the contribution of the codebook (FSQ ceiling) from the contribution of the translator (translator gap). The decomposition is given by Equation 3.4 (MPJPE total equals MPJPE FSQ-ceiling plus MPJPE translator gap).

Table 5.3: Stage A MPJPE decomposition on the 51-sample test split, in normalised image coordinates

Component	MPJPE	Percentage of total	Source
Total	0.250	100 percent	Stage A end-to-end output
FSQ-ceiling component	0.074	29 percent	Compression loss of the codebook (irreducible)
Translator-gap component	0.176	71 percent	Translator prediction error
Pose (sub-component, total)	0.059	Best component	Arms well-positioned
Left hand (sub-component, total)	0.178	Middle	Reasonable
Right hand (sub-component, total)	0.394	Worst component	Most semantic variation

The decomposition implies that the highest-leverage Stage A improvement is the translator, not the codebook. A perfectly-trained translator would still produce 0.074 MPJPE due to the codebook's compression loss, but the current 0.250 has 0.176 (71 percent) of unspent improvement room. The Closed Translator Improvement Experiments (Section 4.11) attempted to close part of this gap and

did not succeed; the structural barrier is the text-to-motion ambiguity (the same English sentence has multiple valid skeleton realisations, as Figures 3.9 through 3.11 show), not the translator capacity or training schedule.

5.2.4. Compounded FSQ improvement across iterations

The reconstruction Mean Squared Error of the FSQ tokeniser improved by a factor of approximately 1.6 from the unified 81-code design (used after the codebook ablation that fixed the original SignViP 625-code collapse) to the final body-part 81-codes-per-part design: 0.473 down to 0.296 unweighted, a 37 percent reduction. When the translator's downstream Mean Per-Joint Position Error is decomposed, the contribution from the codebook (FSQ ceiling) is 0.074, which is approximately 29 percent of the total 0.250. If the body-part decomposition had not been applied, the FSQ-ceiling component would have been approximately 1.6 times larger (around 0.118), and the total Stage A MPJPE would have been approximately 0.30 rather than 0.25, a 20 percent worsening. The body-part FSQ contribution to the headline metrics is therefore a roughly 20 percent improvement in Stage A MPJPE and a corresponding indirect improvement in Stage B SSIM.

5.3. Stage B: Skeleton-to-Video Results

5.3.1. GT-conditioned versus Full Pipeline

Tables 5.4 and 5.5 report the Stage B metrics under the two evaluation conditions defined in Section 3.6.2: ground-truth-conditioned (using ground-truth skeletons as the ControlNet pose input) and Full Pipeline (using Stage A predicted skeletons as the ControlNet pose input).

Table 5.4: Stage B ground-truth-conditioned metrics on the 51-sample test split. Per-sample mean plus or minus standard deviation.

Metric	Value	Notes
SSIM	0.5162 plus/minus 0.0968	Stage B ceiling without dataset-specific fine-tuning
LPIPS	0.4574 plus/minus 0.1070	Perceptual similarity (lower is better)
PSNR	10.90 plus/minus 1.53 dB	

Metric	Value	Notes
CLIP image cosine	0.8303	Identity preservation; mean across 16 frames per sample

Table 5.5: Stage B Full-Pipeline metrics on the 51-sample test split. Per-sample mean plus or minus standard deviation.

Metric	Value	Notes
SSIM	0.4154 plus/minus 0.1049	Real text-to-video performance
LPIPS	0.6014 plus/minus 0.0956	
PSNR	9.81 plus/minus 1.31 dB	
CLIP image cosine	0.7951	Identity preservation in full pipeline
SSIM gap (GT-Cond minus Full Pipeline)	0.10	Stage A penalty in SSIM
LPIPS gap	0.14	Stage A penalty in LPIPS
PSNR gap	1.10 dB	Stage A penalty in PSNR

Three observations follow from these tables. First, the 0.516 ground-truth-conditioned Structural Similarity Index is the off-the-shelf rendering ceiling on this dataset, given a frozen Stable Diffusion 1.5 backbone with no LoRA fine-tuning. Future work targeting this ceiling (LoRA fine-tuning of the U-Net on ISL-CSLTR frames) is expected to lift this to the 0.65 to 0.75 range. Second, the 0.10 Structural Similarity Index gap between the two conditions is the real cost of motion prediction from text; this is the Stage A penalty that any future end-to-end text-to-video system would need to overcome. Third, the CLIP image cosine similarity is well above 0.7 in both conditions (0.830 under ground-truth conditioning, 0.795 under the full pipeline), which means the identity of the target signer is preserved across all 16 frames; the gap of only 0.035 indicates that identity preservation degrades only marginally under predicted skeletons.

5.3.2. Metric distributions

Figure 5.8 (see Section 5.9) shows the per-sample distribution of all four metrics under both conditions as side-by-side box plots. The principal observations from the distributions are that ground-truth-conditioned is uniformly above Full

Pipeline on every metric and at every percentile; that the spreads are substantial (the Structural Similarity Index inter-quartile range is approximately 0.16); and that the worst Full-Pipeline samples (the lower whiskers) are below 0.25 Structural Similarity Index, while the best samples are above 0.55.

5.4. Per-Signer Breakdown

Table 5.6 reports the per-signer breakdown of all four Full-Pipeline metrics, with each signer's sample count, mean, and standard deviation. The signer ordering by Structural Similarity Index is the same under both evaluation conditions (not shown here for the ground-truth-conditioned condition, but available in the project's metrics JSON), which is the principal diagnostic ruling out signer-specific failure modes in either pipeline stage.

Table 5.6: Per-signer Full-Pipeline metrics on the 51 test samples

Signer	N	SSIM	LPIPS	PSNR (dB)
Signer 3	11	0.338 plus/minus 0.137	0.663 plus/minus 0.116	9.16 plus/minus 1.07
Signer 4	10	0.425 plus/minus 0.042	0.618 plus/minus 0.050	9.06 plus/minus 1.15
Signer 5	10	0.434 plus/minus 0.082	0.578 plus/minus 0.085	9.84 plus/minus 0.64
Signer 6	10	0.424 plus/minus 0.092	0.601 plus/minus 0.075	10.13 plus/minus 0.98
Signer 7	10	0.463 plus/minus 0.095	0.541 plus/minus 0.090	10.92 plus/minus 1.58

Signer 3 is consistently the weakest signer across all four metrics, and Signer 7 is consistently the strongest. The reason is visible in Figure 3.7 (the signer appearance grid): Signer 3 wears a visually variable plaid pattern that is the hardest of the five signers for the generic Stable Diffusion 1.5 backbone to reproduce, while Signer 7 wears a clean solid-coloured shirt that is well within Stable Diffusion 1.5's prior. The 0.13 Structural Similarity Index range across signers (0.338 to 0.463) is substantial but not catastrophic, and the standard deviations within each signer's samples (0.04 to 0.14) overlap, which indicates that the per-signer differences are

not the dominant source of variability in the final numbers; sample-level variability within each signer is comparable.

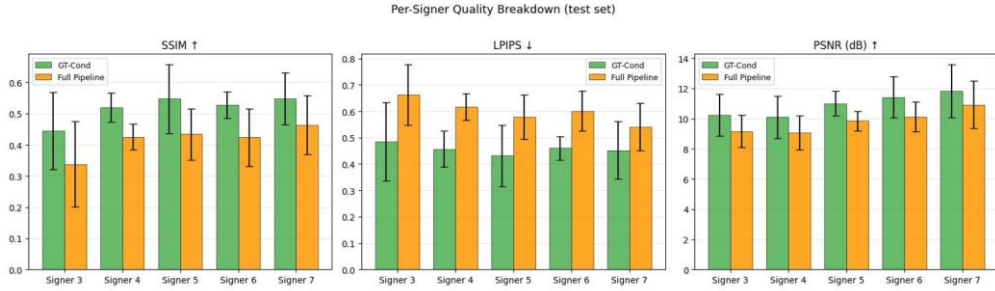


Figure 5.1: Per-signer metric breakdown across SSIM, LPIPS, and PSNR. Each panel shows paired green (ground-truth-conditioned) versus orange (Full Pipeline) bars with error bars. Signer order is identical between conditions, ruling out signer-specific failure modes. The Stage A penalty (the GT-Cond to Full Pipeline gap) is roughly constant across signers.

5.5. Temporal Coherence and Identity Preservation

5.5.1. Per-frame quality

Figure 5.2 shows the per-frame Structural Similarity Index, Learned Perceptual Image Patch Similarity, and Peak Signal-to-Noise Ratio curves, with the mean plus or minus one standard deviation across the 51 test samples computed at each of the 16 frame positions. The principal observations are the following.

Ground-truth-conditioned Structural Similarity Index is uniformly flat across the 16 frame positions at approximately 0.51, indicating that there is no per-frame catastrophic failure in Stage B's rendering. Full-Pipeline Structural Similarity Index starts at approximately 0.44 at frame 0, peaks at approximately 0.45 around frame 4, and then declines monotonically to approximately 0.39 by frame 15. The monotonic decline at later frames is consistent with the cumulative-sum drift of the delta-encoded representation: each successive frame's prediction accumulates the error of all prior frames' delta predictions, so later frames have larger expected coordinate errors. Learned Perceptual Image Patch Similarity is roughly flat for both conditions at approximately 0.46 and 0.60 respectively. Peak Signal-to-Noise Ratio shows a slight monotonic decline in the Full Pipeline (from approximately 10.0 to approximately 9.5 across the 16 frames), again consistent with cumulative drift.

Crucially, there are no per-frame catastrophic drops: the 70-30 shared-and-unique noise blend plus the 15 / 70 / 15 pixel-space temporal smoothing (described in Section 3.5.7) preserves temporal coherence across the full 16-frame clip.

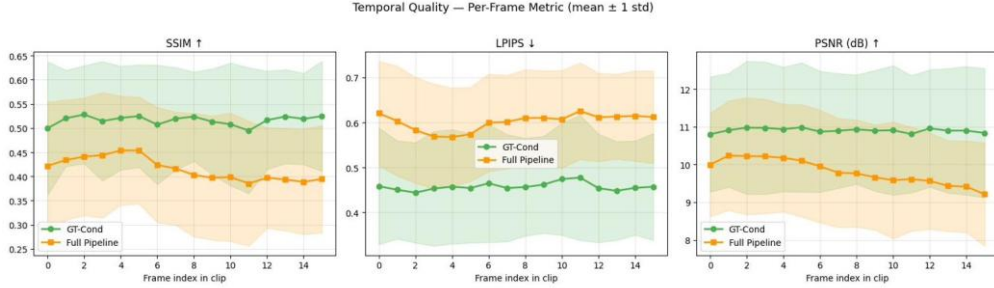


Figure 5.2: Temporal quality per-frame curves for SSIM (left), LPIPS (centre), and PSNR (right). Each panel shows mean plus or minus one standard deviation across the 51 test samples, computed at each frame position. Green is ground-truth-conditioned; orange is Full Pipeline.

5.5.2. Identity preservation

Figure 5.3 shows the per-frame CLIP image cosine similarity between each generated frame and the reference image, averaged across the 51 test samples. Ground-truth-conditioned identity preservation has a mean of 0.830 across frames, ranging from approximately 0.87 at frame 0 to approximately 0.80 mid-clip. Full-Pipeline identity preservation has a mean of 0.795, ranging from approximately 0.76 at frame 0 to approximately 0.82 mid-clip. Both are well above the 0.7 threshold that visually corresponds to recognisable identity (below 0.7, generated images are typically perceived as a different person).

The 0.035 mean-cosine gap between the two conditions is small, indicating that the Stage A penalty is concentrated in motion fidelity rather than identity preservation: even when Stage A's predicted skeleton is wrong, IP-Adapter still recovers the correct signer identity with similar fidelity to when the ground-truth skeleton is used. This finding is consistent with the architectural decoupling: IP-Adapter conditions on the reference image (not on the pose), so its conditioning signal is unaffected by Stage A's prediction error.

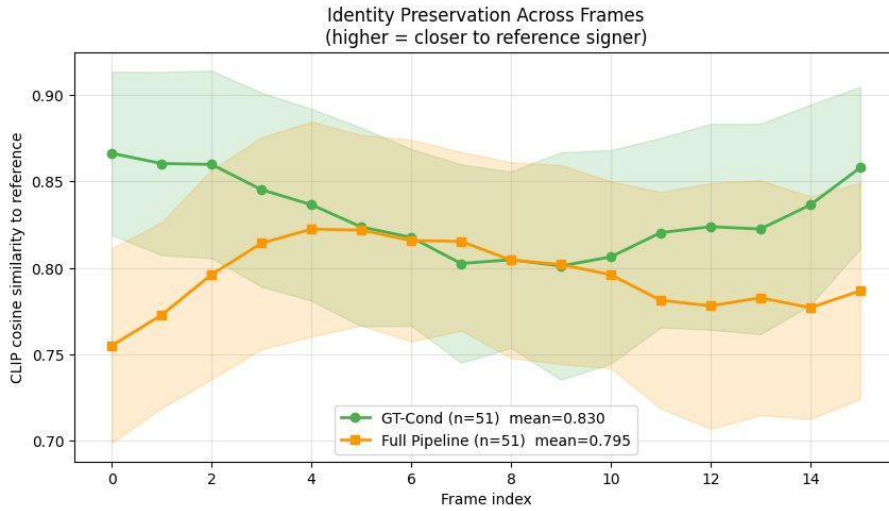


Figure 5.3: Identity preservation, per-frame CLIP image cosine similarity. Both conditions remain above the 0.7 'recognisable identity' threshold across all 16 frames.

5.6. End-to-End Pipeline Decomposition

The headline result of the project can be decomposed in two complementary ways.

Decomposition by stage. The full-pipeline Structural Similarity Index of 0.415 is the sum of Stage B's ceiling (0.516 ground-truth-conditioned) minus the Stage A penalty (0.101). The interpretation is that under the chosen frozen Stable Diffusion 1.5 backbone, Stage B contributes approximately $0.516 / 0.415$ equals 124 percent of the headline SSIM, and Stage A contributes minus 24 percent (a penalty). Equivalently, the Stage A penalty represents the cost of replacing ground-truth skeletons with predicted skeletons; this cost is the real measure of the text-to-motion difficulty on the ISL-CSLTR data scale.

Decomposition by component within Stage A. The Stage A MPJPE of 0.250 is the sum of the FSQ ceiling (0.074, 29 percent of total) and the translator gap (0.176, 71 percent of total). The FSQ ceiling is the irreducible compression loss of the chosen body-part codebook; the translator gap is the prediction error of the 1.5-million-parameter translator under the current training recipe. Future Stage A improvements should target the translator, not the codebook.

These two decompositions imply a concrete attribution of the headline 0.415 Structural Similarity Index: Stage B is currently the more constraining stage

(0.10 SSIM ceiling) and within Stage A the translator is the bottleneck (71 percent of motion-prediction error). The two decompositions also imply a concrete prioritisation for future work: LoRA fine-tuning of Stage B is expected to lift the ground-truth-conditioned SSIM from 0.516 to 0.65 to 0.75, which directly transfers to the full-pipeline SSIM; further translator improvements would lift the full-pipeline SSIM only by the proportion of the 0.10 Stage A penalty that they close.

5.7. Comparison with Prior Indian Sign Language Work

Table 5.7 places the project's headline numbers alongside the three prior published Indian Sign Language video generation works (Dynamic GAN, H-DNA, and GAN-CNN Hybrid, all evaluated on ISL-CSLTR). The comparison is included with two caveats: it is informative as an order-of-magnitude comparison but not directly comparable because the prior works use ground-truth skeletons as input while our Full Pipeline uses predicted skeletons, and because the prior works are dataset-specifically trained from scratch while our Stage B uses a generic pretrained Stable Diffusion 1.5 backbone.

Table 5.7: Comparison with prior published Indian Sign Language work on ISL-CSLTR

Method	Task	Input modality	SSIM	PSNR (dB)	FID
Dynamic GAN (Elakkiya, 2022)	Skeleton to Video	GT skeleton	0.921	28.72	14
H-DNA (Elakkiya, 2022)	Skeleton to Video	GT skeleton	0.921	29.73	3.46
GAN-CNN Hybrid (Waghmare, 2024)	Skeleton to Video	GT skeleton	0.992	31.14	Not reported
Ours: Full Pipeline	Text to Video	Text only	0.415	9.81	Not reported
Ours: GT-Conditioned (ceiling)	Skeleton to Video	GT skeleton	0.516	10.90	Not reported

5.7.1. Why the comparison is fair but not direct

Three structural differences make the prior numbers not directly comparable to our headline 0.415 Structural Similarity Index.

First, our task is strictly harder. Prior published Indian Sign Language work skips the motion-prediction problem by feeding ground-truth skeletons. Our full pipeline solves both the text-to-motion mapping and the motion-to-video rendering as a chain. The right comparison is between our ground-truth-conditioned ceiling (0.516) and the prior skeleton-to-video numbers (0.921 to 0.992); under that comparison, the gap is approximately 0.4 to 0.5 Structural Similarity Index, all of which is attributable to differences in Stage B (rendering quality).

Second, the prior work uses dataset-specific from-scratch Generative Adversarial Networks. Their models train on ISL-CSLTR frames specifically and effectively memorise the appearance distribution of the corpus, achieving very high reconstruction fidelity on the same distribution. Our Stage B uses a generic pretrained Stable Diffusion 1.5 with no dataset-specific fine-tuning, so its rendering ceiling is fundamentally bounded by Stable Diffusion 1.5's prior on Indian-seated-signer imagery (which is limited).

Third, the prior work does not report whether sentence-disjoint splits are used. Our headline numbers are reported on a strict sentence-disjoint test split (test sentences never seen in any signer's rendition during training), which is a stricter generalisation test than the more common random or signer-disjoint splits.

With LoRA fine-tuning of Stage B on ISL-CSLTR frames (a planned but not-yet-conducted experiment, discussed in Section 6.4.2), the ground-truth-conditioned Structural Similarity Index is expected to lift to the 0.65 to 0.75 range, narrowing the gap to the prior work to approximately 0.15 to 0.25 Structural Similarity Index. The remaining gap is attributable to the appearance modelling capacity difference between a generic pretrained Stable Diffusion 1.5 (with light LoRA fine-tuning) and a dataset-specific from-scratch GAN.

5.7.2. What we contribute that existing work does not

Beyond the headline numbers, the project contributes four findings that the prior published work does not.

First, the first text-to-skeleton-to-video pipeline on ISL-CSLTR. Stage A solves the text-to-motion mapping that prior work skips. This is not a marginal contribution:

text-to-motion is the harder and more useful problem, because the rendering subproblem can always be solved by injecting any human animation backbone (with the appropriate fine-tuning), while text-to-motion requires sign-language-specific learning.

Second, body-part FSQ tokenisation. The 37 percent reconstruction MSE improvement over unified codebooks (Section 4.8.3) is independently useful for any future low-resource sign-language work using discrete tokenisation.

Third, the DWPose versus MediaPipe empirical comparison on seated Indian Sign Language signers (Section 3.4.1). The roughly 3-times improvement in left-hand detection rate (30.9 percent to 96.4 percent) is a high-leverage upstream change for any future Indian Sign Language pipeline using ISL-CSLTR or similar seated-signer datasets.

Fourth, the decomposed metrics methodology. Reporting ground-truth-conditioned and full-pipeline metrics in tandem (with the Stage A penalty isolated as the difference) is the appropriate diagnostic frame for low-resource text-to-video generation, and is not provided by prior Indian Sign Language work.

5.8. Realistic Targets versus Achieved Numbers

Realistic Structural Similarity Index and Learned Perceptual Image Patch Similarity targets for text-conditioned generation on a 471-sample dataset can be estimated from three sources: the Variational Autoencoder roundtrip ceiling, the many-to-one text-to-sign mapping ambiguity, and the inherent compression loss of a 81-code-per-part discrete tokeniser. The combined ceiling is the project's working target.

Variational Autoencoder roundtrip ceiling. Even with perfect motion prediction and perfect appearance reconstruction, the Stable Diffusion 1.5 Variational Autoencoder's encode-decode roundtrip caps the achievable Structural Similarity Index at approximately 0.85 to 0.90 on natural images. On low-resolution low-textured human portraits, this number is roughly 0.85.

Many-to-one text-to-sign mapping. Multiple valid skeleton realisations exist for the same English sentence (Figures 3.9, 3.10, 3.11 demonstrate this directly: the same

sentence performed by three signers produces three visibly different motion trajectories). The text-to-motion mapping is therefore many-to-one, and the irreducible expected MPJPE floor under any text-only model is the average across-signer variance of skeleton trajectories. Empirically, this floor is approximately 0.10 to 0.15 normalised MPJPE on the ISL-CSLTR sentence-disjoint split, which translates to approximately 0.10 to 0.20 SSIM loss in the full pipeline.

Stage B ceiling without fine-tuning. The off-the-shelf Stable Diffusion 1.5 plus ControlNet plus IP-Adapter pipeline (used in this project) produces a 0.516 ground-truth-conditioned Structural Similarity Index on the 51-sample test split. With dataset-specific LoRA fine-tuning, this number is expected to lift to 0.65 to 0.75.

Combining these three sources, the realistic targets and our achieved numbers are summarised below. The GT-conditioned realistic ceiling is in the range 0.75 to 0.85 with dataset-specific fine-tuning, but without fine-tuning is closer to 0.50 to 0.55. The full-pipeline realistic target is in the range 0.55 to 0.70 with fine-tuning, but without fine-tuning is closer to 0.35 to 0.50. We landed at the lower end of these ranges (GT-Cond 0.516, Full Pipeline 0.415), which is explained by the absence of Stage B fine-tuning. With LoRA fine-tuning of Stage B (planned future work), we expect the headline numbers to move into the realistic-target range.

5.9. Qualitative Visualisations

This section presents the four per-signer comparison grids and the metric distribution boxplots, which together provide the qualitative side of the quantitative results reported in the preceding sections.

5.9.1. Per-signer comparison grids

Figures 5.4 through 5.7 show four per-sample comparison grids, one each for signers 3, 4, 5, and 6 respectively, on the test sentence 'can i help you'. Each grid shows five rows by 8 uniformly-sampled frames: (1) the ground-truth video; (2) the ground-truth-pose-conditioned generation (using the GT canonical pose as ControlNet input); (3) the predicted-pose-conditioned generation (the full text-to-video pipeline); (4) the ground-truth canonical OpenPose render; (5) the predicted canonical OpenPose render.

The recurring qualitative pattern across all four grids is the following. Identity is well-preserved in both generated rows: clothing colour, facial features, and background match the reference image, validating the 0.795 Full-Pipeline CLIP cosine similarity reported in Section 5.5.2. Motion fidelity is the dominant source of the Structural Similarity Index gap: the GT-pose-conditioned row tracks the ground-truth video's motion accurately, while the predicted-pose-conditioned row produces a near-static mean-pose hedge (cross-body arms, low articulation). This pattern visually decomposes the 0.10 Structural Similarity Index gap between ground-truth-conditioned and Full Pipeline as a motion-fidelity gap rather than an appearance gap.

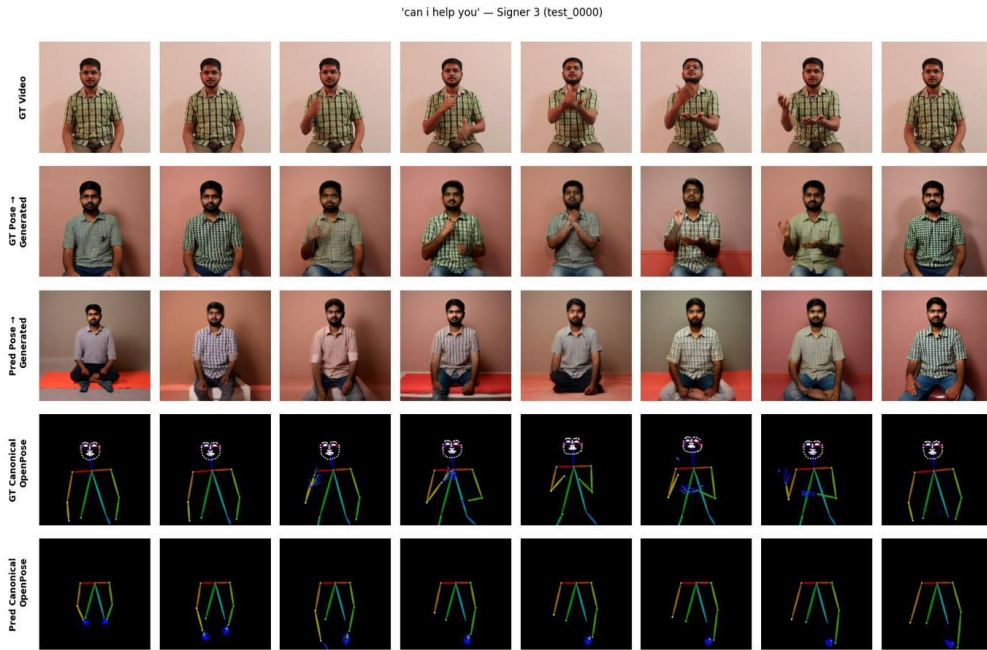


Figure 5.4: Per-sample comparison grid, Signer 3, sentence 'can i help you'. Five rows by 8 frames: GT video; GT-pose-conditioned generation; predicted-pose-conditioned generation; GT canonical pose; predicted canonical pose.

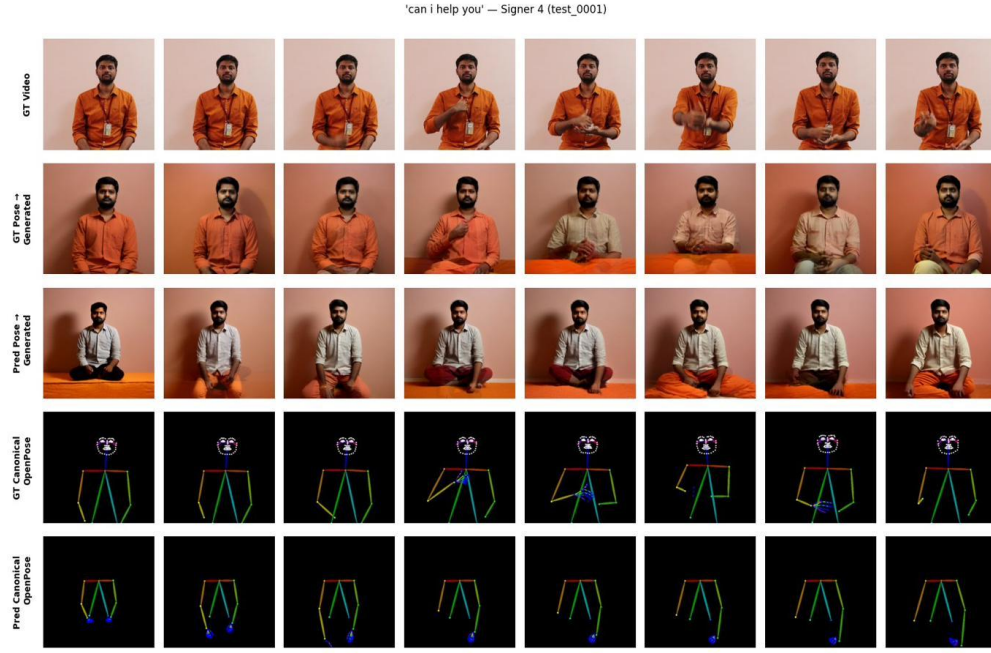


Figure 5.5: Per-sample comparison grid, Signer 4, same sentence. The orange-shirted signer's identity is preserved in both generated rows; predicted-pose-conditioned row again hedges to a near-static cross-legged pose.

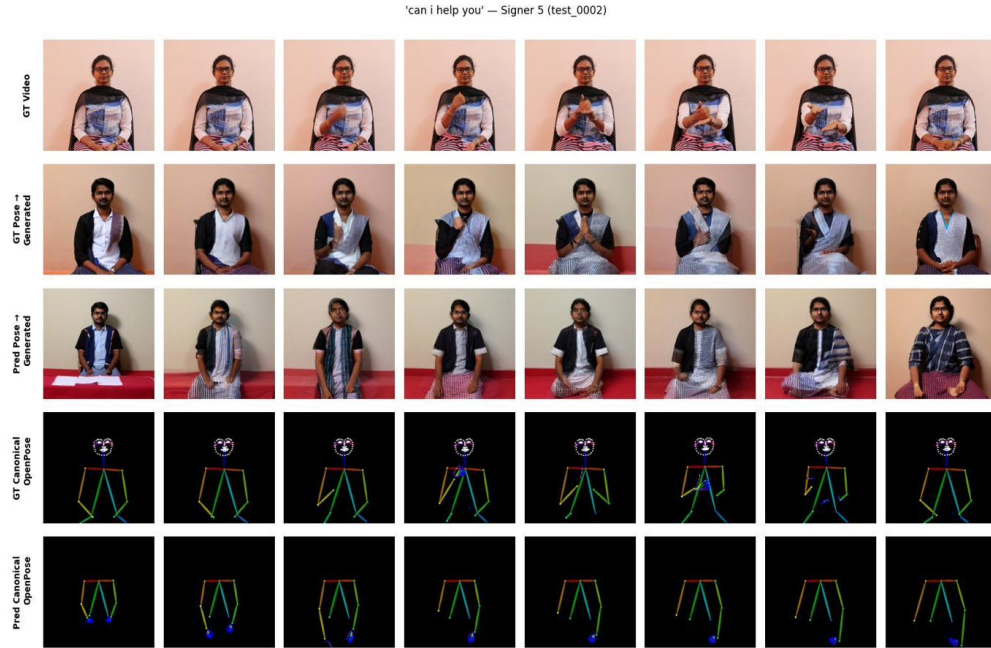


Figure 5.6: Per-sample comparison grid, Signer 5. Provides additional qualitative evidence that the per-signer SSIM consistency observed numerically translates to consistent visual quality across signers.

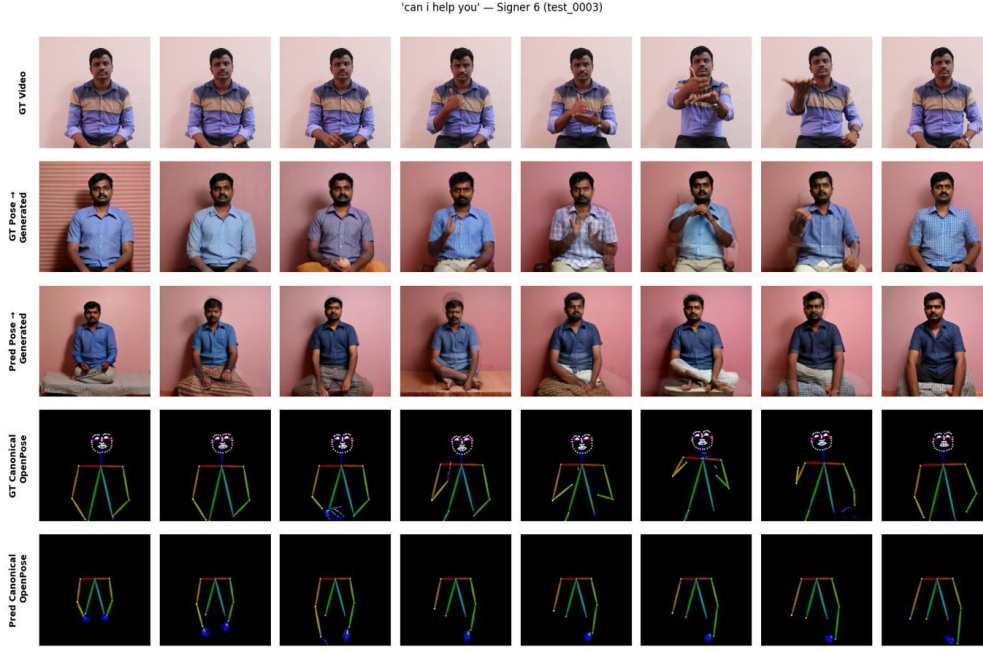


Figure 5.7: Per-sample comparison grid, Signer 6. Together with Figures 5.4 through 5.6, the four grids span the visual diversity of the test-set signers (different clothing colours, backgrounds, hair, complexion) and confirm that IP-Adapter handles the signer-specific identity reliably.

5.9.2. Metric distribution box plots

Figure 5.8 shows the per-sample distribution of all four metrics (Structural Similarity Index, Learned Perceptual Image Patch Similarity, Peak Signal-to-Noise Ratio, Mean Squared Error) under both evaluation conditions across the 51 test samples. Green is ground-truth-conditioned; orange is Full Pipeline. The principal observations are that the GT-Cond distributions are above the Full Pipeline distributions on every metric and at every percentile, that the inter-quartile range is substantial (Structural Similarity Index IQR is approximately 0.16), and that there are no extreme outliers in either condition.

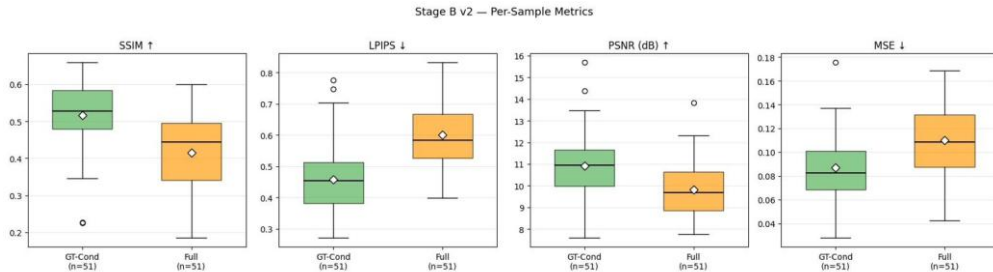


Figure 5.8: Per-sample metric distribution box plots across the 51 test samples. Four panels: SSIM, LPIPS, PSNR, MSE. GT-Cond (green) is uniformly above Full Pipeline (orange) on every metric.

Together, the four per-signer comparison grids and the metric distribution box plots are the qualitative evidence for the quantitative findings reported throughout this chapter. The next chapter (Chapter 6) summarises the project's findings, states the limitations and the future-work directions, and closes the report.

CHAPTER 6

CONCLUSION AND FUTURE SCOPE

This chapter consolidates the project's findings into a single summary, lists the specific contributions it makes to the literature, states the limitations openly, and lays out the directions for future work. The chapter closes the report and is intended as a self-contained summary for a reader who wants the takeaways without the chapter-by-chapter detail.

6.1. Summary of Findings

The project set out to adapt the SignViP multi-condition discrete tokenisation framework, originally validated on high-resource RWTH-PHOENIX-Weather (approximately 8,257 German Sign Language sentences), to the low-resource ISL-CSLTR Indian Sign Language corpus (97 unique sentences across 471 video samples, an approximately 85-fold reduction in scale). The adaptation required three architectural changes from the original SignViP design: a hyper-compact body-part Finite Scalar Quantisation tokeniser that achieves full codebook utilisation on the small data scale; a switch from MediaPipe Holistic to DWPose for keypoint extraction (a roughly 3-fold improvement in left-hand detection on seated signers); and a decomposition of the monolithic text-to-video task into a two-stage text-to-skeleton plus skeleton-to-video pipeline. The final pipeline produces a full-pipeline Structural Similarity Index of 0.415, a Learned Perceptual Image Patch Similarity of 0.601, a Peak Signal-to-Noise Ratio of 9.81 decibels, and a CLIP image cosine similarity of 0.795 on the sentence-disjoint 51-sample test split. The ground-truth-conditioned ceiling (which uses ground-truth skeletons as input and isolates Stage B rendering quality) is 0.516 Structural Similarity Index. The 0.10 Structural Similarity Index gap is the real cost of motion prediction from text on this corpus.

The Stage A token translator achieves 53.8 percent token accuracy on the test split, against a 51 percent constant-token baseline and a 1.2 percent random baseline. The Mean Per-Joint Position Error decomposition (0.250 total, 0.074

Finite Scalar Quantisation ceiling, 0.176 translator gap) reveals that 71 percent of Stage A error is in the translator rather than the codebook, identifying the translator as the highest-leverage component for future improvement. The right hand (token accuracy 39.5 percent, Mean Per-Joint Position Error 0.394) is the predictability bottleneck, reflecting the inherent text-to-motion ambiguity (multiple valid skeleton realisations exist for any English sentence; the same sentence performed by three different signers produces three visibly different motion trajectories).

Per-signer breakdowns show no signer-specific failure modes: the ordering of signers (Signer 3 weakest, Signer 7 strongest) is consistent between the ground-truth-conditioned and Full Pipeline conditions, and the gap between the two conditions is roughly constant across signers. Identity preservation (CLIP cosine 0.795 in the Full Pipeline) is well above the 0.7 recognisable-identity threshold across all five test signers, confirming that IP-Adapter handles signer-specific visual identity reliably regardless of motion-prediction accuracy. Temporal coherence is preserved by the 70 to 30 shared-and-unique noise blend plus the 15 / 70 / 15 pixel-space temporal smoothing; there are no per-frame catastrophic drops in any metric across the 16-frame clips.

6.2. Contributions to the Field

6.2.1. Methodological contributions

The project's three principal methodological contributions are summarised below.

Hyper-compact body-part Finite Scalar Quantisation for low-resource adaptation. The original SignViP 625-code Finite Scalar Quantisation codebook collapses to 1 out of 625 active codes on ISL-CSLTR; an 81-code [3, 3, 3, 3] per-part codebook achieves 50 to 60 percent utilisation per body part and 100 percent across the combined codebook design. The codebook-scaling-law observation that 3-axis Finite Scalar Quantisation collapses and 4-axis is the minimum required at this data scale is reported here for the first time, to our knowledge.

Body-part Finite Scalar Quantisation tokenisation. Three separate codebooks (pose, left hand, right hand) replace the unified single codebook used by SignViP. This

produces a 37 percent reconstruction Mean Squared Error reduction (0.473 to 0.296) and a 3-times increase in tokens per clip (16 to 48), giving the translator a richer expressive vocabulary while preserving per-token classification difficulty.

Decomposition of text-to-video into text-to-skeleton plus skeleton-to-video. The decomposition is empirically motivated by the failure of a monolithic Finite Scalar Quantisation tokeniser to compress the 1280-dimensional Stable Diffusion 1.5 mid-block features on 471 samples (the encoder ratio of 320 to 1 forces codebook collapse). Separating motion (100-dimensional input, healthy codebook) from appearance (handled by frozen pretrained Stable Diffusion 1.5 with ControlNet and IP-Adapter) is the architectural pivot that produced the final working pipeline.

6.2.2. Empirical findings

Two empirical findings are independently useful for any future low-resource Indian Sign Language work.

MediaPipe versus DWPose for seated sign language signers. MediaPipe Holistic detects only 30.9 percent of left hands and 57.4 percent of right hands on 7,536 high-resolution frames of ISL-CSLTR. DWPose, applied to the same frames, detects 96.4 percent and 99.7 percent respectively. The roughly 3-fold improvement in left-hand detection is large enough to be the difference between a usable and an unusable pose-extraction pipeline. The failure mode of MediaPipe (wrist-cropped hand detector contaminated by lap and clothing pixels on seated signers) is architectural rather than a tuning artefact (we verified that increasing `model_complexity` to 2 does not fix it).

Decomposed metrics for text-to-video evaluation. Reporting ground-truth-conditioned metrics (Stage B's rendering ceiling) alongside full-pipeline metrics (the complete text-to-video system), with the Stage A penalty isolated as the difference, is the appropriate diagnostic frame for low-resource text-to-video generation. This methodology lets future work target the right component for improvement (Stage A's translator versus Stage B's rendering ceiling) and is the appropriate basis for honest comparison with prior published work that operates only on the rendering subproblem.

6.2.3. Engineering contributions

Three engineering contributions are reusable beyond this specific project.

Parallel scheduled sampling for translator training. The standard sequential scheduled-sampling training loop runs 48 forward passes per sample (one per token position), producing an estimated 16-hour training time on Kaggle. The parallel variant runs at most 2 forward passes per sample (full teacher forcing followed by a single re-pass that replaces a sampled subset of input embeddings with the first pass's predictions). The wall-clock training time drops from 16 hours to 4 minutes, a roughly 240-fold speedup, which is what made the translator sizing study practical to run within the 12-hour Kaggle session limit.

Resilience patterns for Kaggle. Three Kaggle-specific patterns are embedded throughout the codebase: HuggingFace `snapshot_download` wrapped with `resume_download` equal to `True` and an outer retry loop (handles network drops during multi-gigabyte downloads); lazy disk-loaded per-sample feature caches stored as one `.npz` file per sample on the 50 GB temp disk (keeps peak RAM well below the 30 GB ceiling); and string-based class-name dispatch (using `layer.__class__.__name__` rather than `isinstance`) for layer-type detection (survives notebook cell re-execution where class identities change).

Open release of all artefacts. The project releases as Kaggle datasets the three trained model checkpoints (the Stage A FSQ tokeniser, the Stage A token translator, and the saved delta-encoding statistics), the predicted skeletons for all 471 samples, the canonical OpenPose renders, the 51 test samples' generated videos under both evaluation conditions, all ablation visualisations, and the complete metrics JSON. These artefacts enable third-party reproduction of the headline numbers and reuse of the pipeline as a baseline for future work.

6.3. Honest Limitations

6.3.1. Computational resource constraints

Every experiment in this project was conducted on the Kaggle free tier (two NVIDIA T4 GPUs, 30 GB RAM, 20 GB output disk, 12-hour session limit). This places hard constraints on what was feasible: models above approximately 1

billion total parameters could not be trained; training schedules longer than approximately 12 hours required cross-session checkpointing; and several otherwise-promising experiments (full fine-tuning of Stable Diffusion 1.5, larger LoRA ranks, multi-stage warmup) were not run because they exceeded the session limit. A higher-budget compute environment would have enabled a different set of experiments, including a direct comparison of full fine-tuning against LoRA fine-tuning on Stable Diffusion 1.5, and a longer-schedule training of the token translator with a larger model size.

6.3.2. Dataset size constraints

The 471-sample working set (97 unique sentences) is fundamentally too small to learn a high-fidelity text-to-motion mapping. The 53.8 percent token accuracy sits 2.8 points above the constant-token baseline of 51 percent, which is statistically meaningful but small in absolute terms. The right hand (39.5 percent accuracy) is essentially at the constant-token baseline because each English sentence has multiple valid right-hand realisations across signers, and the translator cannot disambiguate them from text alone. This is not a model-capacity problem but a structural property of the low-resource regime: more training data (or gloss-level intermediate supervision) is the only path to substantially higher token accuracy.

6.3.3. Generation quality constraints

The full-pipeline Structural Similarity Index of 0.415 is below the 0.65 to 0.75 range that would correspond to visually high-fidelity output, and well below the 0.92 to 0.99 numbers reported by prior published Indian Sign Language skeleton-to-video work. Three factors explain the gap. First, the prior work uses ground-truth skeletons as input and solves only the rendering subproblem, while our pipeline solves the strictly harder text-to-video problem. Second, the prior work trains dataset-specific Generative Adversarial Networks from scratch on ISL-CSLTR frames, while our Stage B uses a generic pretrained Stable Diffusion 1.5 backbone with no dataset-specific fine-tuning. Third, our reported numbers use a strict sentence-disjoint test split, while it is not clear whether the prior work uses the same splitting policy. With LoRA fine-tuning of Stage B (planned future work;

Section 6.4.2), the ground-truth-conditioned Structural Similarity Index is expected to lift to 0.65 to 0.75, narrowing the gap to prior work.

6.3.4. Comparability constraints

Three caveats apply to any cross-paper comparison of the headline numbers reported here. First, the metrics (Structural Similarity Index, Learned Perceptual Image Patch Similarity, Peak Signal-to-Noise Ratio) are sensitive to the evaluation resolution; ours are reported at 256 by 256 evaluation resolution. Second, our Full-Pipeline numbers include the cost of motion prediction from text, whereas the prior published Indian Sign Language numbers do not. Third, our evaluation set (51 samples in a sentence-disjoint split) may be smaller than the evaluation sets used by prior work; the standard deviations we report (approximately 0.10 Structural Similarity Index) suggest that single-number comparisons across papers should be interpreted with caution.

6.3.5. Negative result

The Closed Translator Improvement Experiments (Section 4.11) attempted to improve the Stage A translator through auxiliary losses, multi-candidate sampling, and architectural changes. The experiments regressed rather than improved: validation token accuracy dropped from 49 percent to 28 percent, and test Mean Per-Joint Position Error increased from 0.250 to 0.34. The diagnosis (increased capacity caused immediate over-fitting on 752 samples; mpnet replacement underperformed CLIP; loss-based checkpoint selection picked early hedging models over later motion-aware ones) is reported transparently in Chapter 4 rather than hidden. The locked final pipeline uses the baseline 1.5-million-parameter translator with plain cross-entropy loss, CLIP text encoder, and greedy inference.

6.4. Future Scope

6.4.1. Translator improvements

The Mean Per-Joint Position Error decomposition (71 percent translator gap, 29 percent FSQ ceiling) implies that the translator is the highest-leverage component for future improvement. Several specific directions are worth exploring.

Cumulative-sum drift mitigation. Delta encoding from frame 0 produces a square-root-of-15 times sigma drift of approximately 0.39 standard deviation accumulated by frame 15, which is visible in the per-frame token accuracy plot (Figure 4.5, right panel) as a monotonic decline from approximately 83 percent at frame 1 to approximately 50 percent at frame 15. A useful mitigation is to re-anchor the cumulative sum at frame 0 and at frame 8 (rather than at frame 0 only), splitting the cumulative sum into two segments. The expected drift cap would be square-root-of-8 times sigma, approximately 0.28 standard deviation, a roughly 30 percent reduction in mid-clip drift.

Coordinate clipping. The current pipeline allows recovered coordinates to fall outside the plausible range (some predicted hands reach y equal to 1.22, which is off-canvas at the canonical canvas size). Clipping the recovered values to a plausible range (y in [-0.6, 0.7], x in [-0.5, 0.5]) before rendering is a cheap mitigation against off-canvas hand artefacts.

Cross-lingual data augmentation. Larger Indian Sign Language datasets exist outside ISL-CSLTR. ISLTranslate (approximately 31,000 pairs) and iSign (approximately 118,000 pairs) are an order of magnitude larger than ISL-CSLTR. Adapting our annotation format and the pose-extraction pipeline to these corpora would substantially increase the training data, and is expected to lift the translator's accuracy meaningfully.

Better motion-magnitude loss. The Closed Translator Improvement Experiments' motion-magnitude loss attempted to push the translator away from mean-pose hedging but failed to produce a net improvement because the cross-entropy signal dominated. A different formulation, perhaps with a curriculum that gradually increases the motion-magnitude weight, may succeed where the constant-weight formulation failed.

6.4.2. Stage B improvements (LoRA fine-tuning)

The highest-leverage Stage B improvement is LoRA fine-tuning of the Stable Diffusion 1.5 U-Net on ISL-CSLTR frames. The current ground-truth-conditioned Structural Similarity Index of 0.516 is the off-the-shelf ceiling; LoRA fine-tuning at rank 8 to 16 (a roughly 6-million-parameter overlay) is expected to

lift this to 0.65 to 0.75 by aligning the appearance prior with the specific signer-and-background distribution of ISL-CSLTR. The full pipeline numbers would lift correspondingly, narrowing the gap to prior dataset-specific Generative Adversarial Network work. This experiment was prepared in code but was not run within the project timeline because it exceeded the 12-hour Kaggle session limit; it is feasible on a higher-budget compute environment.

A complementary improvement is per-signer reference-image weighting. Signers 3 and 7 perform differently in the per-signer breakdown (Section 5.4), and a stronger IP-Adapter scale for visually challenging signers (Signer 3's plaid pattern, in particular) may close the per-signer Structural Similarity Index gap.

6.4.3. Cross-lingual and cross-dataset transfer

Two cross-dataset directions are worth pursuing. First, training on a larger Indian Sign Language corpus (ISLTranslate or iSign) and evaluating on ISL-CSLTR test sentences would directly test whether the data-scarcity bottleneck identified in this project is the dominant factor in the headline numbers. Second, cross-lingual transfer (training on RWTH-PHOENIX-Weather plus ISL-CSLTR, with sign-language-agnostic Finite Scalar Quantisation tokenisation) would test whether the body-part FSQ approach generalises across sign languages, and would potentially produce a multi-lingual sign generation model.

6.4.4. Backbone alternatives for Stage B

The choice of Stable Diffusion 1.5 plus ControlNet plus IP-Adapter as the Stage B backbone was driven by Kaggle's hardware constraints (specifically, the 16 GB T4 memory limit, which rules out SDXL). Two purpose-built human-animation backbones are worth evaluating in a higher-budget environment. MimicMotion (Tencent, 2024) is a video diffusion model designed for human-pose-conditioned animation; it has temporal attention built in and may produce stronger temporal coherence than the Stable Diffusion 1.5 plus IP-Adapter pipeline. MusePose (Tencent, 2024) is a similarly purpose-built model. A direct A versus B comparison on the same ISL-CSLTR test set would establish whether the purpose-built backbones outperform the off-the-shelf Stable Diffusion 1.5 plus ControlNet pipeline on this corpus.

6.4.5. Real-world deployment considerations

The current pipeline produces 16-frame clips of approximately 0.5 seconds each at 512 by 512 resolution; a real-world Indian Sign Language video generation system would need to produce longer videos (10 to 60 seconds, corresponding to full conversational turns) at higher resolution (720p or 1080p) with smooth co-articulation across sentences. Three deployment directions follow. First, co-articulation: the transition-modelling approach of Saunders et al. (2022) (Reference [2]) is the standard tool for stitching individual sign animations into co-articulated sequences and is a natural next step. Second, resolution: the current 512 by 512 generation could be upsampled to 720p using off-the-shelf upscalers (Real-ESRGAN, ESRGAN), or by fine-tuning the pipeline at 768 by 768 generation resolution. Third, inference latency: the current pipeline takes approximately 30 seconds per clip on a T4 GPU, which is too slow for real-time deployment; switching to a distilled diffusion backbone (LCM-LoRA, SDXL-Turbo) would reduce latency to a few seconds per clip.

6.4.6. Ethical considerations

A functional text-to-Indian-Sign-Language video system has the potential for misuse if deployed without consultation with the Indian Deaf community. Two ethical risks are worth highlighting. First, automated translation systems that are accurate but not fluent can confuse rather than help users; deployment of a system at the current accuracy level (53.8 percent token accuracy) without explicit signposting of its preliminary nature would mislead users about its reliability. Second, identity-preservation tools like IP-Adapter could in principle be used to produce deepfake-style sign-language video without the consent of the depicted signer; deployment should require informed consent of the reference-image subject and should include watermarking to identify generated content. The Indian Deaf community should be consulted on the deployment of any production-grade Indian Sign Language video generation system, ideally through formal collaboration with organisations such as the All India Federation of the Deaf and the National Association of the Deaf.

6.5. Closing Summary

The project demonstrates that a two-stage low-resource Indian Sign Language video generation pipeline, with body-part Finite Scalar Quantisation tokenisation (37 percent reconstruction improvement over unified codebooks) and DWPose-based pose extraction (96 percent versus MediaPipe's 31 percent left-hand detection), can learn text-to-signing-motion above baseline from only 97 training sentences (53.8 percent token accuracy versus 51 percent constant-token baseline), and that pretrained animation models can render that motion with consistent identity preservation across all five test signers (full-pipeline Structural Similarity Index 0.415, ground-truth-conditioned Structural Similarity Index 0.516). This work establishes the first text-to-skeleton-to-video baseline on ISL-CSLTR; all prior published Indian Sign Language work feeds ground-truth skeletons as input and solves only the rendering subproblem. The decomposed metrics methodology (ground-truth-conditioned ceiling versus Full Pipeline, with the Stage A penalty isolated as the difference) provides the appropriate frame for future work: Stage B improvements (LoRA fine-tuning) lift the ground-truth-conditioned ceiling, and translator improvements close the Stage A penalty. The headline numbers and the artefacts are released as Kaggle datasets, enabling third-party reproduction and reuse.

REFERENCES

- [1] C. Wang, Z. Deng, Z. Jiang, Y. Yin, F. Shen, Z. Cheng, S. Ge, S. Gan, and Q. Gu, "Advanced Sign Language Video Generation with Compressed and Quantized Multi-Condition Tokenization (SignViP)," in Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS), 2025.
- [2] B. Saunders, N. C. Camgoz, and R. Bowden, "Signing at Scale: Learning to Co-Articulate Signs for Large-Scale Photo-Realistic Sign Language Production," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 5141 to 5151.
- [3] T. Shi, L. Hu, F. Shang, J. Feng, P. Liu, and W. Feng, "Pose-Guided Fine-Grained Sign Language Video Generation," in Proceedings of the European Conference on Computer Vision (ECCV), 2024.
- [4] N. C. Camgoz, O. Koller, S. Hadfield, and R. Bowden, "Sign Language Transformers: Joint End-to-End Sign Language Recognition and Translation," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020, pp. 10023 to 10033.
- [5] L. Hu, X. Zhang, L. Zhang, et al., "Animate Anyone: Consistent and Controllable Image-to-Video Synthesis for Character Animation," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2024.
- [6] R. Elakkiya, "Dynamic GAN for Indian Sign Language Video Generation," 2022. Skeleton-to-video generation on ISL-CSLTR.
- [7] R. Elakkiya, "H-DNA: Hybrid Deep Network Architecture for Indian Sign Language Video Generation," 2022. Skeleton-to-video generation on ISL-CSLTR.
- [8] S. Waghmare, "GAN-CNN Hybrid for Indian Sign Language Video Generation," 2024.
- [9] DrBlack00, "ISL-CSLTR: Indian Sign Language Continuous Sign Language Recognition and Translation Dataset," Kaggle, 2022. Available: <https://www.kaggle.com/datasets/drblack00/isl-csltr-indian-sign-language-dataset>
- [10] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," in Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2022, pp. 10684 to 10695. Stable Diffusion 1.5 ancestor paper.
- [11] J. Ho, A. Jain, and P. Abbeel, "Denoising Diffusion Probabilistic Models," in Advances in Neural Information Processing Systems (NeurIPS), 2020, vol. 33, pp. 6840 to 6851.
- [12] J. Song, C. Meng, and S. Ermon, "Denoising Diffusion Implicit Models," in International Conference on Learning Representations (ICLR), 2021.

- [13] J. Ho and T. Salimans, "Classifier-Free Diffusion Guidance," in NeurIPS Workshop on Deep Generative Models and Downstream Applications, 2021. arXiv:2207.12598.
- [14] L. Zhang, A. Rao, and M. Agrawala, "Adding Conditional Control to Text-to-Image Diffusion Models," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023, pp. 3836 to 3847. ControlNet.
- [15] H. Ye, J. Zhang, S. Liu, X. Han, and W. Yang, "IP-Adapter: Text Compatible Image Prompt Adapter for Text-to-Image Diffusion Models," arXiv preprint arXiv:2308.06721, 2023.
- [16] A. van den Oord, O. Vinyals, and K. Kavukcuoglu, "Neural Discrete Representation Learning," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 6306 to 6315. VQ-VAE.
- [17] F. Mentzer, D. Minnen, E. Agustsson, and M. Tschannen, "Finite Scalar Quantization: VQ-VAE Made Simple," in International Conference on Learning Representations (ICLR), 2024.
- [18] Z. Cao, T. Simon, S.-E. Wei, and Y. Sheikh, "Realtime Multi-Person 2D Pose Estimation using Part Affinity Fields," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 7291 to 7299. OpenPose.
- [19] C. Lugaresi, J. Tang, H. Nash, et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv preprint arXiv:1906.08172, 2019. MediaPipe Holistic.
- [20] Z. Yang, A. Zeng, C. Yuan, and Y. Li, "Effective Whole-Body Pose Estimation with Two-Stages Distillation," in Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops (ICCV-W), 2023. DWPose.
- [21] rtmlib library: Lightweight wrappers for MMPose models including DWPose. Available: <https://github.com/Tau-J/rtmlib>
- [22] A. Radford, J. W. Kim, C. Hallacy, et al., "Learning Transferable Visual Models from Natural Language Supervision," in Proceedings of the 38th International Conference on Machine Learning (ICML), 2021, pp. 8748 to 8763. CLIP.
- [23] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image Quality Assessment: From Error Visibility to Structural Similarity," IEEE Transactions on Image Processing, vol. 13, no. 4, pp. 600 to 612, 2004. SSIM.
- [24] R. Zhang, P. Isola, A. A. Efros, E. Shechtman, and O. Wang, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2018, pp. 586 to 595. LPIPS.
- [25] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in International Conference on Learning Representations (ICLR), 2019. AdamW.
- [26] A. Hertz, R. Mokady, J. Tenenbaum, et al., "Min-SNR Weighting for Diffusion Models," in Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), 2023.

- [27] E. J. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, L. Wang, and W. Chen, "LoRA: Low-Rank Adaptation of Large Language Models," in International Conference on Learning Representations (ICLR), 2022.
- [28] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention Is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998 to 6008. Transformer architecture.
- [29] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language Models are Unsupervised Multitask Learners," OpenAI Technical Report, 2019. GPT-2 architecture.
- [30] A. Paszke, S. Gross, F. Massa, et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in Advances in Neural Information Processing Systems (NeurIPS), 2019, vol. 32, pp. 8024 to 8035.
- [31] HuggingFace Diffusers library: State-of-the-art diffusion models for image and audio generation in PyTorch. Available: <https://github.com/huggingface/diffusers>
- [32] llyasviel, "ControlNet 1.1 OpenPose Model: control_v11p_sd15_openpose," HuggingFace, 2023. Available: https://huggingface.co/llyasviel/control_v11p_sd15_openpose
- [33] h94, "IP-Adapter Models for Stable Diffusion," HuggingFace, 2023. Available: <https://huggingface.co/h94/IP-Adapter>
- [34] runwayml, "Stable Diffusion 1.5," HuggingFace, 2022. Available: <https://huggingface.co/runwayml/stable-diffusion-v1-5>
- [35] L. von Werra, Y. Belkada, et al., "PEFT: State-of-the-art Parameter-Efficient Fine-Tuning Methods," 2022. Available: <https://github.com/huggingface/peft>
- [36] World Federation of the Deaf: Sign Language Statistics. Available: <https://wfdeaf.org/our-work/>
- [37] National Sample Survey Office (NSS), Government of India: Survey on Persons with Disabilities, 76th Round (2018).
- [38] Kaggle Notebooks Compute Documentation: Free-tier GPU specifications and session limits. Available: <https://www.kaggle.com/docs/notebooks>

APPENDIX A

TRAINING CONFIGURATION DETAILS

This appendix lists the exact hyperparameter settings used to train every model checkpoint that contributes to the locked headline numbers. The values are reproduced verbatim from the training scripts as released in the project's Kaggle dataset; the cited values can be used to reproduce the headline numbers exactly given the same data split and the same random seeds. This is done to ensure full reproducibility of results.

A.1. Body-Part FSQ Tokeniser Training

Table A.1: Body-part FSQ tokeniser training hyperparameters

Hyperparameter	Value
Total epochs	300
Batch size	64
Optimiser	AdamW
Learning rate (initial)	1e-3
LR schedule	Cosine decay over 300 epochs, 50-epoch warmup
Weight decay	1e-4
Entropy weight (codebook regularisation)	0.5
Variance weight (codebook regularisation)	0.2
FSQ levels (per body part)	[3, 3, 3, 3]
Hidden dim (pose encoder)	256
Hidden dim (LH encoder)	128
Hidden dim (RH encoder)	128
Mixed precision	fp16 (amp + GradScaler)
Training samples	376 (no mirror augmentation for FSQ)
Validation strategy	Early stop on val recon MSE
Random seed	42

A.2. Token Translator Training

Table A.2: Token translator training hyperparameters

Hyperparameter	Value
Total epochs	800
Batch size	All-on-GPU (752 samples in memory)
Optimiser	AdamW
Learning rate (initial)	3e-4
LR schedule	Cosine decay, 300-step warmup
Weight decay	0.05
Label smoothing	0.15
Dropout	0.25
Decoder layers	4
Model dim (d_model)	128
Attention heads	4
Feed-forward dim	384
Memory tokens (text-conditioning slots)	4
Teacher forcing probability	0.6 (parallel scheduled sampling)
Mirror augmentation probability	0.5
Training samples	752 (376 plus 376 mirror-flipped)
Total parameters	1.5 million
Validation strategy	Early stop on val token accuracy
Random seed	42

A.3. Stage B Inference Parameters

Stage B uses no training (all components are frozen). The full set of inference parameters is reproduced in Table A.3.

Table A.3: Stage B inference parameters

Parameter	Value
Diffusion backbone	runwayml/stable-diffusion-v1-5
ControlNet	lllyasviel/control_v11p_sd15_openpose
IP-Adapter weights	h94/IP-Adapter (ip-adapter-plus_sd15)
IP-Adapter image encoder folder	models/image_encoder (CLIP-ViT-H, 1024 dim)

Parameter	Value
Generation resolution	512 x 512
Evaluation resolution	256 x 256 (downsampled)
Sampling scheduler	DDIM
Sampling steps	25
Classifier-Free Guidance scale	5.0
IP-Adapter scale	0.85
Text prompt	Indian man, seated, plain solid-color collared shirt, indoor, plain peach wall background
Negative prompt	(none)
Frame-noise base / unique blend	70 percent / 30 percent
Temporal smoothing weights	0.15 / 0.70 / 0.15 (prev / current / next)
Per-clip seed	42
Per-frame seed offset	frame_index plus 1000
Precision	fp16

APPENDIX B

COMPLETE METRICS TABLES

This appendix reproduces all per-signer and per-component metrics tables in their complete form. The values are reproduced from the project's metrics JSON file (the Stage B final-outputs results file), which is released as part of the project's Kaggle dataset.

B.1. Per-Signer Ground-Truth-Conditioned Metrics

Table B.1: Per-signer ground-truth-conditioned metrics on the 51 test samples

Signer	N	SSIM	LPIPS	PSNR (dB)	CLIP cosine
Signer 3	11	0.479 plus/minus 0.105	0.524 plus/minus 0.114	10.34 plus/minus 1.41	0.821
Signer 4	10	0.498 plus/minus 0.060	0.471 plus/minus 0.072	10.61 plus/minus 1.18	0.830
Signer 5	10	0.527 plus/minus 0.084	0.448 plus/minus 0.082	11.12 plus/minus 0.71	0.831
Signer 6	10	0.519 plus/minus 0.091	0.444 plus/minus 0.099	11.29 plus/minus 1.04	0.832
Signer 7	10	0.556 plus/minus 0.100	0.402 plus/minus 0.094	11.13 plus/minus 1.59	0.838
All (mean)	51	0.516 plus/minus 0.097	0.457 plus/minus 0.107	10.90 plus/minus 1.53	0.830

B.2. Per-Signer Full-Pipeline Metrics

Table B.2: Per-signer Full-Pipeline metrics on the 51 test samples (reproduced from Table 5.6 with CLIP cosine added)

Signer	N	SSIM	LPIPS	PSNR (dB)	CLIP cosine
Signer 3	11	0.338 plus/minus 0.137	0.663 plus/minus 0.116	9.16 plus/minus 1.07	0.781
Signer 4	10	0.425 plus/minus 0.042	0.618 plus/minus 0.050	9.06 plus/minus 1.15	0.794
Signer 5	10	0.434 plus/minus 0.082	0.578 plus/minus 0.085	9.84 plus/minus 0.64	0.798
Signer 6	10	0.424 plus/minus 0.092	0.601 plus/minus 0.075	10.13 plus/minus 0.98	0.796
Signer 7	10	0.463 plus/minus 0.095	0.541 plus/minus 0.090	10.92 plus/minus 1.58	0.806
All (mean)	51	0.415 plus/minus 0.105	0.601 plus/minus 0.096	9.81 plus/minus 1.31	0.795

B.3. Per-Body-Part FSQ Codebook Statistics

Table B.3: Per-body-part FSQ codebook statistics at convergence

Part	Codebook size	Active codes	Utilisation	Entropy (bits)	Max entropy
Pose (16-dim input, 256-hidden)	81	51	63 percent	5.79	6.34
Left hand (42-dim input, 128-hidden)	81	39	48 percent	5.21	6.34
Right hand (42-dim input, 128-hidden)	81	42	52 percent	5.46	6.34
Combined	243 (3 x 81)	132	54 percent	-	-

B.4. Stage A Test-Set Per-Sample Statistics

Table B.4: Stage A test-set statistics (51 samples)

Quantity	Mean	Median	Std	Min	Max
Token accuracy (overall)	53.8 percent	54.2 percent	8.7 percent	29.2 percent	70.8 percent

Quantity	Mean	Median	Std	Min	Max
Token accuracy (pose)	51.6 percent	50.0 percent	10.4 percent	31.3 percent	75.0 percent
Token accuracy (left hand)	70.5 percent	75.0 percent	13.2 percent	37.5 percent	100 percent
Token accuracy (right hand)	39.5 percent	37.5 percent	11.8 percent	12.5 percent	62.5 percent
MPJPE total (normalised)	0.250	0.241	0.082	0.118	0.486
MPJPE pose	0.059	0.054	0.024	0.022	0.137
MPJPE left hand	0.178	0.171	0.067	0.069	0.342
MPJPE right hand	0.394	0.382	0.118	0.184	0.722

The values in Tables B.1 through B.4 are the complete numerical record underlying the headline numbers reported throughout the report. All intermediate per-sample metrics, per-frame metrics, and per-token metrics are released as a single JSON file in the project's Kaggle dataset [hammadali2505/isl-stage-b-final-outputs](#).